ARM PrimeCell

UART (PL011)

Design Manual

PL011 DDES 0000 D
ARM Confidential

ARM PrimeCell UART (PL011) Design Manual

Proprietary notice

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell UART

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	14th September, 2000	First release.
B	15th February, 2001	Version for REL1v1. The fractional Baud Rate divider has been corrected, and the Low Power IrDA counter does not count when the IrDA modes are disabled. Synchronisers have also been added for the UARTTXDMACLR and UARTRXDMACLR signals.
C	17th September, 2001	Version for REL1v2. Uart disable function error has been fixed. CharTxComp signal has been synchronised to the clock domain it is used in. An equation error in the design manual has been corrected.
D	17th December, 2001	Version for REL1v3, Repeated character transmission has been fixed. Transmit state machine and transmit FIFO control logic has been changed.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM DDI 0183	ARM PrimeCell UART (PL011) Technical Reference Manual
2	PL011 INTM 0000	ARM PrimeCell UART (PL011) Integration Manual
3	HP 5964-0245E	Hewlett-Packard IrDA Data Link Design Guide
4	IrDA Specification	SIR Physical Layer Link Specification Ver 1.1
5	ARM DUI 0006	AMBA Compliance Test Suite User Guide
6	ARM DDI 0138	Example AMBA System Technical Reference Manual
7	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ATPG	Automatic Test Pattern Generation

IrDA	Infrared Data Association
IP	Intellectual Property
RTL	Register Transfer Level
SIR	Serial infrared
STA	Static Timing Analysis
UART	Universal Asynchronous Receiver Transmitter

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell UART (Universal Asynchronous Receiver Transmitter). The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification are described in Section 6.

Section 4 presents an overview of the UART design, describing the functional partitioning of the UART and the signal interconnections. Section 5 presents detailed descriptions on the UART design with state machine diagrams. Section 6 describes the testbench and the test programs used to generate test vectors for functional verification of the UART.

3 INTRODUCTION

The UART is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

The UART is functionally similar to the standard 16C550 device. For further information on this block refer to the ARM PrimeCell UART (PL011) Technical Reference Manual [Ref. 1]. The ARM PrimeCell UART (PL011) Integration Manual [Ref. 2] describes how the block may be integrated into a typical industry standard ASIC design flow.

For details about implementing an infrared interface refer to the Hewlett-Packard IrDA Data Link Design Guide [Ref.3] or similar documentation by other infrared component vendors.

For further details about the infrared interface refer to the IrDA SIR Physical Layer Link Specification [Ref.4]

The AMBA Compliance Test Suite User Guide [Ref.5] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.6] and User Guide [Ref.7] describe an example microcontroller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of BusTalk, TICTalk test vectors for functional verification and integration testing.

4 UART DESIGN OVERVIEW

4.1 Functional Description

The ARM PrimeCell UART performs serial-to-parallel conversion on data received from a peripheral device through its UARTRXD input and parallel-to-serial conversion on data received from the APB interface. The transmit and receive paths are buffered with internal FIFOs allowing up to 16 bytes to be stored in both receive and transmit modes. The ARM PrimeCell UART includes a programmable baud-rate generator which generates transmit and receive timing reference signals from the main UART clock input, UARTCLK. A block diagram of the device is shown in Figure 1 ARM PrimeCell UART Block Diagram.

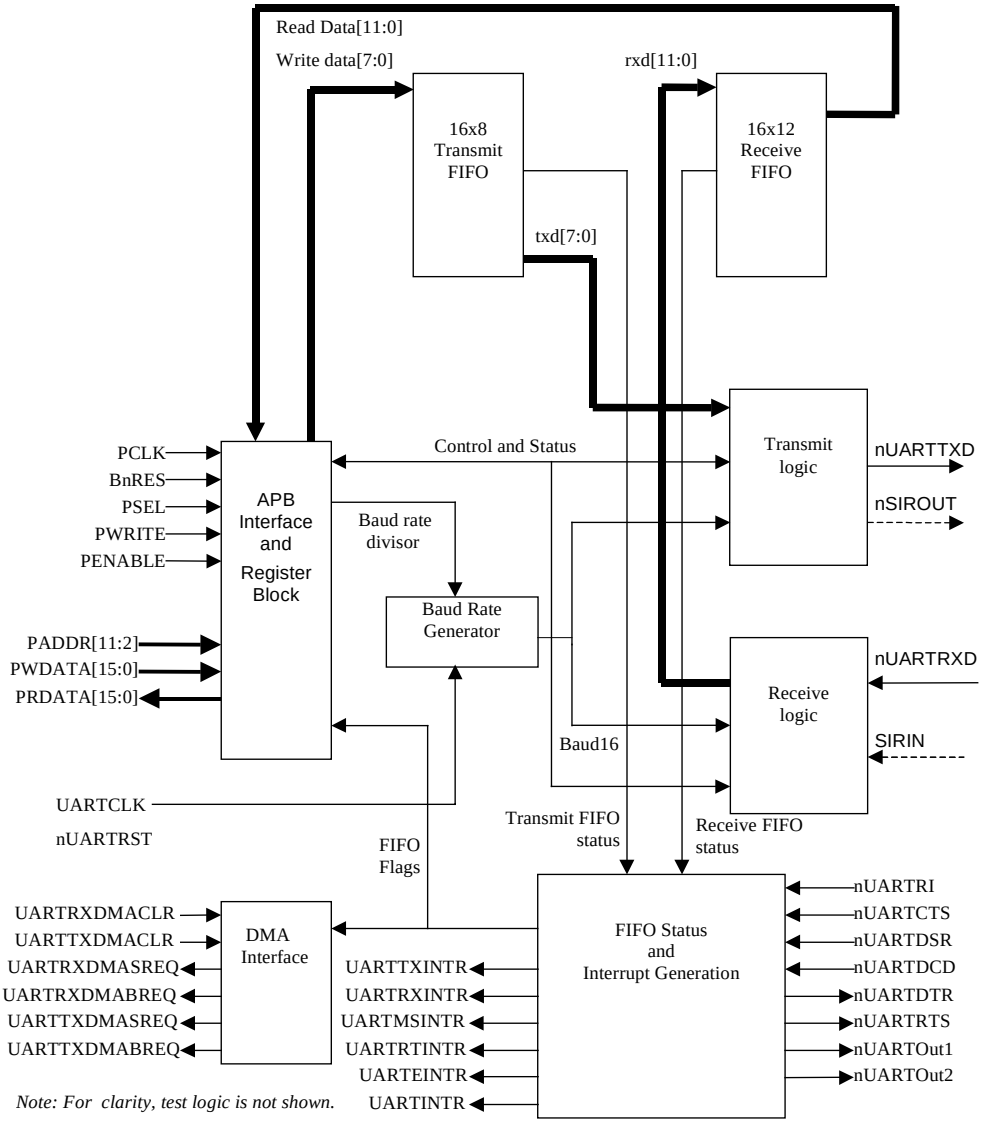

Figure 1 ARM PrimeCell UART Block Diagram

The ARM PrimeCell UART also contains an Infrared Data Association (IrDA) Serial Infrared (SIR) protocol encoder and decoder, also called an endec, outlined in Figure 2 IrDA SIR Endec Block Diagram. Optionally, this encoder can be used to encode the Tx and Rx signals. The encoded data stream can be used to drive an infrared interface directly. This interface also has a low power mode, in which, the width of the pulse in the encoded data stream is determined by an internally generated signal, IrLPBaud16. The frequency of the IrLPBaud16 signal, which governs the width of the fixed duration pulse, is controlled by the programmable divisor value accessed via the UARTILPR register

For further information on implementation of an IrDA SIR link refer to Hewlett-Packard IrDA Data Link Design Guide [Ref.3] and SIR Physical Layer Link Specification Ver 1.1 [Ref.4].

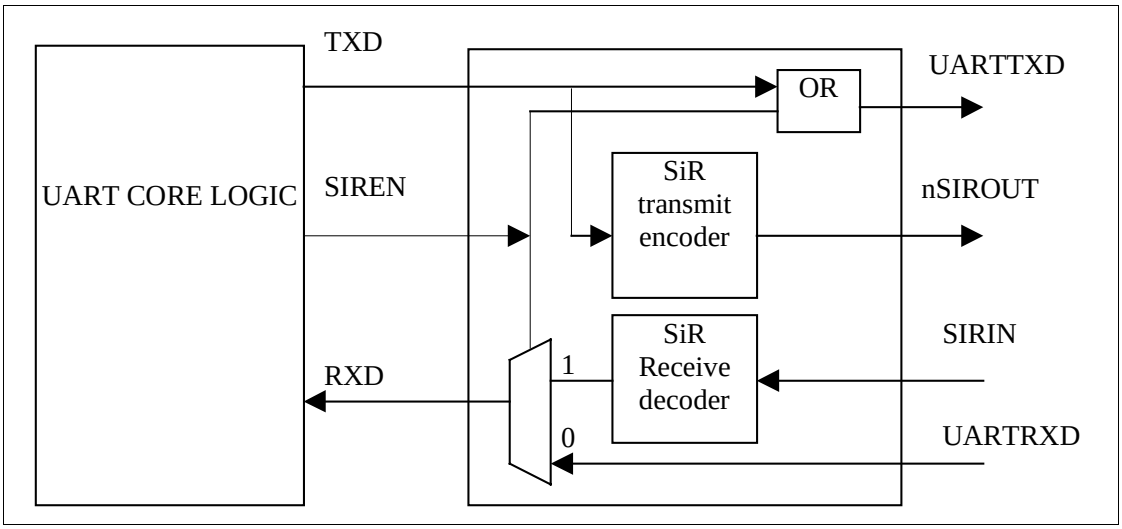

Figure 2 IrDA SIR Endec Block Diagram

4.2 Signal Description

The following tables summarise the UART signal list. Table 4.2-1 lists the APB interface signals while Table 4.2-2 lists the block specific non-APB signals.

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB Bus Clock
PRESETn	IN	Reset Controller	Bus reset (active low).
PENABLE	IN	APB Bridge	This signal is used to time all accesses on the peripheral bus.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the peripheral has been selected by the APB Bridge.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
PADDR[11:2]	IN	APB Bridge	This is part of the peripheral address bus, which is used for decoding the internal address space.
PWDATA[15:0]	IN	APB Bridge	Data is written into the peripheral on this bus.
PRDATA[15:0]	OUT	APB Bridge	Data is read out of the peripheral via this bus.

Table 4.2-1: APB signal descriptions

Signal Name	Type	Source / Destination	Description
UARTCLK	IN	Clock Generator	UART Reference clock
nUARTRST	IN	Reset Controller	UART reset signal to UARTCLK clock domain, active LOW. The reset controller must use PRESETn to assert nUARTRST asynchronously but negate it synchronously with UARTCLK.
UARTMSINTR	OUT	Interrupt Controller	UART Modem status interrupt (active HIGH)
UARTRTINTR	OUT	Interrupt Controller	UART Receive Timeout interrupt (active HIGH)
UARTRXINTR	OUT	Interrupt Controller	UART Receive FIFO interrupt (active HIGH)
UARTTXINTR	OUT	Interrupt Controller	UART Transmit FIFO interrupt (active HIGH)
UARTEINTR	OUT	Interrupt Controller	UART Error interrupt (active HIGH)
UARTINTR	OUT	Interrupt Controller	UART interrupt (active HIGH). A single combined interrupt generated as an OR function of the five individually maskable interrupts above.
UARTTXDMASREQ	OUT	DMA Controller	UART transmit DMA single request (active HIGH)
UARTRXDMASREQ	OUT	DMA Controller	UART receive DMA single request (active HIGH)
UARTTXDMABREQ	OUT	DMA Controller	UART transmit DMA burst request (active HIGH)
UARTRXDMABREQ	OUT	DMA Controller	UART receive DMA burst request (active HIGH)
UARTTXDMACLR	IN	DMA Controller	Transmit DMA request clear.
UARTRXDMACLR	IN	DMA Controller	Receive DMA request clear.
SCANENABLE	IN	Test Controller	UART scan enable signal for both clock domains

SCANINPCLK	IN	Test Controller	UART input scan signal for the PCLK domain
SCANINUCLK	IN	Test Controller	UART input scan signal for the UARTCLK domain
SCANOUTPCLK	OUT	Test Controller	UART output scan signal for the PCLK domain
SCANOUTUCLK	OUT	Test Controller	UART output scan signal for the UARTCLK domain

Table 4.2-2: Block Specific signal descriptions

Signal Name	Type	Source / Destination	Description
nUARTCTS	IN	PAD	UART Clear To Send modem status input, active LOW.
nUARTDCD	IN	PAD	UART Data Carrier Detect modem status input, active LOW.
nUARTDSR	IN	PAD	UART Data Set Ready modem status input, active LOW.
nUARTRI	IN	PAD	UART Ring Indicator modem status input, active LOW.
UARTRXD	IN	PAD	UART received serial data input.
SIRIN	IN	PAD	SIR received serial data input.
UARTTXD	OUT	PAD	UART transmitted serial data output, defaults to the marking state of 1 when reset.
nSIROUT	OUT	PAD	SIR transmitted serial data output, active LOW.
nUARTDTR	OUT	PAD	UART Data Terminal Ready modem status output, active LOW.
nUARTRTS	OUT	PAD	UART Request To Send modem status output, active LOW.
nUARTOut1	OUT	PAD	UART Out1 modem status output, active LOW.
nUARTOut2	OUT	PAD	UART Out2 modem status output, active LOW.

Table 4.2-3: I/O pad signal descriptions

4.3 Programmer's Model

4.3.1 Functional Mode Registers

Address	Type	Width	Reset Value	Name	Description
UartBase + 0x000	R/W	12/8	0x00	UARTDR	Data read or written from the interface. It is 12 bits wide on a read, and 8 on a write.
UartBase + 0x004	R/ W	4/ 0	0x00	UARTRSR/ UARTECR	Receive Status Register (Read) / Error Clear Register (Write)
UartBase + 0x008-0x014	-	-	-	-	Reserved
UartBase + 0x018	R	9	0b-10010---	UARTFR	Flag register (Read Only)
UartBase + 0x01C	-	-	-	-	Reserved
UartBase + 0x020	R/W	8	0x00	UARTILPR	IrDA low power counter register.
UartBase + 0x024	R/W	16	0x0000	UARTIBRD	Integer baud rate divisor register.
UartBase + 0x028	R/W	6	0x00	UARTFBRD	Fractional baud rate divisor register.
UartBase + 0x02C	R/W	8	0x00	UARTLCR_H	Line control register, HIGH byte.
UartBase + 0x030	R/W	16	0x0300	UARTCR	Control register.
UartBase + 0x034	R/W	6	0x12	UARTIFLS	Interrupt FIFO level select register.
UartBase + 0x038	R/W	11	0x000	UARTIMSC	Interrupt mask set/clear.
UartBase + 0x03C	R	11	0x00-	UARTRIS	Raw interrupt status.
UartBase + 0x040	R	11	0x00-	UARTMIS	Masked interrupt status.
UartBase + 0x044	W	11	-	UARTICR	Interrupt clear register.
UartBase + 0x048	R/W	3	0x00	UARTDMACR	DMA control register.
UartBase + 0x04C-07C	-	-	-	-	Reserved.

UartBase + 0x080-08C	-	-	-	-	Reserved (for test purposes).
UartBase + 0x090-FCC	-	-	-	-	Reserved.
UartBase + 0xFD0-FDC	-	-	-	-	Reserved for future ID expansion.
UartBase + 0xFE0	R	8	0x11	UARTPeriphID0	Peripheral Identification register bits 7:0.
UartBase + 0xFE4	R	8	0x10	UARTPeriphID1	Peripheral Identification register bits 15:8.
UartBase + 0xFE8	R	8	0x14	UARTPeriphID2	Peripheral Identification register bits 23:16.
UartBase + 0xFEC	R	8	0x00	UARTPeriphID3	Peripheral Identification register bits 31:24.
UartBase + 0xFF0	R	8	0x0D	UARTPCellID0	PrimeCell identification register bits 7:0.
UartBase + 0xFF4	R	8	0xF0	UARTPCellID1	PrimeCell identification register bits 15:8.
UartBase + 0xFF8	R	8	0x05	UARTPCellID2	PrimeCell identification register bits 23:16.
UartBase + 0xFFC	R	8	0xB1	UARTPCellID3	PrimeCell identification register bits 31:24.

Table 4.3-1 UART Register summary

Please see the ARM PrimeCell UART (PL011) Technical Reference Manual [Ref. 1] for a detailed description of each register.

4.3.2 Test Mode Registers

Address	Type	Width	Reset Value	Name	Description
UartBase + 0x80	R/W	3	0x0	UARTTCR	Test Control Register.
UartBase + 0x84	R/W	8	0x00	UARTITIP	Integration test input read/set register.
UartBase + 0x88	R/W	14	0x000	UARTITOP	Integration test output read/set register.
UartBase + 0x8C	R/W	11	0x---	UARTTDR	Test data register.

Table 4.3-2 Test Mode Registers

Please see ARM PrimeCell UART (PL011) Technical Reference manual [Ref.1] for a detailed description of each register.

4.3.3 Register Memory Map

Address	Read Location	Write Location
UART Base + 0x000	UARTDR	UARTDR
UART Base + 0x004	UARTRSR	UARTECR
UART Base + (0x008 to 0x014)	Reserved	Reserved
UART Base + 0x018	UARTFR	------
UART Base + 0x01C	Reserved	Reserved
UART Base + 0x020	UARTILPR	UARTILPR
UART Base + 0x024	UARTIBRD	UARTIBRD
UART Base + 0x028	UARTFBRD	UARTFBRD
UART Base + 0x02C	UARTLCR_H	UARTLCR_H
UART Base + 0x030	UARTCR	UARTCR
UART Base + 0x034	UARTIFLS	UARTIFLS
UART Base + 0x038	UARTIMSC	UARTIMSC
UART Base + 0x03C	UARTRIS	------
UART Base + 0x040	UARTMIS	------
UART Base + 0x044	------	UARTICR
UART Base + 0x048	UARTDMACR	UARTDMACR
UART Base + (0x04C to 0x07C)	Reserved	Reserved
UART Base + 0x080	UARTTCR	UARTTCR
UART Base + 0x084	UARTITIP	UARTITIP
UART Base + 0x088	UARTITOP	UARTITOP
UART Base + 0x08C	UARTTDR	UARTTDR
UART Base + 0x090 to 0xFCC	Reserved	Reserved
UART Base + 0xFD0 to 0xFDC	Reserved for future ID expansion	Reserved for future ID expansion
UART Base + 0xFE0	UARTPeriphID0	------
UART Base + 0xFE4	UARTPeriphID1	------
UART Base + 0xFE8	UARTPeriphID2	------
UART Base + 0xFEC	UARTPeriphID3	------
UART Base + 0xFF0	UARTPCellID0	------
UART Base + 0xFF4	UARTPCellID1	------
UART Base + 0xFF8	UARTPCellID2	------
UART Base + 0xFFC	UARTPCellID3	------

Table 4.3-3 Register Memory Map

4.4 Block Partitioning

The sub-blocks in the design are listed below.

- APB Interface (Input module and Output module)
- Register block
- Transmit logic
- Receive logic
- Baud Rate Generator
- Fractional Baud Rate Generator
- Transmit FIFO
- Receive FIFO
- SIR encoder/decoder
- Modem status Interrupt generation logic
- Test Logic
- Synchronisers
- DMA logic
- Asynchronous Interrupt logic
- Revision designator

The design of the above-mentioned sub-blocks is detailed below.

4.4.1 APB Interface

The APB interface comprises an input and output module. This interface deals with reads and writes to registers from the APB. A simplified diagram of the APB interface block is shown in Figure 3 APB Interface Block diagram.

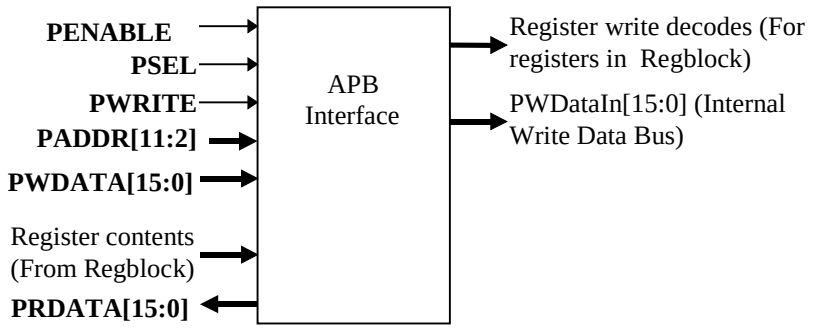

Figure 3 APB Interface Block diagram

Input Module

This module provides the APB write interface. It generates decodes for writes to all registers in the device. Writes to the UARTDR register actually result in writes to the transmit FIFO.

Each register addressable from the APB, is clocked by PCLK and is written to when the appropriate write enable is HIGH. The write enable terms are generated from a combinatorial address decode and PENABLE, PSEL and PWRITE signals.

Non-AMBA intra-chip inputs such as UARTTXDMACLR and UARTRXDMACLR are multiplexed with their corresponding integration test register bits. The block diagram in Figure 4 Input module illustrates the structure of the input module. In this block diagram, the write decodes are generated in the APB interface block, while the registers are actually located in the register block, RegBlock.

Since UARTCLK is the main clock for the UART core logic, control register bits are synchronised to UARTCLK before use in the UART core.

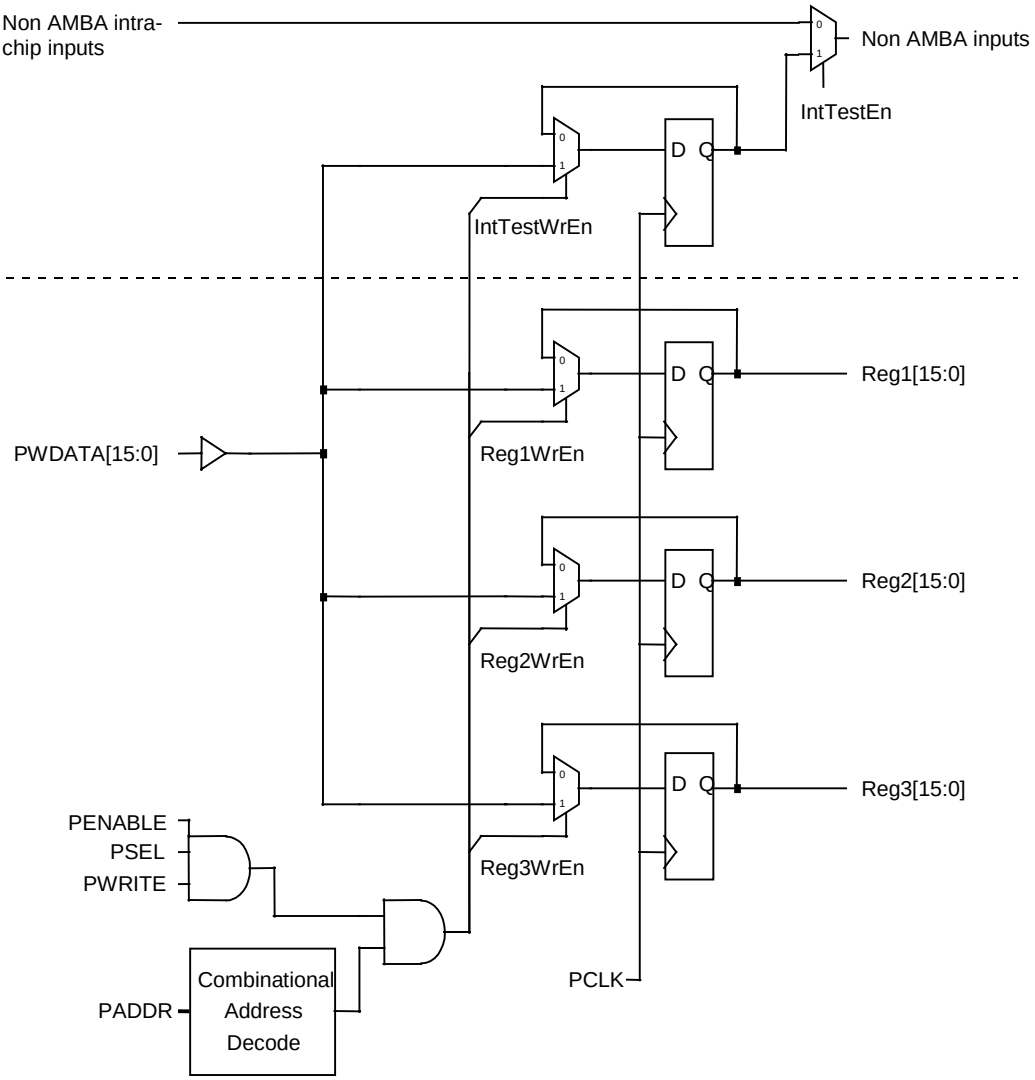

Figure 4 Input module

Output module

The output module in the APB interface consists of a multiplexer for selecting which of the various sources for read data is driven onto the read data bus. The output of the multiplexer is fed to a bank of flip-flops clocked by the system PCLK. A combinatorial address decode is used to select which of the output sources are driven onto the bus. Non-AMBA outputs such as UARTTXD, nSIROUT, the interrupt signals, the DMA request signals and the modem output signals are also made observable through APB read accesses. The output multiplexer is left such that, by default, zeros are driven on the data bus. This ensures that there are no restrictions placed on the type of

external bus connection method used for the data bus on the APB. A block diagram of the output module is shown in Figure 5 Output Module. The registers shown in this block diagram are actually located in the register block, except that for the non_AMBA outputs which is located in the Test block, while the output multiplexer and the final data register on the output of the multiplexer are located in the APB interface block.

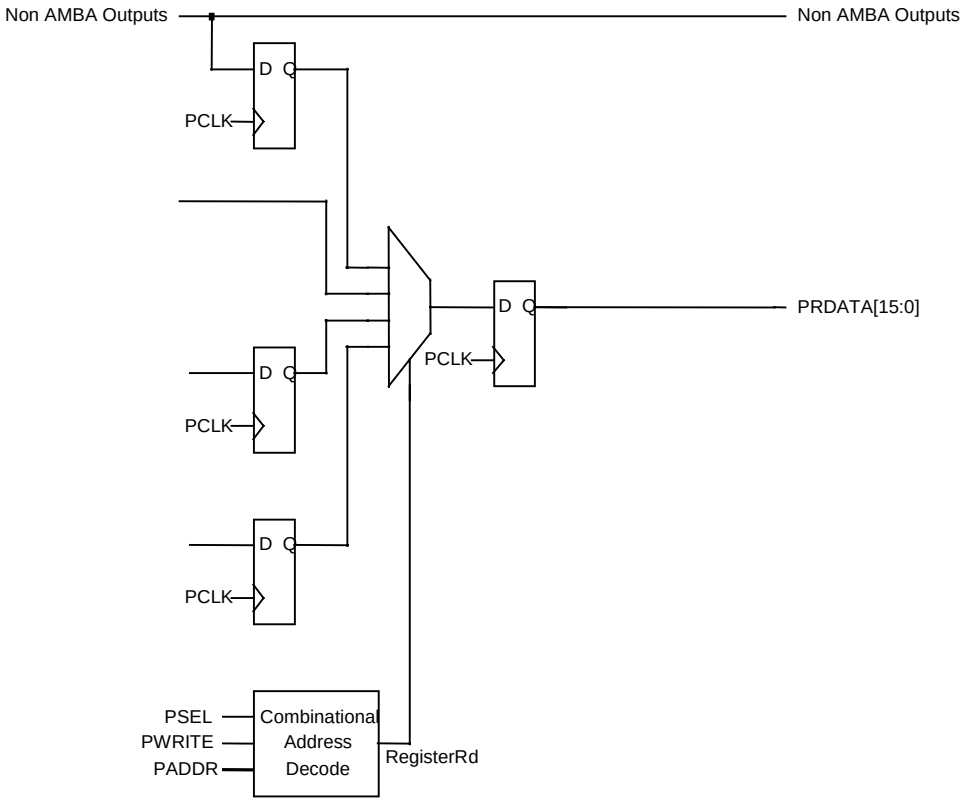

Figure 5 Output Module

4.4.2 Register Block

Besides the registers themselves, this block also contains the control logic required to synchronise the contents of the Line Control Register and the Integer Baud rate divider register (UARTLCR_H and UARTIBRD) and the IrDA low power counter register (UARTILPR) to the UARTCLK domain.

A two-buffer technique is used to synchronise the contents of the Line control register and the Integer Baud rate divider register (UARTLCR_H and UARTIBRD) to the UARTCLK domain. A separate 2-buffer technique is used to synchronise the contents of the IrDA Low power counter register (UARTILPR) to the UARTCLK domain. Similarly, another separate 2-buffer technique is used to synchronise the contents of the Fractional baud rate divider register (UARTFBRD) to the UARTCLK domain.

In this 2-buffer technique, writes from the APB to the UARTLCR_H and UARTIBRD registers, go to the respective first stage buffers. In the case of a write to the UARTIBRD register, bits 15:8 go to the LCRM 1st stage buffer and bits 7:0 go to the LCRL 1st stage buffer. When there is a write to UARTLCR_H, a control signal toggles. This signal is double-synchronised to the UARTCLK domain. An edge on the synchronised signal is detected by generating a delayed version and XOR-ing the synchronised signal with its delayed version. The delayed version is not updated unless the CharTxComp and CharRxComp signals are asserted which extends the write pulse. This pulse is then ANDed with the same CharTxComp and CharRxComp signals which delays the write pulse so that it only occurs when the character complete signals are valid. Thus, a one-UARTCLK-wide load pulse is generated with every write to the UARTLCR_H register. When this load signal is asserted, the contents of the first stage LCR buffer is copied into the second stage buffer in the UARTCLK domain.

Figure 6 Two-Buffer synchronisation mechanism for the LCR registers, illustrates the mechanism.

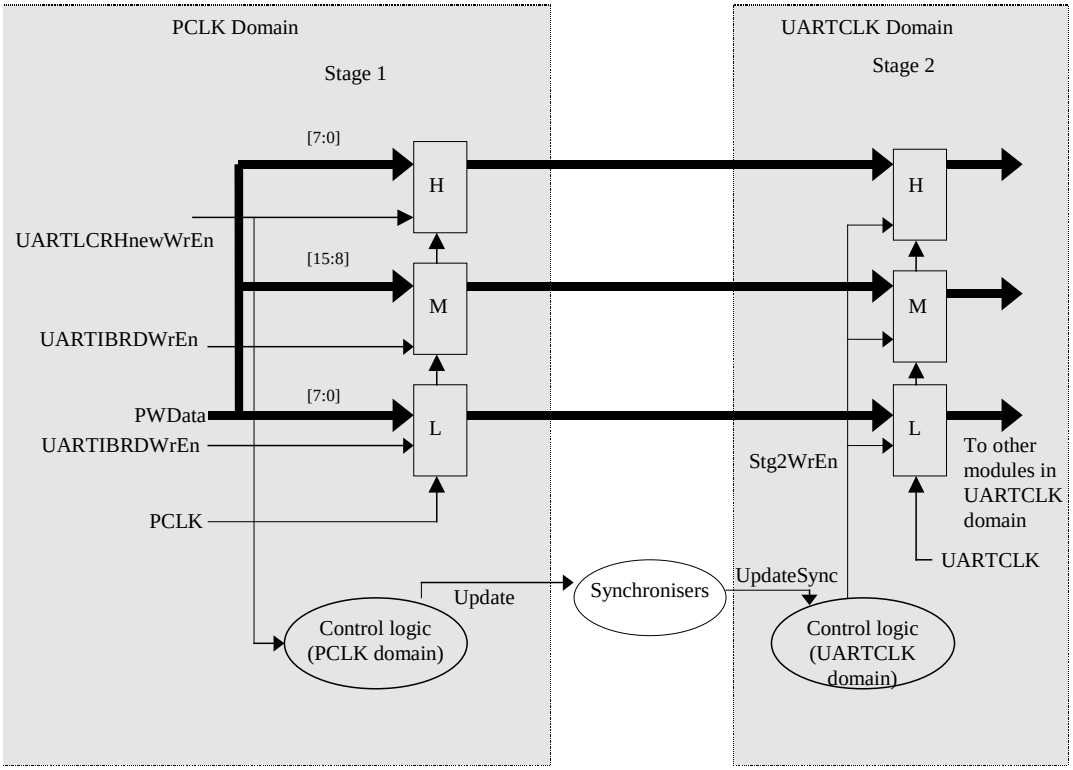

Figure 6 Two-Buffer synchronisation mechanism for the LCR registers

An identical set of buffers and control logic is used to synchronise the UARTILPR register to the UARTCLK domain, and the UARTFBRD register to the UARTCLK domain. Figure 7 Two-Buffer synchronisation mechanism for the UARTILPR register, illustrates the 2-buffer mechanism for the UARTILPR register.

When synchronisation is in progress, it is expected that there are no further writes immediately to the LCR register the UARTILPR register or the UARTFBRD register. This is a reasonable assumption since these registers are control registers and are not expected to be written to frequently.

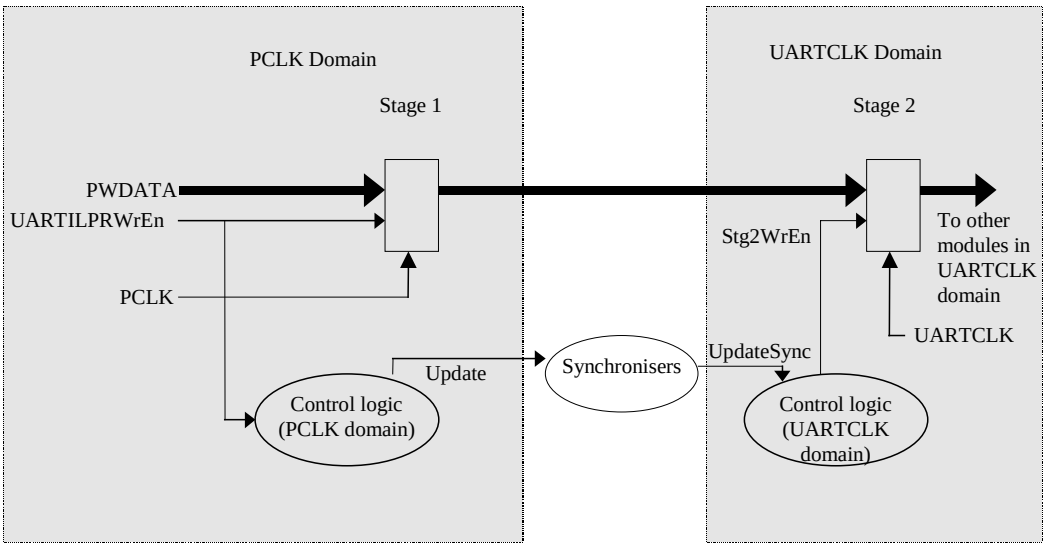

Figure 7 Two-Buffer synchronisation mechanism for the UARTILPR register

4.4.3 Baud Rate Generator

The baud rate generator contains three counters, all clocked by UARTCLK. These are:

- a 16-bit down counter, which is used to be either a "÷ n+1" or a "÷ n" counter. This is used to generate the integer part of the divisor.
- a 6-bit counter which is used to control the 16-bit counter to generate the appropriate fractional part of the divisor,
- a 8-bit counter to generate the IrLPBaud16 signal.

16-bit Counter

The combination of the 16-bit and 6-bit counters is used to generate the Baud16 signal that provides a timing reference to time the bit period. The 16-bit counter is re-loaded when it reaches one with either n or n+1 depending on the state of the carry output from the 6-bit counter. The output of this counter is used to generate the Baud16 pulse which is high for one UARTCLK cycle when the counter value is equal to '1'.

Note 'n' is the integer bit rate divisor value, as programmed into the UARTIBRD register.

6-bit Counter

The 6-bit counter is used to control the 16-bit counter re-load value. It is incremented by the value m every time the Baud16 pulse is high. When the counter rolls over, a carry signal is generated which is held high for one Baud16 period. This carry signal when HIGH re-loads the above 16-bit counter with 'n+1' and with value 'n' when LOW.

Note 'm' is the fractional bit rate divisor value as programmed into the UARTFBRD register.

8-bit Counter

The 8-bit down counter is used to generate the IrLPBaud16 signal which is used to time the pulse widths in the IrDA-encoded transmit bit stream in the low power mode. When the counter underflows, the content of the UARTILPR register is reloaded into this counter and a one UARTCLK-wide pulse is produced on the IrLPBaud16 signal.

This block also contains the UARTCLK-domain part of the control logic used to synchronise the contents of the Line control register and the baud rate divisor registers (UARTLCR_H, UARTIBRD, UARTFBRD and UARTILPR) to the UARTCLK domain.

The ZeroBaud output signal is meant to indicate to the transmit and receive blocks that an illegal divisor value has been programmed. A divisor is illegal if zero has been written into the integer baud rate divisor register (UARTIBRD) or all 1's (0xFFFF) have been written into the integer baud rate divisor register (UARTIBRD) with the value of the fractional baud rate divisor register (UARTFBRD) being greater than zero. When this signal is asserted, all transmission and reception are aborted and the state machines return to their idle states.

The Baud rate counters do not run when the UART is disabled, and IrLPBaud16 counter only runs when either IrDA mode is active. The counters will be enabled immediately that the UART is enabled, but will only be disabled once the CharTxComp and CharRxComp signals are both high, as well as the UART being disabled

Figure 8 Baud Rate Generator shows the block diagram of the Baud Rate generator.

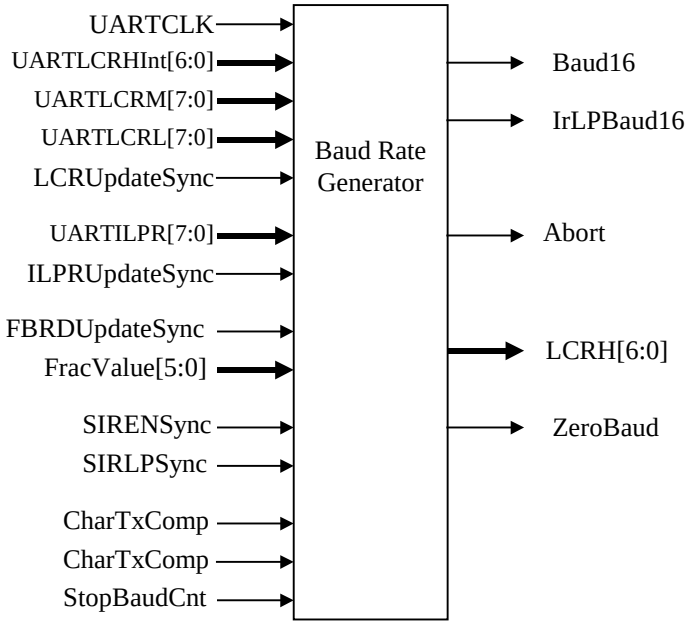

Figure 8 Baud Rate Generator

4.4.4 Transmit logic

The transmit logic converts the parallel transmit data into a serial bit stream with a bit rate derived from the time period of the Baud16 signal. This block performs parity generation, data framing and parallel-to-serial shifting.

Transmit data from the Transmit FIFO is framed according to the transmission parameters specified in the UARTLCR_H register. This block operates on UARTCLK, with the Baud16 signal providing the bit rate information. The block diagram of the transmit logic is shown in Figure 9 UART Transmit Logic.

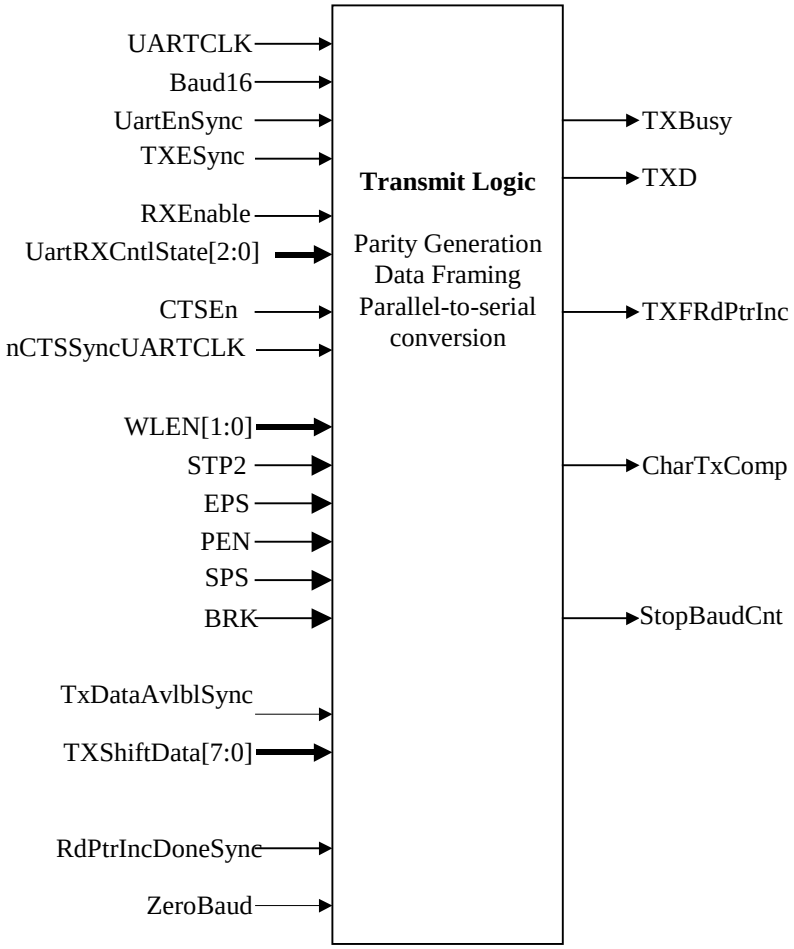

Figure 9 UART Transmit Logic

When the TXDataAvlblSync signal is asserted, it indicates to the Transmit section the existence, in the transmit FIFO, of data to be transmitted. On detecting an asserted TXDataAvlblSync signal, the data present on the TXShiftData[7:0] bus, which corresponds to the contents of the Transmit Shift register in the FIFO, is framed in accordance with the transmit parameters specified by the control signals WLEN (number of data bits per transmit frame), STP2 (one or two stop bits), EPS (Even Parity select), SPS (Stick Parity Select), PEN (Parity enable) and BRK (Break). The transmit FIFO has an internal transmit shift register, whose contents are always driven onto the TXShiftData signal. Thus, the TXShiftData signal remains stable for the duration of transmission of the entire data byte. An internal counter in the Transmit block keeps track of the number of bits transmitted. The data bit in TXShiftData[7:0] pointed to by the current value of this counter, is taken as the data bit to be transmitted. The data bits to be transmitted are latched into a D-type clocked by UARTCLK one by one in accordance with the bit rate information derived from the Baud16 signal. The transmit data stream is available on the TXD signal.

When a complete frame has been transmitted, the TXFRdPtrInc signal is asserted to indicate to the FIFO controller that the Read pointer may be incremented. The assertion of the RdPtrIncDoneSync signal indicates to the transmitter that the FIFO controller has completed the increment. In case the TXDataAvlblSync signal continues to remain asserted, indicating existence of transmit data, the sequence is repeated for the next data byte.

When the Transmit section is busy transmitting, the TXBusy signal is asserted. This signal is synchronous to UARTCLK. This signal is synchronised to the PCLK domain and fed to the Tx FIFO block where the BUSY signal is generated. As explained later in Section 4.4.6 Transmit FIFO, the BUSY signal is asserted when data is available in the Transmit FIFO or if the Transmitter is busy transmitting. Thus, the BUSY signal is asserted from

the time the first data is written into the Transmit FIFO till the time when the last available data byte has been fully transmitted by the transmitter.

When the ZeroBaud input signal is asserted, it indicates that either an illegal value of zero has been written to the UARTIBRD register or that all 1's (0xFFFF) have been written to the UARTIBRD register with the value of the UARTFBRD register being greater than zero. On assertion of this signal, transmission is aborted and the transmit state machine returns to the Idle state.

4.4.5 Receive Logic

The receive logic performs serial-to-parallel conversion on the serial data stream received on the RXD input pin. The Baud16 input signal is used as an enable signal for an internal receive counter. The RXD input is synchronised to the UARTCLK clock domain. Three readings are continuously taken, at Baud16 intervals, of the RXD input line and the majority value is kept. This majority value is the RXDSampled signal. When the data input (RXDSampled) goes LOW, the receive down counter, which counts on Baud16 being high, is loaded with a value of 8. The RXDSampled signal is continuously sampled until the count reaches zero, and if it remains LOW a valid start bit is said to have been detected.

After the start bit is detected, the receive counter is loaded with 15. This count decrements with every Baud16 pulse. When the receive counter counts to 0, the RXDSampled line is sampled for the next data bit and the receive counter is reloaded with 15. Thus, the sampling of the RXDSampled line occurs at the middle of every bit period. The sampled data is shifted into a receive buffer.

When a full frame of data, including the stop bits, has been received, the received data is checked for conformance to the transmission parameters specified by the UARTLCR_H register. Parity and frame error checks are performed on the received data with the results reflected in the error bits for that word. At the end of each frame, the data bits received, the stop bits and the parity bit are checked to see if all are zeros. In that case, a Break is said to have been detected. When Break is detected, a Zero byte with the Break error bit set is written into the receive FIFO. The receiver then waits for the RXD line to go high before the next data frame can be sampled and shifted in.

When data is ready to be written into the receive FIFO, the RXFIFOWr signal that is synchronous to UARTCLK, is asserted. Data to be written into the FIFO along with the error bits associated with the data, is driven on FIFOData[10:0]. When the RXFIFOWrDone signal is asserted, it indicates that the write is complete. The feedback signal is required since writes to the receive FIFO occur on PCLK. The delay of a few clock cycles for synchronisation does not affect data reception because the receive bit rate is much slower than PCLK and UARTCLK.

The block diagram of the receive logic is shown in Figure 10 UART Receive Logic.

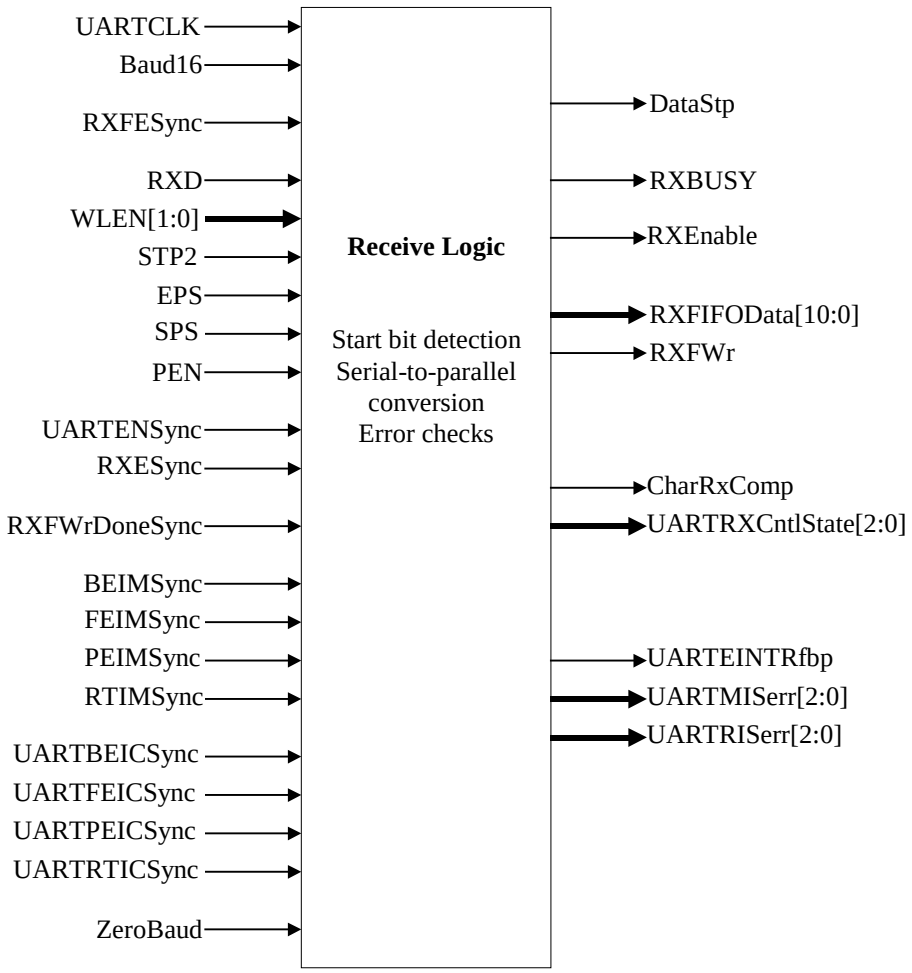

Figure 10 UART Receive Logic

The UARTENSync signal is the ‘UARTEN’ bit in the UARTCR register, which is synchronised to the UARTCLK domain. The RXESync signal is the ‘RXE’ bit in the UARTCR register, which is synchronised to the UARTCLK domain. When either of these signals are de-asserted, the receive logic is disabled once that current character has completed.

If the receive FIFO is not empty, signified by RXFESync being low, and the RXD line is detected as being idle i.e. held in its high state for more than a 32 bit period, then the DataStp signal is asserted and becomes the asynchronous Receive Timeout (UARTRTRIS) interrupt. This interrupt informs the system that unread data still remains in the receive FIFO after the receive line has gone idle. A nine bit counter is used to time this 32 receive time out period and this is effectively disabled by continuous loading of it’s maximum value when either the RTIMSync, the receive time out interrupt mask signal is low, or the receive FIFO empty status signal RXFESync is high. If both the latter conditions are not satisfied, then the counter is enabled, but is reloaded with it’s maximum value on detection of every valid stop bit, otherwise it decrements with every Baud16 pulse. If it reaches zero, then the UARTRTRIS interrupt is asserted. When the user does not require the UARTRTRIS function, de-assertion of the RTIMSync signal ensures that power loss is minimised by the disabled counter logic. The UARTRTRIS interrupt can be cleared by reading the contents of the receive FIFO until it becomes empty, which is expected action of the receive timeout interrupt service routine. Alternatively it can be cleared by writing a ‘1’ to the RTIC bit within the UARTICR register. This clear action immediately de-asserts the interrupt in the PCLK domain, but there is a synchronisation latency before it reloads the counter with its maximum value. If the data has not been read out of the FIFO within a further 32 bit periods, then the UARTRTRIS interrupt will be raised again.

The errors, frame, break and parity, which are generated due to non-conformance to the transmission parameters as defined in the UARTLCR_H register also generate interrupts. When an error is generated the raw interrupt status (UARTFERIS, UARTPERIS or UARTBERIS) is set, but the masked interrupt status (UARTFEMIS, UARTPEMIS or UARTBEMIS) is only set if the relevant interrupt mask (PEIMSync, PEIMSync or BEIMSync) is set. The raw and masked error interrupts are individually clearable by writing to the relevant bit within the Interrupt Clear register (UARTICR). The Break, framing and parity error masked interrupts are ORed together to generate a combined interrupt, UARTEINTRfbp, signal which is later ORed with the Overrun Error interrupt to generate a combined error interrupt, UARTEINTR, output.

4.4.6 Transmit FIFO

The block diagram of the transmit FIFO is shown in Figure 11 Transmit FIFO. It consists of a 16-deep 8-bit wide circular buffer. Reads and writes to the FIFO result in access to buffer locations addressed by a read pointer and a write pointer respectively. The pointer values are compared to deduce the FIFO fill level. The transmit interrupt, both raw and masked status, is also generated by the Transmit FIFO control logic.

Writes to the Transmit FIFO occur through the APB interface in the PCLK domain. The UARTDRWrEn signal is a decode of writes to the UARTDR register. When the write enable signal UARTDRWrEn is asserted, data present on the PWDATAIn[15:0] bus is written, on the rising edge of PCLK, into the FIFO location pointed to by the current value of the write pointer. At the end of every FIFO write, the write pointer is incremented.

The Transmit FIFO contains a transmit Shift register which holds the data byte that is currently being transmitted.

Both the read pointer and the write pointer are clocked on PCLK. Reads from the transmit FIFO are done by the Transmit section, which runs on UARTCLK. The Tx FIFO always drives the RdData bus with the contents of the transmit shift register. When the FIFO is non-empty, this is indicated by the TXDataAvlbl signal. After the Transmit section has completed transmitting one data byte, the TXFRdPtrInc signal is asserted by the transmit logic. This signal is synchronised to the PCLK domain (TXFRdPtrIncSync) and is used to increment the read pointer. The FIFO location pointed to by the read pointer is copied into the transmit shift register. When the increment operation is complete, the RdPtrIncDone signal is asserted to indicate the completion to the transmit section.

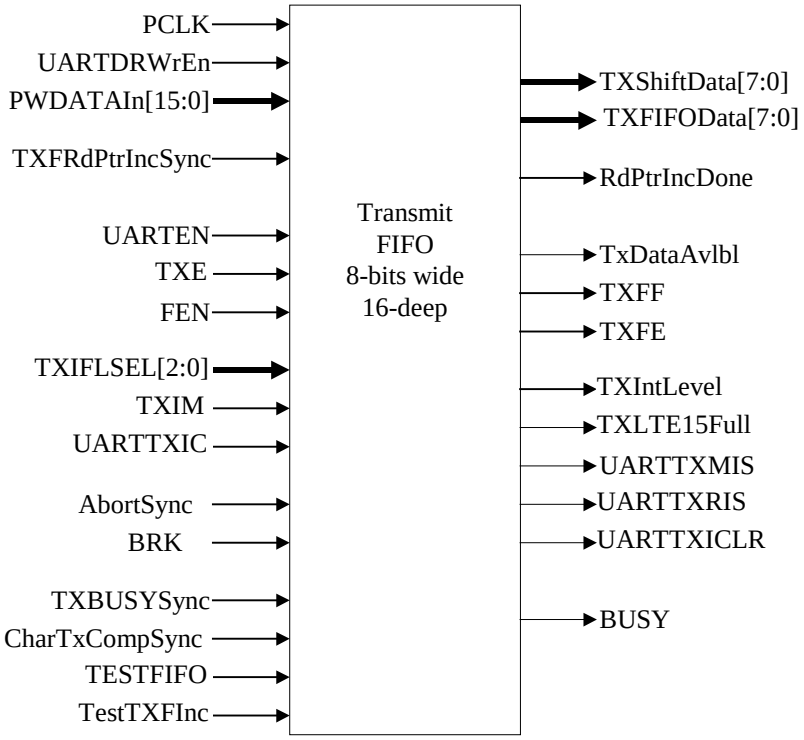

Figure 12 Transmit FIFO

When the TESTFIFO bit is set in the UART test Control register (UARTTCR) data can be read out of the Transmit FIFO, by reading from the Test Data register (UARTTDR). Writes to the Transmit FIFO occur in the normal way. In the TESTFIFO mode the read pointer is incremented after a read from the UARTTDR register. The UART does not have to be enabled in order to read from the UARTTDR register.

The interface between the FIFO control logic and the data buffer is shown in Figure 13 Transmit FIFO Control logic interaction with Data buffer. The WrPtr[3:0] signal represents the Write pointer, which points to the next empty location in the data buffer into which the next write can occur. The RdPtr[3:0] represents the Read pointer, which points to the next data byte that will be loaded into the Transmit shift register. The RegFileWrEn signal is derived from the UARTDRWrEn input to the FIFO by qualifying with the FIFO fill status to prevent writes to the transmit FIFO if it is already full. PCLK is used to clock the storage elements in the register file. The Register bank includes the output multiplexer for reads. This multiplexer uses the RdPtr[3:0] bus to select the buffer location whose contents have to be driven out onto the TxFIFOData[7:0] bus.

The transmit FIFO has a FIFO-disabled mode in which one entry in the FIFO is used as the transmit holding register. The transmit shift register in the transmit FIFO is common to both the FIFO enabled mode and the FIFO disabled mode.

The transmit interrupt is generated by the Transmit FIFO control logic dependent upon the transmit FIFO interrupt level. The transmit FIFO interrupt level is programmable by writing to the UARTIFLS (UART Interrupt FIFO level select) register and can be set to 1/8, ¼, ½, ¾ or 7/8 full. The FIFO control logic generates the raw Transmit Interrupt (UARTTXRIS) when the FIFO is enabled and the Transmit FIFO is filled to it's programmed level or below or when the FIFO is disabled and the transmit holding buffer is empty. The masked Transmit Interrupt (UARTTXMIS) is set when the raw Transmit Interrupt is set and the Transmit Interrupt mask (TXIM) is also set. The raw transmit interrupt is cleared when the condition that caused the interrupt is no longer present, i.e. when there are more than the programmed level of valid entries in the FIFO with the FIFO enabled, or when the Holding register is full with the FIFO disabled, or when the Transmit Interrupt clear bit is written to in the Interrupt clear register (UARTICR).

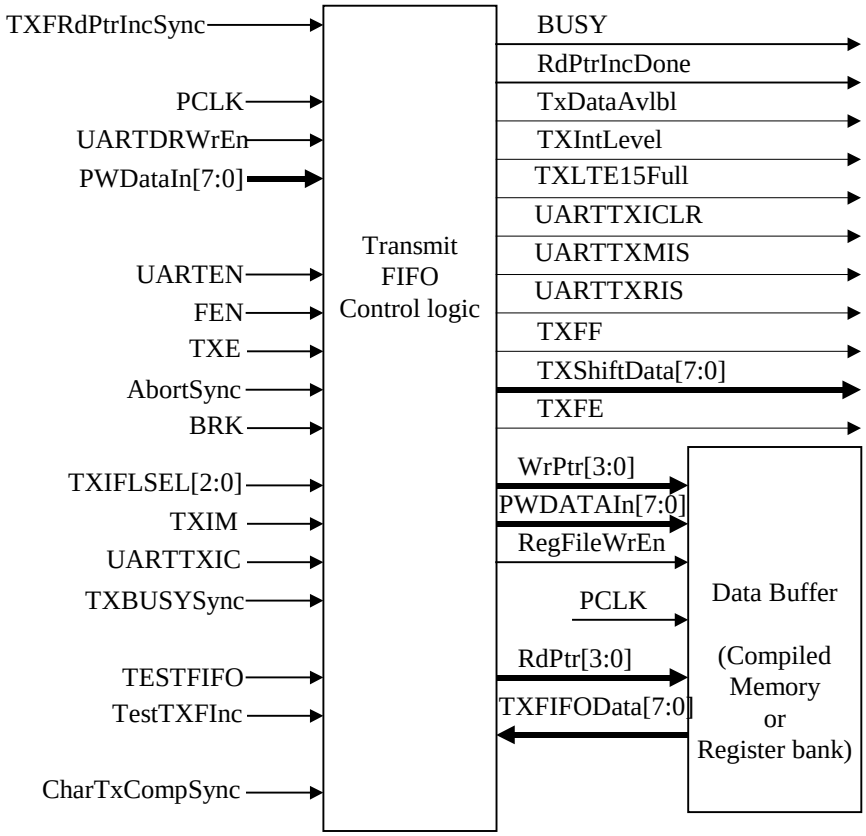

Figure 13 Transmit FIFO Control logic interaction with Data buffer

The TXFF signal, the TXFE signal and the BUSY signal are readable through the UARTFR register. When asserted, the TXFF signal indicates that the transmit FIFO is full if the FIFO is enabled or that the holding register is full when the FIFO is disabled. When the TXFE signal is asserted, it indicates that the transmit FIFO is empty. The BUSY signal is asserted when the transmit FIFO is not empty or if the Transmitter is busy as indicated by the assertion of the TXBUSY signal. This signal is independent of whether the UART is enabled or not.

The transmit FIFO control logic also includes the transmit part of the hardware flow control. When the CTS flow control is enabled, the transmitter can only transmit data when the nCTSSyncUARTCLK signal is asserted. The transmit hardware flow control is selectable via the CTSEn signal in the UART Control Register (UARTCR).

4.4.7 Receive FIFO

The block diagram of the receive FIFO is shown in Figure 14 Receive FIFO. It is similar to the transmit FIFO in most respects. The receive FIFO is 12 bits wide and 16-deep. Writes to the receive FIFO occur from the Receive section. Reads from the receive FIFO occur through the APB interface.

For the purpose of evaluating the FIFOFillLevel, it would be convenient to have the read pointer and the write pointer operate on the same clock domain. Operating them on UARTCLK would mean data cannot be served consistently onto the APB since two successive reads from the APB could occur with a separation of just one PCLK period (and the synchronisation of the read pointer increment signal in itself would consume two PCLK cycles). Hence both the read pointer and the write pointer operate on PCLK.

On receiving a full byte of data, the Receiver section drives data on the RXFIFOData[11:0] bus and asserts the write enable signal (RXFWr) synchronous to UARTCLK. This signal is synchronised to PCLK and is seen as the RXFWrSync input of the receive FIFO. When the write is complete, the RXFWrDone signal is asserted by the

FIFO control logic. This signal is used by the receive section to de-assert the write enable signal. After every write, the write pointer is incremented.

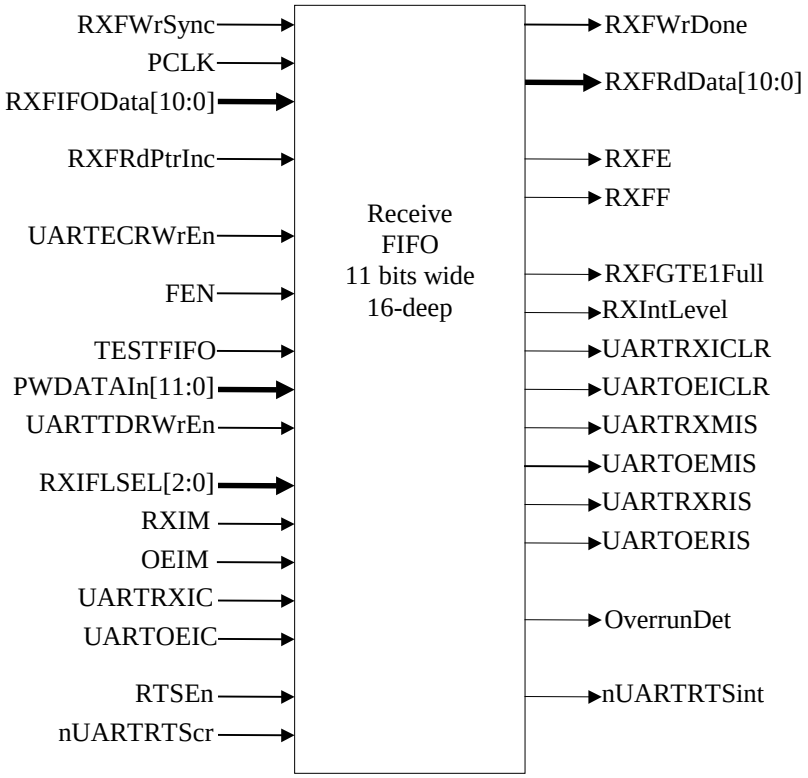

Figure 14 Receive FIFO

On the read side, the contents of the FIFO element selected by the current value of the read pointer is always driven on the RXFRdData[11:0] bus. The RXFRdPtrInc signal, which indicates when to increment the read pointer, is derived from a decode of read accesses to the UARTDR register.

When the TESTFIFO bit is set in the UART Test Control register (UARTTCR) data can be written into the Receive FIFO, by writing to the UARTTDR register. Reads from the Receive FIFO occur in the normal way.

Writes to the Receive FIFO occur in the normal way. In the TESTFIFO mode the write pointer is incremented after a write to the UARTTDR register. The UART does not have to be enabled in order to write to the UARTTDR register.

The raw Receive Interrupt (UARTRXRIS) is asserted when there are the programmed FIFO interrupt level or more valid entries in the FIFO with the FIFO enabled or when the Holding register is full with the FIFO disabled. The programmed FIFO interrupt level is set by writing to the Interrupt FIFO Level Select (UARTIFLS) register. The masked status of the Receive interrupt (UARTRXMIS) is asserted when the UARTRIS signal is asserted and the Receive Interrupt mask (RXIM) is set. The interrupt is de-asserted when the FIFO contents have been read out such that there are less than the programmed level of valid entries in the FIFO when the FIFO is enabled or when the FIFO is empty when the FIFO is disabled, or by writing to the RXIC bit within the Interrupt Clear (UARTICR) register.

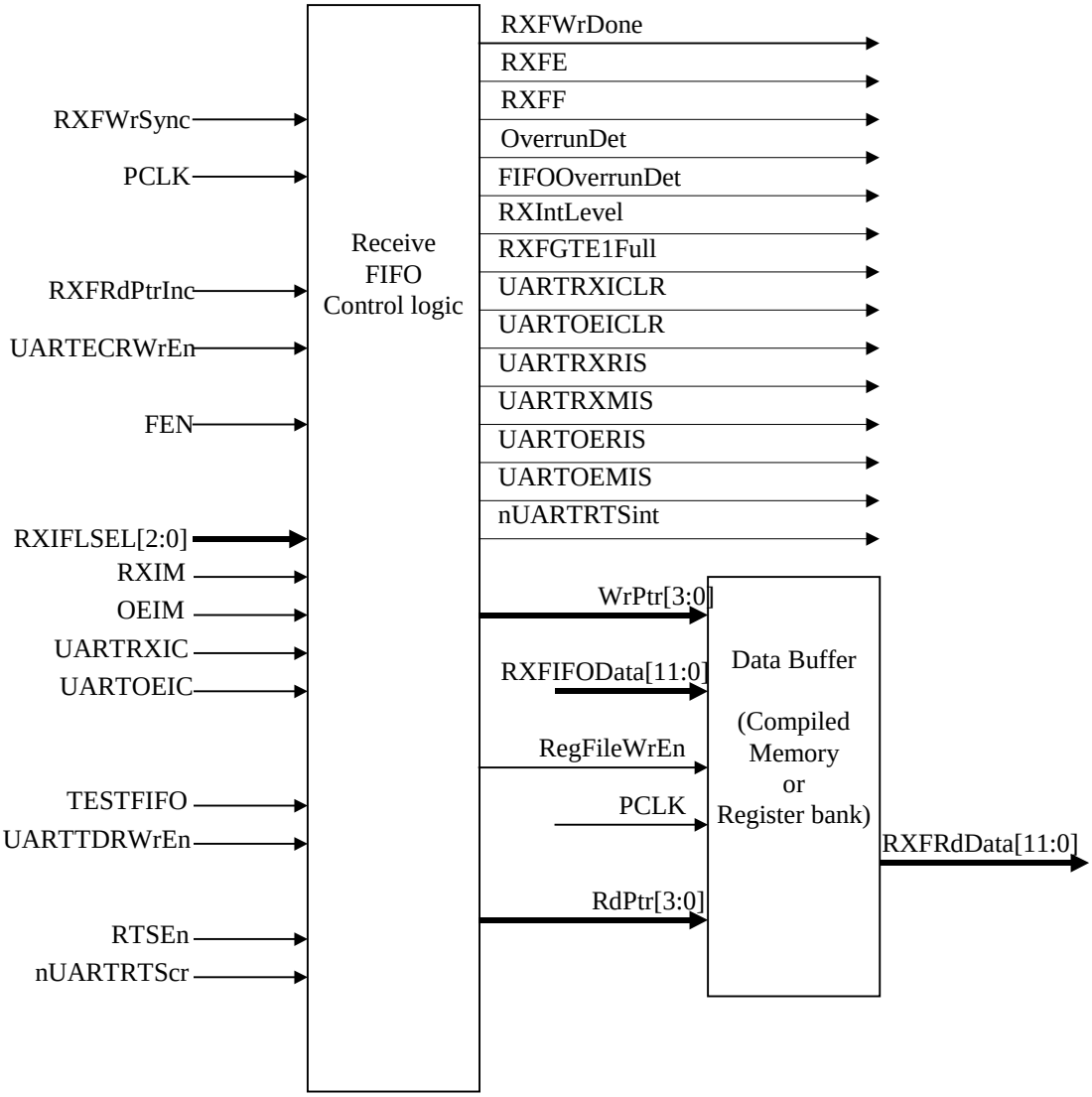

Figure 15 Receive FIFO Control logic interaction with Data buffer

The receive FIFO controller asserts the OverrunDet signal when the FIFO is already full and an additional data frame is received, thereby resulting in an overrun of the FIFO. This signal is readable as the 'Overrun Error' bit in the UARTRSR register. When the overrun condition occurs, the OverrunDet output signal is asserted. When there is a write to the UARTRSR/UARTECR register, the OverrunDet signal is cleared. An Overrun Interrupt is generated when an Overrun error occurs. The raw overrun error interrupt status (UARTOERIS) is set when a rising edge is detected on the OverrunDet signal, and the masked status is set when the UARTOERIS is asserted and the Overrun Error Interrupt mask (OEIM) is set. The UARTOERIS is de-asserted when the UARTICR register is written to. The Overrun error bit is added to the RxFIFOData in the receive FIFO logic.

The RXFE signal and the RXFF signal indicate the empty and full status of the receive FIFO. These signals are readable through the 'RXFE' bit and the 'RXFF' bit in the UARTFR register.

The receive part of the hardware flow control is also in the Receive FIFO control logic. The enable signal, RTSEn is set in the UARTCR register. When RTSEn is asserted, and if there is space in the receive FIFO, the nUARTRTS output is asserted. This remains asserted until the receive FIFO is filled to it's programmed interrupt level, as indicated by RXIntLevel being asserted. When RXIntLevel becomes asserted the nUARTRTS signal is de-asserted indicating that no more data can be accepted. RXIntLevel is de-asserted once data has been read out of the receive FIFO such that it is filled to less than it's programmed interrupt level, and nUARTRTS is re-

asserted. When RTSEn is de-asserted the nUARTRTS output gets the value which is set on the nUARTRTScr signal from the UARTCR register.

Figure 15 Receive FIFO Control logic interaction with Data buffer illustrates the interface between the FIFO control logic and the data buffer. Writes to the register bank occur to the location pointed to by the current value of the WrPtr[3:0] signal which indicates the value of the write pointer. In the case of reads, the contents of the location pointed to by the current value of the RdPtr[3:0] signal is always driven onto the RXFRdData[11:0] output lines. The control logic drives the RdPtr[3:0] lines with the value of the read pointer.

4.4.8 SIR Encoder/Decoder

The SIR encoder converts the transmit bit stream into an IrDA-compliant encoded bit stream on the nSIROUT line. The outputs of this block, UARTTXDint and nSIROUTint are internal versions of the UART output signals UARTTXD and nSIROUT. In this protocol, a zero is transmitted as a high pulse and a one is transmitted as a zero. The width of the pulse is specified as $3/16^{th}$ of the selected bit period. The SIR decoder converts the IrDA-compliant receive signal into a bit stream for the UART core. The SIR receive logic interprets a high state as a logic one and low pulses as logic zeros. For more details please refer to the SIR Physical Layer Link Specification Ver 1.1 [Ref. 4].

In the low power mode, the width of the pulse is set to 3 times the time period of the IrLPBaud16 signal. The time period of the nSIROUT bit stream is still equal to the programmed bit period and is governed by the period of the Baud16 signal. It must be noted that, in the low power mode, there exists an upper limit on the allowable bit rate if the integrity of the IrDA pulse stream is to be maintained. The upper limit is determined by the requirement that the width of 3 times the period of the IrLPBaud16 signal should be less than one bit period. With a nominal IrLPBaud16 frequency of 2MHz, this translates to a maximum allowable baud rate of 666.6 Kbps. This is not a problem since the specification [Ref. 1] states that the UART is to support bit rates of only upto 115.2Kbps for the SIR endec. There also exists a lower limit on the frequency of UARTCLK if the low power mode of the IrDA is to be used. The reload value for the IrLPBaud16 counter can be a minimum of 1. Given that IrLPBaud16 needs to have a minimum frequency of 1.42MHz to satisfy IrDA requirements, UARTCLK needs to be at least 1.42MHz to meet the minimum pulse width requirement. In the case of transmission, the IrLPBaud16 signal is used to control the width of the transmit pulse stream only in the low power mode. On the other hand, in the case of reception, the IrLPBaud16 signal is used to sample the pulses in the input SIRIN stream both in the normal mode and in the low power mode. Thus, the IrLPBaud16 signal is generated irrespective of whether the IrDA section is programmed to operate in the normal mode or in the low power mode but it is not generated when both IrDA modes are disabled.

The SIR receiver section contains a 4-bit binary counter, which operates on UARTCLK in the normal mode with the Baud16 signal as an enable signal. When the input SIRIN signal is high, the decoded output signal, RXD, is driven high to indicate a 1. A low on the SIRIN line is sampled multiple times on IrLPBaud16 for glitch rejection and converted into a low on the RXD line. The time period for which the RXD line is pulled low corresponds to approximately one bit period at the programmed bit rate. After the counter rolls over, input sampling restarts. The SIRLPSync signal is the UARTCLK-synchronised version of the SIRLP mode selection bit in the UARTCR register. This signal determines the IrDA encoding strategy i.e. whether the IrDA transmitter block is to operate in the normal mode or in the low power mode. The SIRENSync signal is the SIREN bit in the UARTCR register, which has been synchronised to UARTCLK. This signal enables / disables the SIR section.

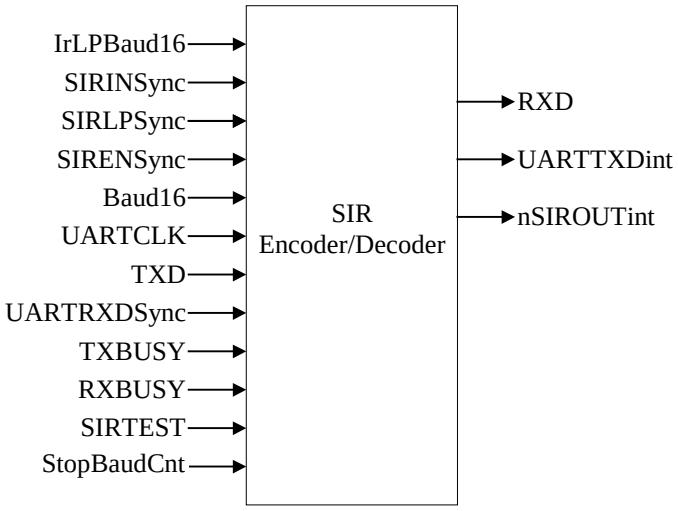

Figure 16 SIR Encoder / Decoder

The block diagram of the SIR Encoder/Decoder is shown in Figure 16 SIR Encoder / Decoder. It converts the transmit bit stream on TXD into an IrDA encoded bit stream on nSIROUT. When a 1 is to be transmitted a 0 is driven on the SIR output signal, nSIROUT. When a 0 is to be transmitted, an internal two-bit binary counter is used to clock out a high pulse with a width of 3 times the time period of the Baud16 signal as required by the SIR Physical Layer Link specifications [Ref. 4]. The counter is clocked by UARTCLK, with the Baud16 signal used as the enable signal. In the low power mode, the IrLPBaud16 signal is used as the enable signal to determine the transmit pulse width for a 0.

4.4.9 Modem Status Interrupt Generation Logic

The modem block generates individual interrupts for each of the modem input signals as well as one combined interrupt. The block diagram of this block is shown in Figure 17 Modem status interrupt generation.

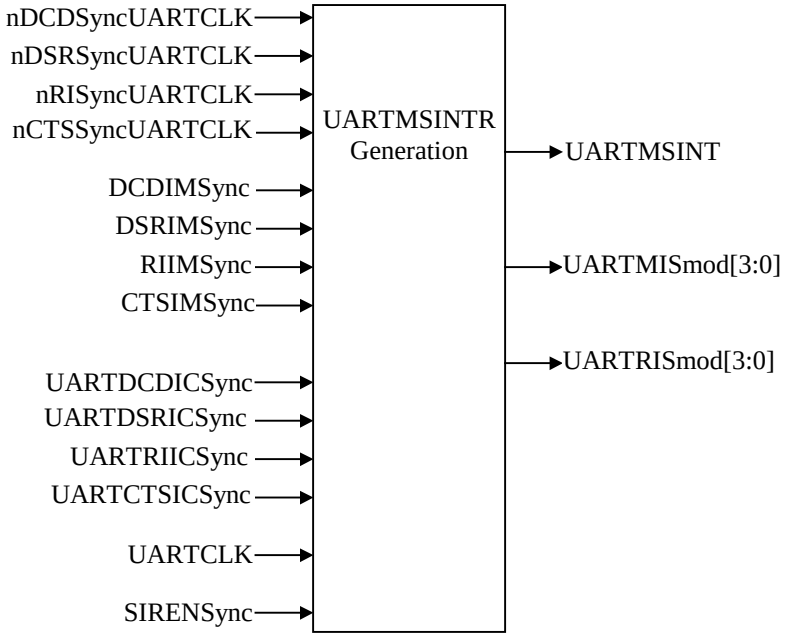

Figure 17 Modem status interrupt generation

The raw individual interrupts (UARTDCDRIS, UARTDSRRIS, UARTRIRIS and UARTCTSRIS) are generated when there is a change in any of the modem signals nDCD, nDSR, nRI or nCTS.

The modem signals are fed to the UartModem block after synchronisation to the UARTCLK domain as nDCDSyncUARTCLK, nDSRSyncUARTCLK, nRISyncUARTCLK and nCTSSyncUARTCLK. A change in the signal level is detected by clocking the signals through D-types clocked by the rising edge of UARTCLK and by XORing the signal value with the delayed version of the signal. UARTCLK is assumed to be free running.

The individual masked interrupts (UARTDCDMIS, UARTDSRMIS, UARTRIMIS and UARTCTSMIS) are set when the raw interrupt is asserted and the relevant interrupt mask (DCDIMSync, DSRIMSync, RIIMSync or CTSIMSync) is set. The interrupt masks are synchronised to the UARTCLK domain before entering the UartModem block. The raw interrupts are individually clearable by writing a ‘1’ to the respective UARTICR register bit. The raw and masked interrupts are output from the block as two 4-bit buses, UARTMISmod and UARTRISmod. The combined modem interrupt (UARTMSINT) is generated by ORing the four masked modem interrupts together. The UARTMSINT signal later in the logic becomes the UARTMSINTR output signal.

4.4.10 Asynchronous Interrupt Generation

This block contains the asynchronous combinatorial interrupt logic. The block diagram of this block is shown in Figure 18 Asynchronous Interrupt Generation

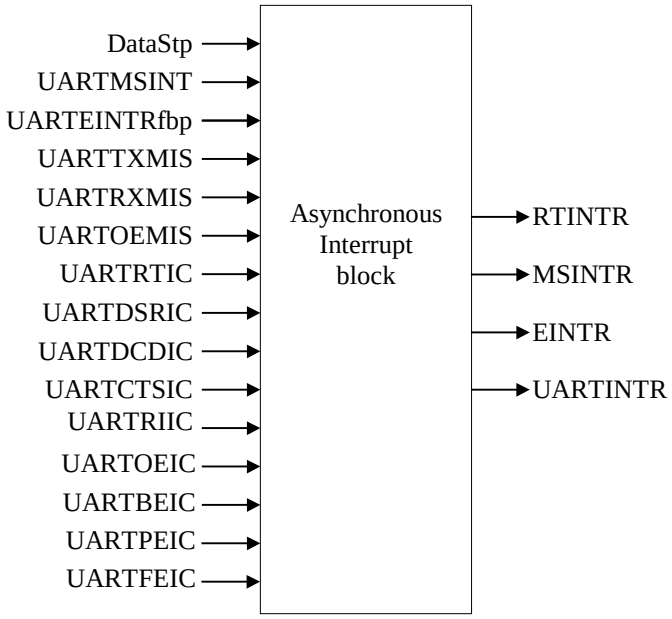

Figure 18 Asynchronous Interrupt Generation

Interrupts generated in the UARTCLK domain are treated as being asynchronous to the final processor clock domain. These interrupts are asserted in the UARTCLK domain, but synchronously removed by writing to bits within the UART interrupt clear register (UARTICR) which reside in the PCLK domain. To reduce the latency of interrupt clearance, the reset action is initiated and removed in the PCLK domain, but time must be allowed for the actual clear signal to be synchronised back to the UARTCLK domain. Having cleared within the UARTCLK domain, the asynchronous interrupt path is re-enabled.

When any of the modem interrupt clear signals UARTDSRIC, UARTDCDIC, UARTCTSIC or UARTRIIC are high, then the combined modem status interrupt MSINTR will be forced low, thereby disabling the interrupt path for this signal.

When any of the error interrupt signals UARTEOIC, UARTBEIC, UARTPEIC or UARTFEIC are high, then the combined error interrupt EINTR will be forced low, thereby disabling the interrupt path for these signals.

When the UARTRTIC is high, then receive timeout interrupt RTINTR will be forced low, thereby disabling the interrupt path for this signal.

The UARTINTR interrupt is the OR of the individual RTINTR, MSINTR, EINTR, UARTTXMIS and UARTRXMIS interrupts.

4.4.11 DMA Interface

This block provides an interface to connect to a DMA controller. It generates four DMA signals, two for the transmit request UARTTXDMASREQ, UARTTXDMABREQ, and two for the receive requests UARTRXDMASREQ, UARTRXDMABREQ.

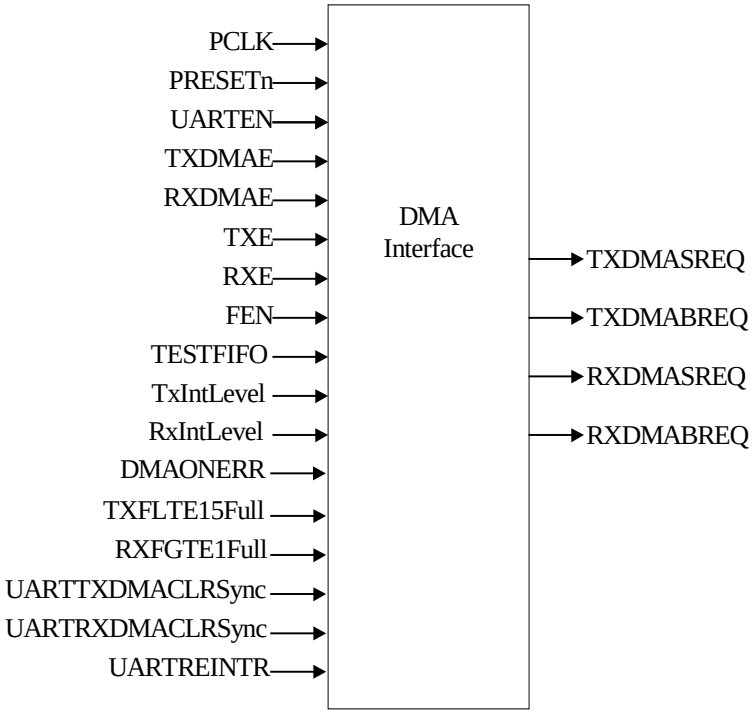

Figure 19 DMA Interface

DMA Transmit Interface

The block diagram is shown in Figure 19 DMA Interface. In normal operation, the DMA transmit interface is enabled when the UART is enabled, the transmit section is enabled, and transmit DMA is enabled i.e. when signals UARTEN, TXE and TXDMAE are all high. It can also be enabled in test mode, in which case only TESTFIFO and TXDMAE have to be high.

The level within the transmit FIFO is depicted by the signals TxIntLevel and TXFLTE15Full. The TxIntLevel is set when the data within the transmit FIFO is less than programmable watermark level whilst the TXFLTE15Full is set when there is less fifteen or less spaces within the transmit FIFO. The transmit DMA single request TXDMASREQ signal is set when TXFLTE15Full is active high. The transmit DMA burst request TXDMABREQ is set when the transmit FIFO function is enabled and the TxIntLevel is active high. Therefore, it is possible for both TXDMASREQ and TXDMASREQ to be active together.

The DMA controller has to be programmed with the most suitable burst length implied by the programmed transmit FIFO watermark level.

The TXDMASREQ and TXDMABREQ signals are cleared by the application of the UARTTXDMACLR signal from the DMA controller, which signifies that the end of a single/burst transfer is about to occur. They are also cleared when the transmit DMA function is disabled i.e. when TXDMAE is low.

DMA Receive Interface

In normal operation, the DMA receive interface is enabled when the UART is enabled, the receive section is enabled, and receive DMA is enabled i.e. when signals UARTEN, RXE and RXDMAE are all high. It can also be enabled in test mode, in which case only TESTFIFO and RXDMAE have to be high.

The level within the receive FIFO is depicted by the signals RxIntLevel and RXFGTE1Full. The RxIntLevel is set when the data within the receive FIFO is more than programmable watermark level whilst the RXFGTE1Full is set when there is data within the receive FIFO. The receive DMA single request RXDMASREQ signal is set when RXFGTE1Full is active high. The receive DMA burst request RXDMABREQ is set when the receive FIFO function is enabled and the RxIntLevel is active high. Therefore, it is possible for both RXDMASREQ and RXDMASREQ to be active together.

The DMA controller has to be programmed with the most suitable burst length implied by the programmed receive FIFO watermark level.

The RXDMASREQ and RXDMABREQ signals are cleared by the application of the UARTRXDMACLR signal from the DMA controller, which signifies that the end of a single/burst transfer is about to occur. They are also cleared when the receive DMA function is disabled i.e. when RXDMAE is low.

4.4.12 Test logic

The block diagram is shown in Figure 20 Test logic. It contains the test mode registers in the UART, the logic for the loopback mode and the logic for the integration tests.

The integration test logic allows all inputs and outputs to be read from and written too.

The inputs can be read via the integration input (UARTITIP) register and the outputs can be written to via the integration output (UARTITOP) register.

All the inputs to the UART enter the UartTest block and are multiplexed with the signals from the UARTITIP register under the control of the integration test enable (ITEN) signal.

Similarly, all the UART outputs exit from the UartTest block and are multiplexed with the signals from the UARTITOP register under the control of the ITEN signal.

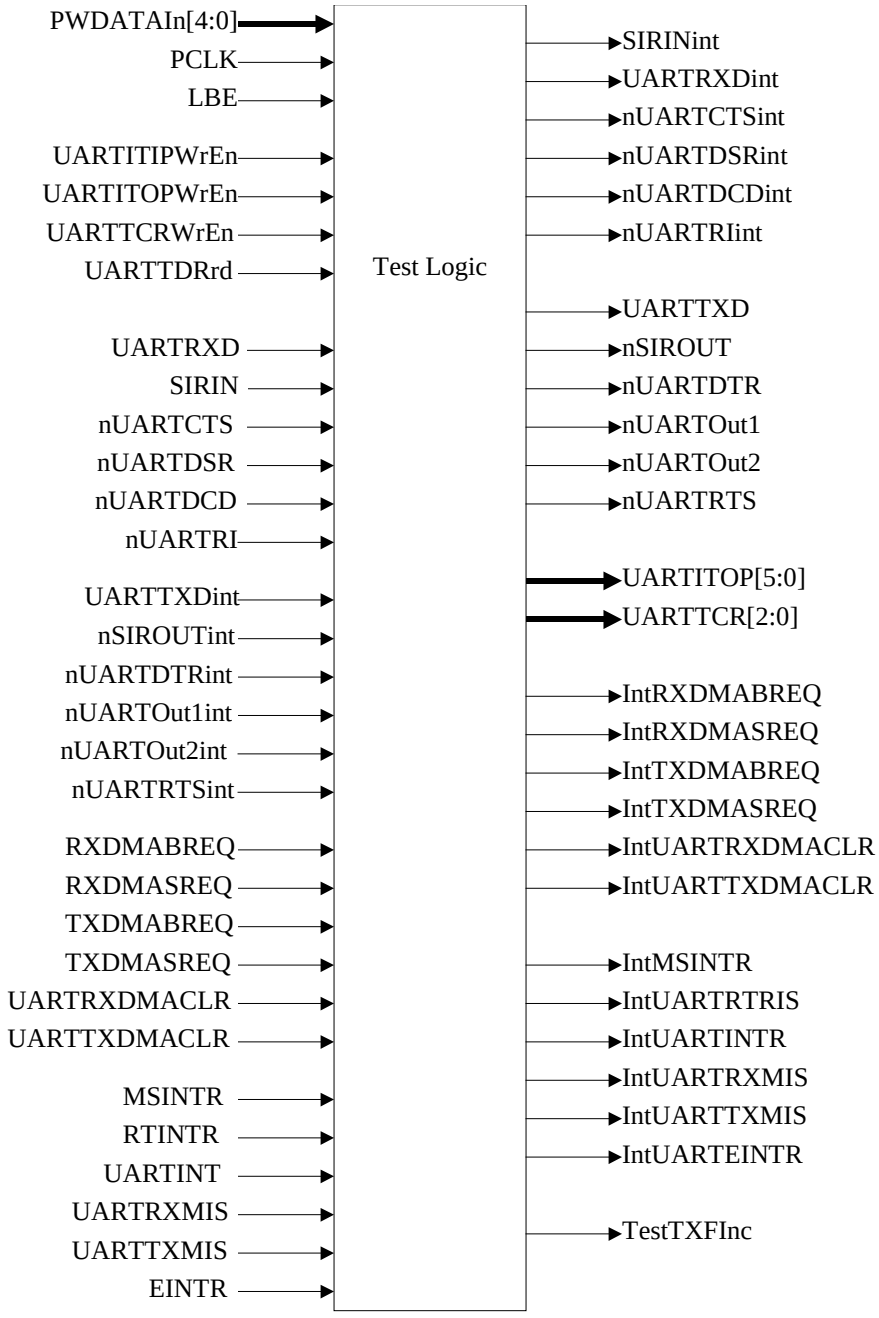

Figure 20 Test logic

4.4.13 Synchronisers

The synchronisers for signals crossing clock the two domains have been grouped into two sub-blocks:

- Signals crossing over from the PCLK domain into the UARTCLK domain (SynctoUARTCLK), as shown in Figure 21 Synchronisers for signals crossing into UARTCLK domain.
- Signals crossing over from the UARTCLK domain into the PCLK domain (SynctoPCLK), as shown in Figure 22 Synchronisers for signals crossing into PCLK domain.

The signals crossing clock domains are double-synchronised with flip-flops operating on the rising edge of the clock into which the signals get synchronised.

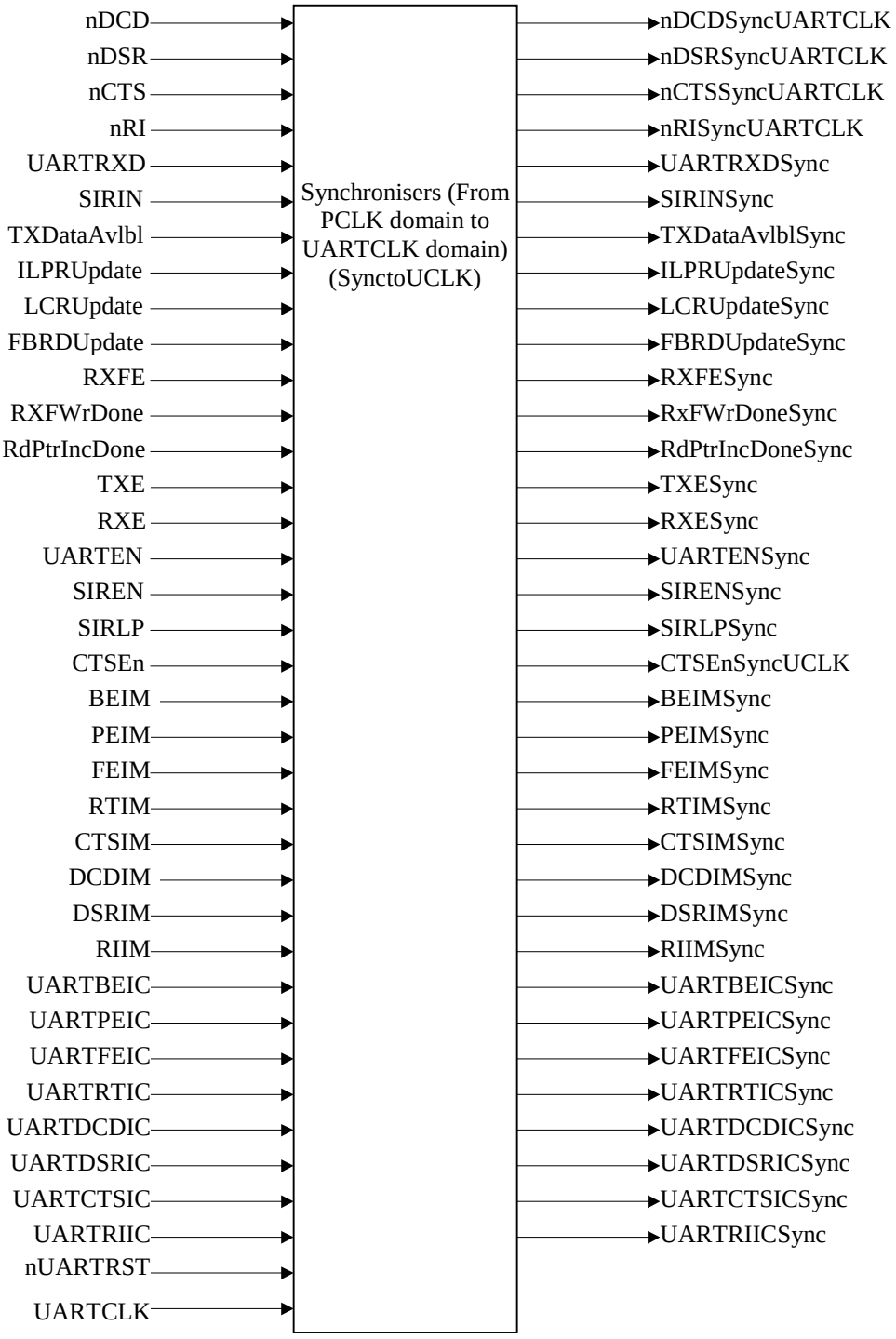

Figure 21 Synchronisers for signals crossing into UARTCLK domain

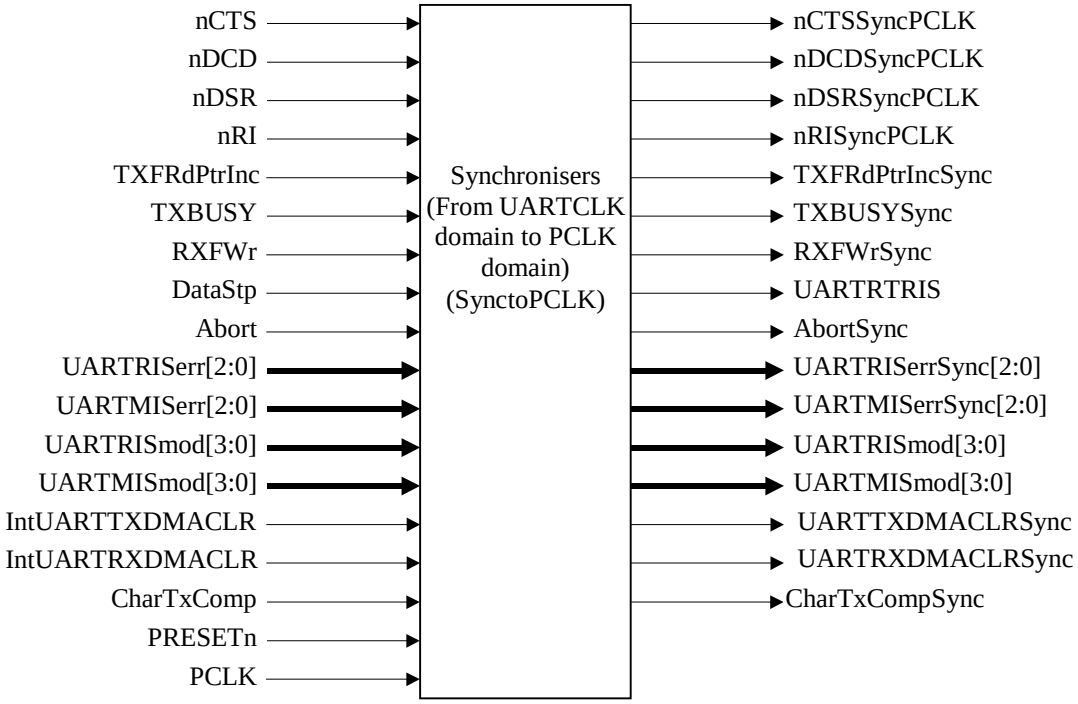

Figure 22 Synchronisers for signals crossing into PCLK domain

4.4.14 Revision Designator block

The block diagram is shown in Figure 23 Revision Designator block. It contains a single AND gate to be used as a place-holder cell to mark the Revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

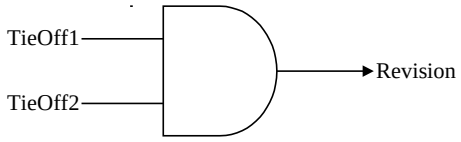

Figure 23 Revision Designator block

5 UART IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The design hierarchy in the UART is shown in Figure 24 UART Design Hierarchy. A short description about the functionality of each of the sub-blocks is also given within brackets.

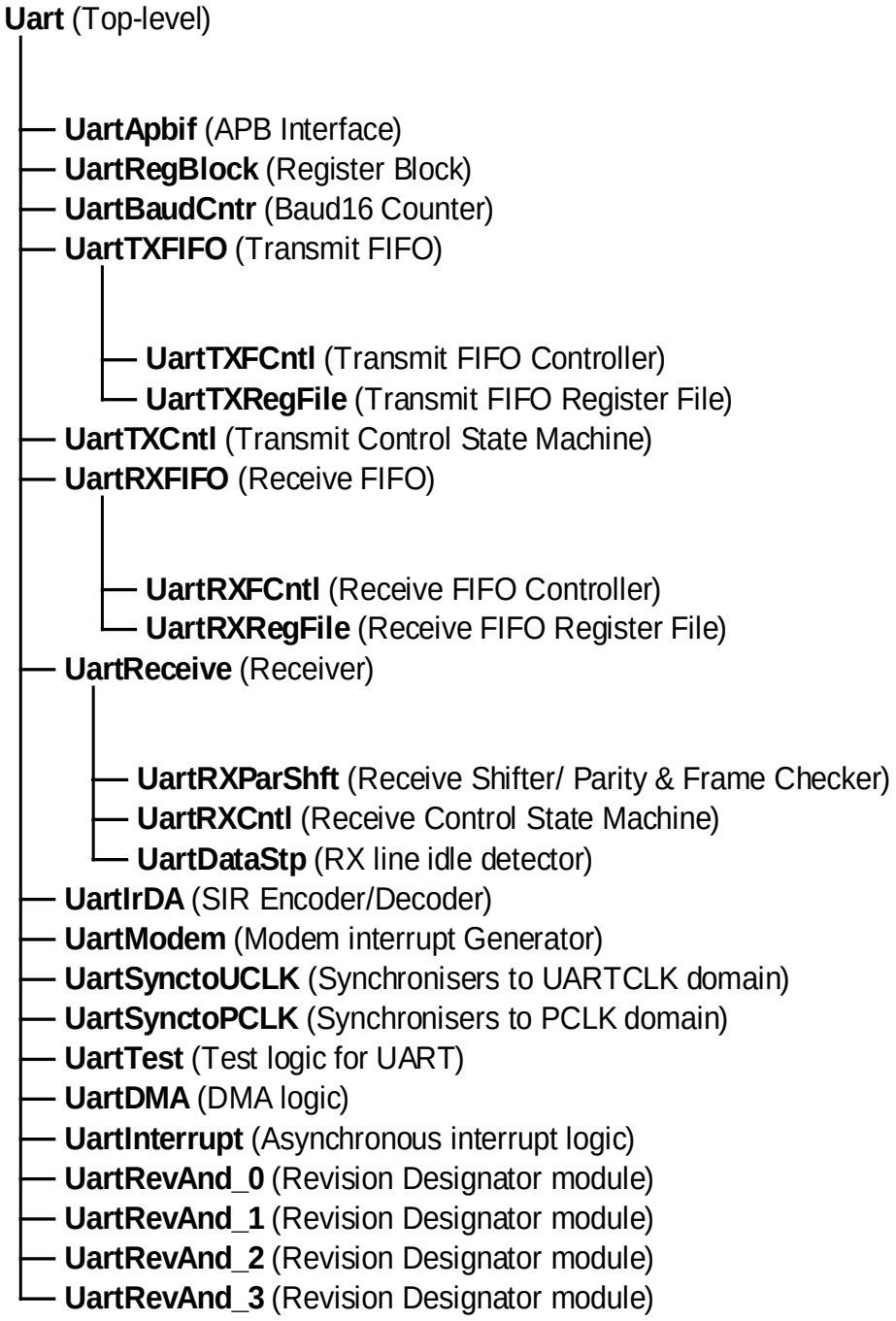

Figure 24 UART Design Hierarchy

5.2 UART Top Level Block (Uart)

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the UART.

5.3 UART APB Interface (UartApbif)

The APB Interface generates decodes for writes into the registers in the address space of the UART. It also contains the read interface for the UART.

An internal write data bus, PWDATAIn[15:0], is generated by gating the write data bus input to the UART, PWDATA[15:0], with PSEL and PWRITE. This prevents the data bus internal to the UART from toggling when the device is not being written into or when it is not selected.

5.3.1 Write Interface

The write strobe for each register is generated with a decode of PSEL, PWRITE, PADDR[11:2] and PENABLE. Data is clocked in to the selected register on the rising edge of PCLK on which the register Write enable is asserted.

5.3.2 Read Interface

The read interface involves an output multiplexer followed by a bank of flip-flops, which drive data onto the data bus. These flip-flops are clocked by the rising edge of PCLK. Select inputs to the output multiplexer are generated with a combinatorial decode of PWRITE, PADDR and PSEL signals. Since these signals are stable from the rising edge of PCLK, almost one full PCLK period is available for the output of the multiplexer to stabilise before data is clocked in on the next rising edge of PCLK. Data is sampled by the APB Bridge on the next rising edge of PCLK on which PENABLE is de-asserted.

The external data bus from multiple peripherals could be connected using one of many techniques so as to get a single read data bus driving the APB Bridge. The connection could be a multiplexer bus implementation or an OR bus implementation. For an OR bus implementation it is required that the peripheral drive zeros on the data bus when it is not selected. In the UART, the output multiplexer is left such that, when the device is not selected, it drives zeroes onto the read data bus. This ensures that there is no restriction placed on the type of external data bus connection used.

5.3.3 UARTRSR register read/write

Reads from the receive FIFO should be in a particular order – read UARTDR, then read UARTRSR for the status bits.

The UARTRSR register in the Register block is meant to store the error status bits corresponding to the byte last read from the receive FIFO. To implement this, the write decode for the UARTRSR register is generated by ANDing UARTDRrd signal (read from the UARTDR register i.e. the Receive FIFO) and the PENABLE signal. Thus, whenever there is a read from the UARTDR register, the status bits corresponding to that byte are latched into the UARTRSR register in the register block.

The Overrun bit in the UARTRSR register should be cleared when there is a write to the UARTRSR register. To implement this, the write decode for the UARTRSR register is fed to the Receive FIFO, where the Overrun condition is set, so that the OverrunDet signal from the FIFO can be cleared.

5.4 UART Register Block (UartRegBlock)

This block contains the normal mode registers in the UART. In addition, it contains the PCLK-domain intermediate buffers for the synchronisation of the UARTLCR_H, UARTIBRD, UARTFBRD and the UARTILPR registers to the UARTCLK domain. This block also generates the signals that clear the interrupts and the corresponding bit locations in the UARTICR register.

These clear signals are generated when a '1' is written into the corresponding bit location of the UARTICR register. For interrupts that are generated in the UARTCLK domain, the clear signal that is generated in UartRegBlock is synchronised to the UARTCLK domain. A delayed version of this signal is generated and a 1 UARTCLK wide pulse is generated when a rising edge on the clear signal is detected. This pulse clears the raw interrupt. The raw interrupt gets synchronised back to the PCLK domain and once it has been de-asserted, the corresponding bit location in the UARTICR register is cleared to 0.

5.4.1 Synchronisation of the UARTLCR_H and UARTIBRD registers to the UARTCLK domain

A 2-buffer technique is used to synchronise the contents of the UARTLCR_H and UARTIBRD registers to the UARTCLK domain. Out of the two buffers, the first stage is located in the PCLK domain in the Register Block (UartRegBlock) and the final stage is located in the UARTCLK domain in the Baud counter. (UartBaudCntr). It should be noted that the FEN (FIFOs enable) bit in the UARTLCR_H register does not need to be synchronised to the UARTCLK domain as the FIFOs are in the PCLK-domain. Consequently, the second stage buffer for the UARTLCR_H register, namely the LCRH signal, is only 6 bits wide, while the UARTLCR_H register is 7 bits wide.

When there is a write to the UARTIBRD register, the first stage buffers are updated with the new data written. The UARTLCRM buffer is updated with bits 15 down to 8 of the new data, and the UARTLCRL buffer is updated with bits 7 down to 0 of the new data.

When there is a write to the UARTLCR_H register, besides updating the content of the register, the LCRUpdate signal toggles to indicate to the UARTCLK domain that the synchronisation process has to start. This signal is double-synchronised to the UARTCLK domain and is seen as the LCRUpdateSync signal in the UARTCLK domain. A delayed version of this signal is generated by clocking this signal through a D-type clocked by UARTCLK. The delayed version is only updated once the current character has completed transmission or reception indicated by CharTxComp and CharRxComp being asserted. By XOR-ing the LCRUpdateSync signal with its delayed version an extended write pulse is generated. This pulse is then AND-ed with the same CharTxComp and CharRxComp signals which delays this pulse so that it only occurs once the character complete signals are valid and a load pulse that is one-UARTCLK wide is generated on the LCRStg2WrEn signal. When the LCRStg2WrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the UARTCLK domain. This ensures that the UARTIBRD and UARTLCR_H registers do not get updated until the current character has completely finished being transmitted or received.

5.4.2 Synchronisation of the UARTFBRD register to the UARTCLK domain

A 2-buffer technique is used to synchronise the contents of the UARTFBRD register to the UARTCLK domain. Out of the two buffers, the first stage is located in the PCLK domain in the Register Block (UartRegBlock) and the final stage is located in the UARTCLK domain in the Baud counter (UartBaudCntr).

When there is a write to the UARTFBRD register, the first stage buffer is updated with the new data written. Besides updating the content of the register, the FBRDUpdate signal toggles to indicate to the UARTCLK domain that the synchronisation process has to start. This signal is double-synchronised to the UARTCLK domain and is seen as the FBRDUpdateSync signal in the UARTCLK domain. A one-UARTCLK wide pulse is generated on the FBRDStg2WrEn signal in a similar way to the LCRStg2WrEn signal described in section 5.4.1. When the FBRDStg2WrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in

the UARTCLK domain This ensures that the UARTFBRD register does not get updated until the current character has completely finished being transmitted or received.

5.4.3 Synchronisation of the UARTILPR register to the UARTCLK domain

A 2-buffer technique is used to synchronise the contents of the UARTILPR register to the UARTCLK domain. Out of the two buffers, the first stage is located in the PCLK domain in the Register Block (UartRegBlock) and the final stage is located in the UARTCLK domain in the Baud counter (UartBaudCntr).

When there is a write to the UARTILPR register, the first stage buffer is updated with the new data written. Besides updating the content of the register, the ILPRUpdate signal toggles to indicate to the UARTCLK domain that the synchronisation process has to start. This signal is double-synchronised to the UARTCLK domain and is seen as the ILPRUpdateSync signal in the UARTCLK domain. A one-UARTCLK wide pulse is generated on the ILPRStg2WrEn signal in a similar way to the LCRStg2WrEn signal described in section 5.4.1. When the ILPRStg2WrEn signal is asserted, the contents of the first stage buffer are copied into the second stage buffer in the UARTCLK domain This ensures that the UARTILPR register does not get updated until the current character has completely finished being transmitted or received.

5.5 Baud Counter (UartBaudCntr)

This block consists of

- The final stage buffer for the UARTLCR_H, UARTIBRD registers
- The final stage buffer for the UARTFBRD register
- The final stage buffer for the UARTILPR register
- The Baud counters used to generate the Baud16 signal that serves as the main bit-timing reference for the transmit and receive blocks.
- The Baud counter used to generate the IrLPBaud16 signal that serves as the IrDA pulse width timing reference for the SIR endec block.

The final stage LCR buffers (LCRH[6:0], LCRM[7:0] and LCRL[7:0]) are loaded with the values on the input signals UARTLCRHInt[5:0], UARTLCRM[7:0] and UARTLCRL[7:0] when the LCRStg2WrEn signal is asserted. The LCRStg2WrEn signal is generated as described in section 5.4.1. The LCRH[5:0] output from this block conveys the transmission parameters to the transmit and receive sections. The LCRM and LCRL values are used as the integer value for the Baud counter that generates the Baud16 signal.

The final stage buffer (FBRD[5:0]) for the UARTFBRD register is loaded with the value on the UARTFBRD[5:0] input signal when the FBRDStg2WrEn signal is asserted. The FBRD value is used as the fractional value for the Baud counter that generates the Baud16 signal.

The ZeroBaud output signal is asserted when either LCRM, LCRL are zero, or when LCRM and LCRL are all 1’s and FBRD is not equal to zero, indicating to the transmitter and receiver sections that transmission and reception should be stopped since an illegal value has been programmed as the bit rate divisor.

The final stage buffer (ILPR[7:0]) for the UARTILPR register is loaded with the value on the UARTILPR[7:0] input signal when the ILPRStg2WrEn signal is asserted. The ILPR value is used as the reload value for the Baud counter that generates the IrLPBaud16 signal.

Baud16

The Baud16 output from the Baud Rate generator is a stream of pulses with an average frequency of 16 times the transmit/receive bit rate. Thus, on average, 16 Baud16 time periods equals one bit time of the transmit/receive bit stream.

The baud rate divisor is made up of a 16-bit integer part and a 6-bit fractional part.

Baud Rate Divisor = UARTCLK/(16xBaud Rate) = BRD_I + BRD_F Equation 1

where BRD_I is the integer part and BRD_F is the fractional part. This is as shown in Figure 25 The baud rate divisor.

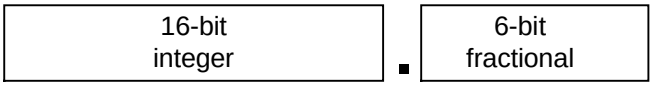

Figure 25 The baud rate divisor

The 16-bit integer is loaded via the UARTIBRD register and the fractional part loaded via the UARTFBRD register. The integer value will be referred to as ‘n’ in the following discussion while the fractional value will be referred to as ‘m’.

Integer Division

Assuming that the fractional part is zero, the integer part of the division is carried out by loading a 16-bit counter with ‘n’ and counting down until the counter reaches the value 1. The counter is now re-loaded with ‘n’ and a Baud16 signal is generated at this point. The Baud16 signal is held HIGH for one UARTCLK cycle.

The Baud16 signal is a stream of pulses with a width of one UARTCLK period and a frequency of UARTCLK/(BaudRateDivisor) on average. BaudRateDivisor is as given in Equation 1 above. To generate the Baud16 pulses, a LstCntBaud16bit1 signal is used to store the value of the least significant bit of the count on the previous UARTCLK. This signal is used to detect the counter counting down to one. Thus the iBaud16 signal is asserted when the current value of the counter is one and the counter value on the previous UARTCLK was not one, or when the integer divisor value is equal to one, indicated by the assertion of the LstCntBaud16bit1 signal. Figure 26 The generation of LstCntBaud16bit1 and iBaud16 shows when the LstCntBaud16bit1 signal is asserted and how iBaud16 is generated.

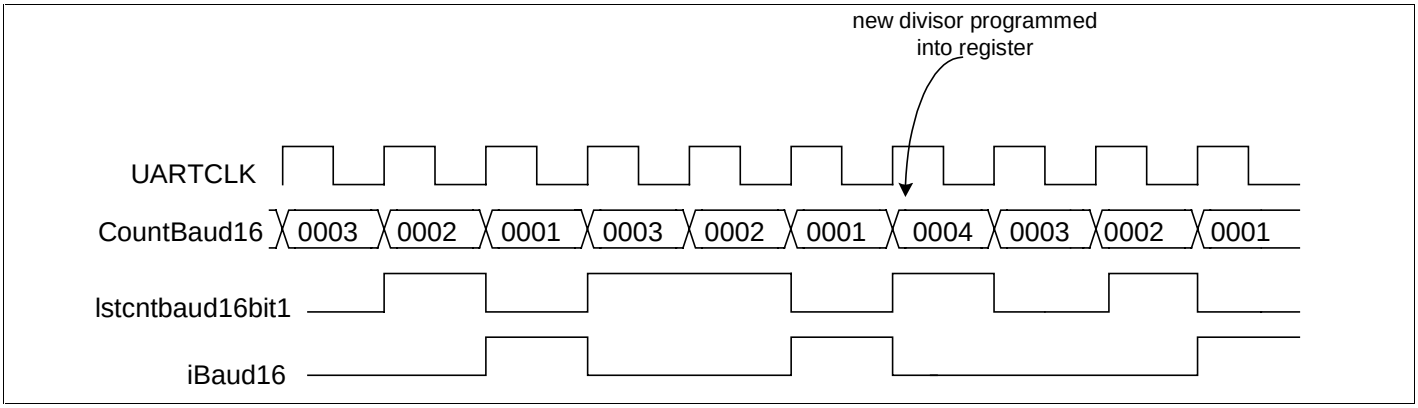

Figure 26 The generation of LstCntBaud16bit1 and iBaud16

Fractional Division

To implement a fractional division, the following equation has been implemented:

$$Baud16 = \left(\frac{UARTCLK}{n} * \frac{64 - m}{64}\right) + \left(\frac{UARTCLK}{n+1} * \frac{m}{64}\right)$$ Equation 2

When implemented this, will give a Baud16 series of pulses which on average have the following frequency:

$$Baud16 = \left(\frac{UARTCLK}{n + {}^{m}/_{64}} \right)$$ Equation 3

To implement this, the 16-bit counter needs to count (64-m) cycles of divide by ‘n’ and ‘m’ cycles of divide by ‘n+1’ in a total of 64 Baud16 cycles. This is implemented by having the 16-bit counter loaded with either ‘n’ or ‘n+1’ and every time it counts down a Baud16 pulse is generated; the value loaded depends on the state of a carry signal generated from a 6-bit counter.

When the Baud16 pulse is generated a 6-bit counter is incremented by ‘m’. Whenever the 6-bit counter rolls over (i.e. the value is greater than 63), a carry signal is generated and held HIGH for one Baud16 cycle. This carry signal is used to control the value re-loaded into the 16-bit counter the next time it has counted down to 1 and requires a re-load. With this combination, the above requirement of having (64-m) cycles of divide by ‘n’ and ‘m’ cycles of divide by ‘n+1’ in a total of 64 Baud16 cycles is satisfied.

Figure 27 Counting example when m=19 and n=3 shows an example sequence when the fractional part is non-zero:

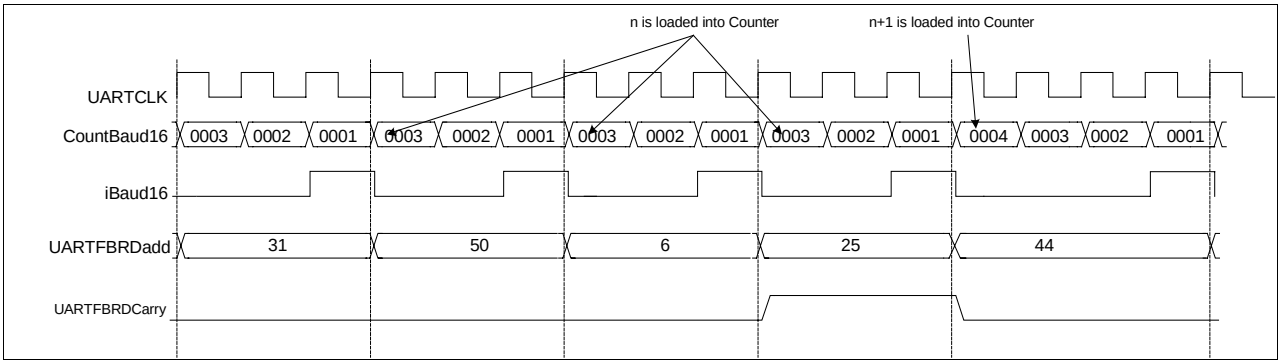

Figure 26 Counting example when m=19 and n=3

In addition to the above, the 16-bit counter is also reloaded with ‘n’ when the following condition occurs:

- When there is a write to the UARTIBRD or UARTFBRD registers as indicated by the assertion of the LCRStg2WrEn signal. A delayed version (DelLCRStg2WrEn) of this signal is generated by clocking LCRStg2WrEn through a flip-flop clocked by UARTCLK. The assertion of the DelLCRStg2WrEn signal means that the reload value has changed. If the counter is not reloaded, then it would lead to a delay in the change in the reload value taking effect. This delay could be significant in case the previous reload value is extremely large because the counter could take a long time to count down to zero.
- When ZeroBaud is asserted indicating that an illegal divisor value has been programmed into the bit rate divisor registers (UARTIBRd and UARTFBRD).

The 16-bit counter is reloaded with ‘n’, and the 6-bit counter reloaded with the value in FBRD when one of the following conditions occur:

- When the StopBaudCnt signal is asserted. The StopBaudCnt signal is asserted when the transmit side of the UART is not enabled i.e. when the UARTENSync and TXESync input signals are not asserted, and the transmit state machine is in the idle state, and when the receive side of the UART is not enabled i.e. when the UARTENSync and RXESync input signals are not asserted, and the receive state machine is in the idle state. The state machines must return to the idle state before the Baud16 signal stops to ensure that the current character finishing being transmitted or received. If the counter was not reloaded, the next time the UART was enabled it would start counting from the old value.

The above is illustrated in Figure 27 Fractional Baud Rate Division:

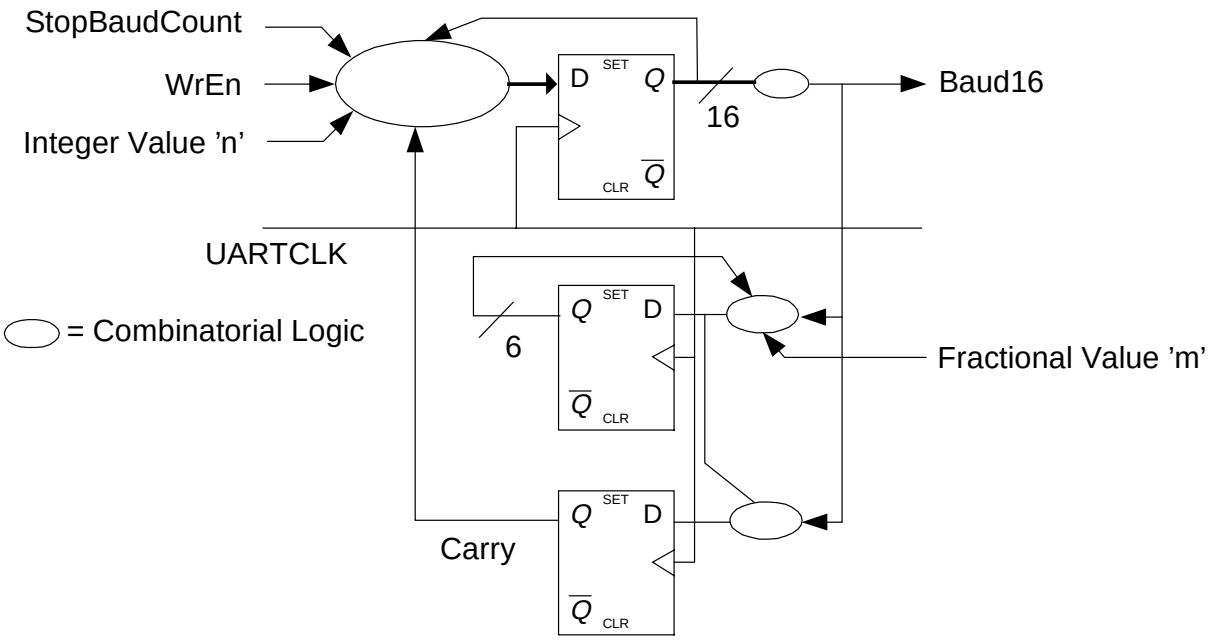

Figure 27 Fractional Baud Rate Division

Note Figure is for illustration only and does not show the full implementation details. For this, the reader should refer to the RTL.

IrLPBaud16

The IrLPBaud16 signal is a pulse stream with a time period approximately equal to 2 MHz. The exact value of the time period of this signal depends on the value programmed into the UARTILPR register. The Baud counter (SIRLPC) used to generate the IrLPBaud16 signal is 8 bits wide and operates on UARTCLK . This counter is reloaded when any of the following conditions occurs:

- When the counter is at a count of one. This is the normal case of reloading the counter on underflow.
- When there is a write to the UARTILPR registers as indicated by the assertion of the ILPRStg2WrEn signal. A delayed version (DelILPRStg2WrEn) of this signal is generated by clocking ILPRStg2WrEn through a flip-flop clocked by UARTCLK. The assertion of the DelILPRStg2WrEn signal means that the reload value has changed. If the counter is not reloaded, then it would lead to a delay in the change in the reload value taking effect. This delay could be significant in case the previous reload value is large because the counter could take a long time to count down to zero.
- When the StopBaudCnt signal is asserted. The StopBaudCnt signal is asserted when the transmit side of the UART is not enabled i.e. when the UARTENSync and TXESync input signals are not asserted, and the transmit state machine is in the idle state, and when the receive side of the UART is not enabled i.e. when the UARTENSync and RXESync input signals are not asserted, and the receive state machine is in the idle state. The state machines must return to the idle state before the Baud16 signal stops to ensure that the current character finishing being transmitted or received. If the counter was not reloaded, the next time the UART was enabled it would start counting from the old value.
- When the StopLPCntr signal is asserted and SIRENSync and SIRLPSync are both deasserted. The StopLPCntr signals is asserted when SIRENSync and SIRLPSync are both deasserted and CharTxComp and CharRxComp are both asserted indicating that the current character has completed. This causes the low power counter to stop counting only when the current word has completed, and to start counting as soon as either SIRENSync or SIRLPSync is asserted.

The LstSIRLPCBit0 signal is used to store the value of the Least Significant Bit of the count on the previous UARTCLK. This signal is used to detect the counter counting down to zero. Thus, the IrLPBaud16 signal is asserted when the current value of the counter is zero and the previous value was one. The IrLPBaud16 signal is used in timing the pulse width of the IrDA bit stream in the low power mode.

The Abort signal is asserted when one of the following conditions is satisfied:

- An illegal value is written into the Bit rate divisor registers UARTIBRD and UARTFBRD
- The 'break' bit in the UARTLCR_H register is set.

The Abort signal is synchronised to the PCLK domain and is seen as the AbortSync signal by the Transmit FIFO. This signal is used by the transmit FIFO to control data writes in to the Transmit Shift register. Although transmission is aborted when the UART is disabled, the UARTENSync signal is not used in the generation of the Abort signal because the UARTEN signal in the PCLK domain is available as a block input to the Transmit FIFO, which also operates on PCLK.

5.6 Transmit FIFO (UartTXFIFO)

The Transmit FIFO block instantiates the transmit FIFO control logic (UartTXFCntl) and the data storage elements (UartTXRegFile). Writes to the transmit FIFO occur from the APB through the APB interface, which operates on PCLK. Reads from the Transmit FIFO occur from the Transmit logic, which operates on UARTCLK.

The transmit FIFO is 8-bits wide and 16-deep. The FIFO can be disabled if required. In the FIFO disabled mode, one byte in the FIFO is used as the holding register. The transmit shift register is located in the FIFO and is common to the FIFO enabled mode and the FIFO disabled mode.

The APB interface in the UART drives the UARTDRWrEn input to the transmit FIFO with the decode of writes to the UARTDR register. The UARTDRWrEn input pulse is one PCLK wide. When the UARTDRWrEn input is asserted, the data on the PWDATAIn[7:0] input is latched into the Register file (UartTXRegFile) location pointed to by the current value of the write pointer. The write pointer is controlled by the UartTXFCntl block. The Read pointer is also generated by the control logic. It points to the next location in the Transmit Register file, whose contents will be driven into the Transmit Shift register located within the control logic (UartTXFCntl). The contents of the shift register are driven onto the TXShiftData[7:0] output of the FIFO.

The TXDataAvlbl output signal indicates to the transmit logic that valid data is available in the FIFO for transmission. When the transmit section has completed transmission of one byte, it asserts the Read pointer increment signal which is seen as the TXFRdPtrIncSync input signal. After the Transmit FIFO control logic has completed the incrementing operation, it asserts the RdPtrIncDone (Read pointer increment done) signal. On detecting this signal, if there is further valid data available in the FIFO, the transmit section starts transmitting the data driven onto the TXShiftData[7:0] lines which correspond to the next data byte in the FIFO.

Thus writes, which occur from the PCLK domain occur without any handshake signals, but reads from the FIFO, which occur from the UARTCLK domain, operate with suitable handshake signals through synchronisers.

The UARTEN input signal is required to control writes into the transmit shift register. The AbortSync or BRK signal is used to check whether the transmission of a particular data byte has been aborted before completion. This information is also used in the updating of the Transmit shift register. The FEN (FIFO enable) input signal is used by the control logic to manipulate the status signals and the FIFO suitably.

The TXFF (Transmit FIFO Full) signal indicates the fill status of the FIFO. When the FIFO is enabled, the assertion of the TXFF signal indicates that there are 16 bytes in the transmit FIFO. When the FIFO is disabled, the assertion of TXFF indicates that the holding register is full. This signal is readable through the 'TXFF' bit in the UARTFR register.

The raw and masked status of the Transmit Interrupt, UARTTXINTR, is generated in the UartTXFCntl block. When the FIFO is enabled, the transmit interrupt is asserted if the FIFO fill level becomes less than or equal to the programmed interrupt level. The programmed interrupt level is set by writing to the UARTIFLS register. When the FIFO is disabled, the transmit interrupt signal is asserted when the holding register is empty. The interrupts are

de-asserted when the condition creating the interrupt is no longer present, or a 1 is written to the corresponding bit in the interrupt clear register (UARTICR).

5.7 Transmit FIFO control logic (UartTXFCntl)

The Transmit FIFO control logic controls the read and write pointers for the FIFO. It also generates the FIFO fill level status signals and the raw and masked Transmit interrupt status signals, UARTTXRIS and UARTTXMIS. The UARTDRWrEn input is a one-PCLK-wide pulse driven by the write decode to the UARTDR address. The TXFRdPtrIncSync signal is an indication originating from the transmit section that the read pointer can be incremented. The FEN signal indicates whether the FIFO is enabled or disabled. If the FIFO is disabled, one byte in the transmit FIFO is used as the transmit holding register. The TXFIFOData[7:0] input is the content of the Register File being pointed to by the current value of the read pointer. This data is copied on to the transmit Shift register when the transmit section has completed the transmission of the current data byte. The UARTEN input signal indicates whether data transmission is currently enabled or not. For transmission to occur the TXE signals must also be asserted. These signals are used to determine whether the next write should go into the FIFO or straight into the transmit shift register. The AbortSync or BRK input signal indicates whether the transmit section has aborted the transmission of any further data being transmitted.

The WrPtr[3:0] output signal and the RdPtr[3:0] output signals indicate the current value of the write pointer and the read pointer respectively. The TXFF signal indicates whether the transmit FIFO is full or not. The TXFE signal indicates whether the transmit FIFO is empty or not. The TXDataAvlbl signal indicates whether data is available in the transmit FIFO for transmission. The TXFRdPtrIncDone signal indicates the completion of the read pointer increment operation. The TXShiftData[7:0] output is the data currently being transmitted by the transmitter. The UARTTXRIS and UARTTXMIS signals are the Transmit raw and masked interrupt signals. The UARTTXIClr signal clears the raw interrupt status and also the Transmit interrupt bit in the UARTICR register.

A FlushFifo signal is generated when the FIFO enable signal is de-asserted after the UART is disabled. This is used to clear any data, which is in the FIFO.

A data byte from the FIFO has to be loaded into the transmit shift register when the current character has finished being transmitted and any of the following conditions occur:

- When there is a rising edge detected on the TXFRdPtrIncSync input signal. The Shift register needs to be filled with the next data byte only if the UART is enabled i.e. UARTEN and TXE are asserted, and there is no Abort condition.
- When the Abort condition has just been de-asserted with the UART already enabled i.e. UARTEN and TXE are asserted or UART transmit logic has just been enabled and there are no abort conditions

The rising edge on the TXFRDPtrIncSync signal, the UARTEN and the TXE signal and the falling edge on the AbortSync signal are detected by clocking these signals into flip-flops within the Transmit FIFO control logic and thus generating delayed versions of those signals. When any of the above conditions are satisfied the LoadShiftReg signal is set to ‘1’.

The TXShiftRegE internal signal in this block indicates whether the next write to the transmit FIFO should fill data directly into the transmit shift register. The following considerations are applicable while generating this signal:

- If the UART is disabled, or if the Abort condition exists, then any data that is written should accumulate in the FIFO and the shift register should not be written to.
- If the FIFO is empty and if the LoadShiftReg signal is asserted without a simultaneous write access to the FIFO, then the Shift register is empty.
- If there is a write access, the Shift register is no longer empty.

The TXShiftRegEFE signal detects an edge on TXShiftRegE when the UART is enabled i.e. UARTEN and TXE are asserted. This signal is used to detect when a word is written straight to the Shift register in the FIFO disabled mode.

The TXShiftData[7:0] signal represents the transmit shift register. The following considerations apply when generating this signal:

- If there is a data write access to the FIFO with the LoadShiftReg signal being asserted and the FIFO being empty, data on the PWDATAIn[7:0] bus is written directly into the shift register. If there is a data write access to the FIFO and the Shift register is already empty as indicated by the assertion of the TXShiftRegE signal, then too, data on the PWDATAIn[7:0] bus is written into the Shift register.
- If the LoadShiftReg signal is asserted and the FIFO is not empty, then copy the next data byte available on the TXFIFOData[7:0] bus into the shift register.

The ShiftDataAvlbl signal, when asserted, indicates that the Shift register contains valid data that is being currently transmitted by the Transmit logic. This signal is used in the generation of the TXDataAvlbl output signal. The following considerations apply when generating this signal:

- If the transmit FIFO is empty with no data write access to the FIFO and the LoadShiftReg signal is asserted, the shift register is said to be empty and the ShiftDataAvlbl signal is cleared.
- If UART is disabled or BRK is asserted and a rising edge on CharTxCompSync signal clears ShiftDataAvlbl.
- A high on the signal FlushFifo clears ShiftDataAvlbl.
- If the LoadShiftReg signal is asserted with the FIFO being non-empty or if the LoadShiftReg signal is asserted with the FIFO being empty but there is a simultaneous data write access to the FIFO or if the TXShiftRegE signal is asserted with a simultaneous data write access to the FIFO, the shift register is loaded with valid transmit data. Hence, the ShiftDataAvlbl signal is set.

The WrPtrIncValid signal is asserted when a valid write access occurs to the FIFO or to the Holding buffer. The following considerations apply when generating this signal:

- When there is a write access to the Transmit FIFO, the write data is clocked into the FIFO only if the FIFO is not already full. Also, if the FIFO and the Shift register are empty with the UART being enabled and the Abort condition being de-asserted, then a write access to the Transmit FIFO should not accumulate in the FIFO, but should be directly clocked into the Transmit shift register. In normal operation, the Interrupt service routine that writes data into the FIFO is not expected to write more data than is required to fill the FIFO. This check is done to protect from such writes in case they occur.

The WrPtr[3:0] signal is used as the write pointer signal. The following considerations apply when generating this signal:

- When the FIFO is disabled, keep the write pointer at zero. The HldBufValid signal is used to keep track of the fill status of the holding buffer when the FIFO is disabled. When the FIFO is disabled, the pointers are reset to zero. When the FIFO-disabled mode is entered, the HldBufValid signal is at ‘0’, thus indicating that the FIFO is empty.
- When the FIFO is enabled, increment the Write pointer every time the WrPtrIncValid signal is asserted.

The RdPtr[3:0] signal is the Read pointer for the FIFO. The following considerations apply when generating this signal:

- If the FIFO is disabled, then keep the read pointer at zero.
- If the LoadShiftReg signal is asserted and the FIFO is not already empty, then increment the read pointer by 1.

The FIFOFillLevel[4:0] signal indicates the number of locations in the FIFO that have been filled. This signal takes a range of values starting with zero, when the FIFO is empty, one when there has been one write to the transmit FIFO and upto sixteen when the FIFO has all its sixteen locations filled. The FIFOFillLevel is found by subtracting the value of the read pointer from the value of the write pointer with the ‘Wrap’ bit prepended to it. There exists a possibility that the write pointer increments through “1111” to “0000”. The direct difference between the pointers will not give the fill level. To take this into account, the ‘Wrap’ signal has been introduced.

The Wrap signal is set when the Write pointer has wrapped around but the read pointer hasn't. The signal is reset when the read pointer also subsequently wraps around. The Wrap signal is prepended to the write pointer and from the resultant value, the read pointer is subtracted to determine the fill level of the FIFO.

The following considerations apply when generating the Wrap signal:

- The WrPtrIncValid signal is asserted when the write pointer is to be incremented. The RdPtrIncValid signal is asserted when the read pointer is about to be incremented. When the write pointer is at "1111" and the WrPtrIncValid signal is asserted and the RdPtrIncValid signal is not asserted then set the Wrap.
- When the read pointer is at "1111" and the RdPtrIncValid signal is asserted, and the WrPtrIncValid signal is not asserted, clear Wrap. To combine this condition with the previous condition, toggle Wrap when either of the two conditions occur.
- When both the pointers are at "1111" with the FIFO being full (Wrap already set) and both the WrPtrIncValid signal and the RdPtrIncValid signal are asserted, then the FIFO is still full, but retain the value of the Wrap bit i.e. let it remain set, thus indicating that the FIFO is still full.

5.8 Transmit FIFO Register File (UartTXRegFile)

The Register File is implemented as a bank of D-types. This block can be replaced with a compiled memory if required. The Register File also contains a 4-to-16 decoder to decode the Write pointer and a 16-to-1 bus multiplexer for the read data bus.

5.9 Transmit Control state machine (UartTXCntl)

The transmit section performs parallel to serial conversion on the transmit data and drives the TXD line with the bits to be transmitted at the programmed bit period. It also contains the logic to provide independent transmit and receive hardware flow control.

The WLEN[1:0] input signal indicates the number of bits to be transmitted. When the TXDataAvlblSync signal is asserted, data transmission starts on the next Baud16 pulse. The control logic in the UartTXCntl block contains a 3 bit counter BitCnt, which counts the number of data bits transmitted. During transmission, the TXBUSY signal is pulled high. After the last stop bit corresponding to one byte of data has been transmitted, the TXFRdPtrInc signal is asserted. After the Transmit FIFO responds with the RdPtrIncDone signal, transmission proceeds if more data is available in the transmit FIFO as indicated by the TXDataAvlblSync input signal. The frequencies of PCLK and UARTCLK should be such that the handshake between the transmitter and the Transmit FIFO is completed within the duration of the stop bit. Hence, the transmission of the next data byte can start immediately after the end of the stop bit of the current byte.

The states in this state machine are:

- ST_IDLE. Idle State
- ST_DATAAVLBL. State to wait for the next Baud16 pulse after data has become available
- ST_STARTBIT. State to time the Start bit duration.
- ST_SHIFT. State where data bits are shifted out one by one and their bit periods are timed.
- ST_PARITY. State where the duration of the Parity bit is timed
- ST_XSTOP. State where the extra stop bit duration is timed.
- ST_STPRDPTRINC. State where the stop bit is timed and interaction with the Transmit FIFO occurs.
- ST_BREAK. State in which the state machine stays when BRK bit is set.
- ST_EXITBREAK. State to wait for one bit time after BRK is released.

The AbortTransmit signal used in the state machine is used to transition to the ST_STPRDPTRINC state after any of the following conditions is satisfied. The UARTLCR_H , UARTIBRD and UARTFBRD registers are not updated

until the end of the current character so the AbortTransmit signal does not get asserted until the end of the current character.

- An illegal value is written into the Bit rate divisor registers UARTIBRD and UARTFBRD. This is detected by the assertion of the ZeroBaud signal.
- The ‘BRK’ bit in the UARTLCRH register is set.

If the AbortTransmit signal is asserted before the start bit of a character has been shifted out, the state machine transitions to the ST_IDLE state since transmission has not started.

The nBreak signal is the inverted version of the BRK bit. This signal is used to pull the TXD line low when the BRK bit is set.

The CharTxComp signal is used to indicate that the transmission of the current character has completed.

The main registers and counters used in this state machine are:

- TXDReg. The output of this register drives the transmit serial line.
- BitCount[2:0]. This register keeps track of the number of bits that have been transmitted.
- BitPeriodCnt[3:0]. This register is used to time the duration of the transmit bit period by counting the number of Baud16 pulses received.

The main comparators used in this state machine are:

- BitCountComp. This comparator checks if the BitCount[2:0] counter has reached the number of data bits programmed in the encoded WLEN[1:0] bits. This is used to check whether the required number of bits in the current byte have been transmitted.
- BitPeriodCmp. This comparator checks if the BitPeriodCnt[3:0] counter has counted down to zero. This is used to check whether 16 Baud16 pulses have arrived after a bit has been shifted out.

Figure 28 Transmit state machine: Idle, Data Available, StartBit, shows the respective state transitions.

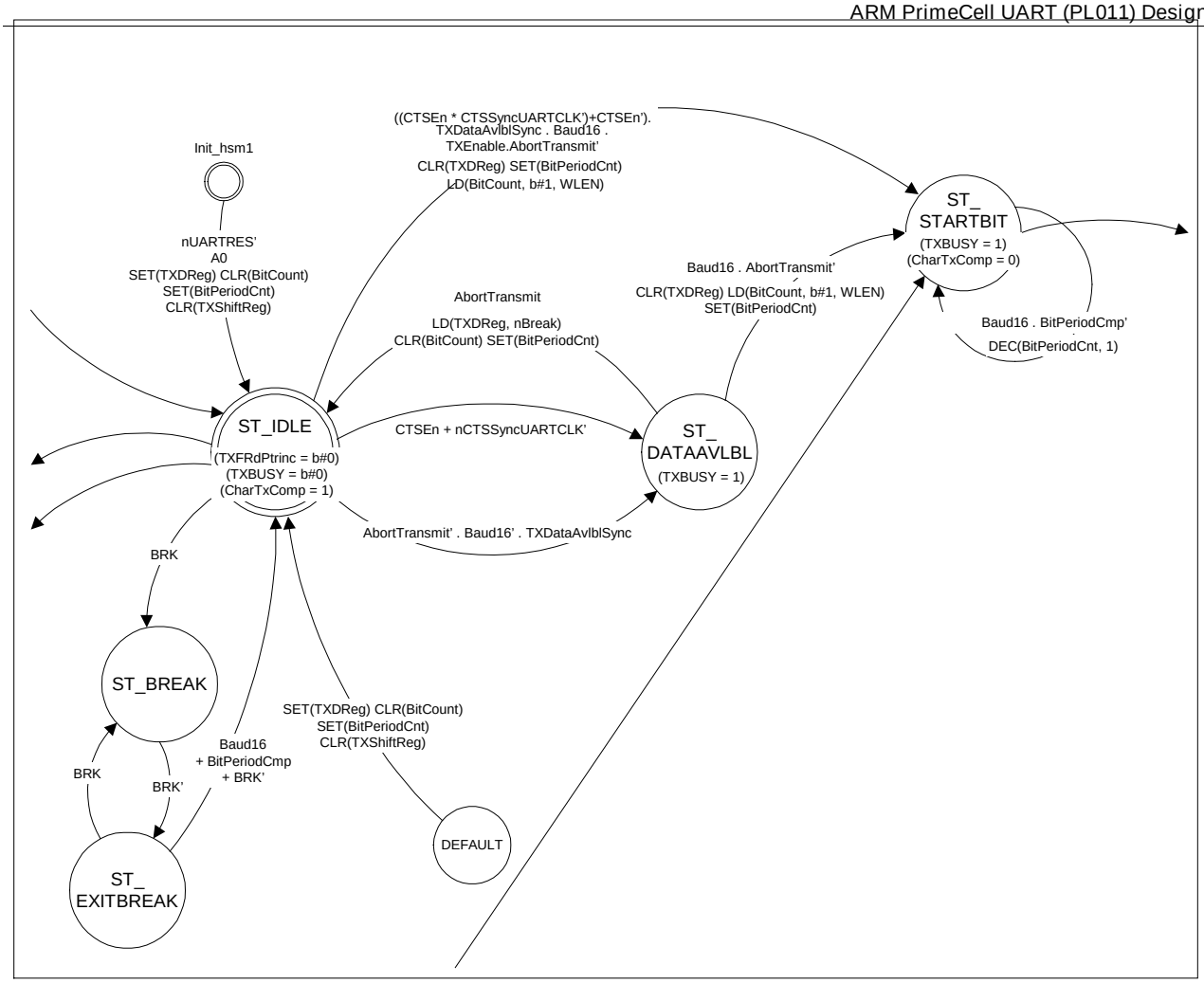

Figure 28 Transmit state machine: Idle, Data Available, StartBit and Break

When nUARTRST is asserted, the state machine enters the ST_IDLE state, the TXDReg bit is set to the inactive transmit level of ‘1’, the CharTxComp signal is set to ‘1’, the BitCnt counter and the TXShiftReg are cleared to zero and the BitPeriodCnt counter is set to all ones.

When TXDataAvlblSync is asserted, it indicates that transmit data is available in the FIFO. To ensure the correct bit period, the state machine synchronises the transmission of bits to the Baud16 signal. If a Baud16 pulse arrives when the TXDataAvlblSync signal is set, the start bit is shifted out and the state machine enters the ST_STARTBIT state. If CTSEn is asserted, the start bit will only be shifted out and the state machine change to state ST_STARTBIT when nCTSSyncUARTCLK is asserted low, otherwise it will remain in ST_IDLE. If the Baud16 signal is not present when the TXDataAvlblSync signal is asserted, the state machine enters the ST_DATAAVLBL state and remains in this state till the next Baud16 pulse arrives. If CTSEn is asserted, the state machine only enters the ST_DATAAVLBL state when nCTSSyncUARTCLK is asserted low , otherwise it remains in the ST_IDLE state. While in the ST_DATAAVLBL state, if AbortTransmit is asserted, the state machine enters the ST_IDLE state since transmission has not started. This behaviour is different from the case when AbortTransmit is asserted while in other states. If the abort is due to BRK bit being set, the TXDReg is loaded with 0 to send a Break.

As the ST_STARTBIT state is entered, the TXDReg line is made ‘0’ indicating a start bit, and the CharTxComp signal is made ‘0’ indicating that a character has begun transmission. The BitPeriodCnt counter is loaded with “1111”. In the ST_STARTBIT state, every time a Baud16 pulse is seen, the BitPeriodCnt counter decrements by 1 till the counter reaches zero. When the counter reaches zero, it transitions to the ST_SHIFT state. The section of the state machine where data is shifted out is shown in Figure 29 Transmit state machine: StartBit and Shift.

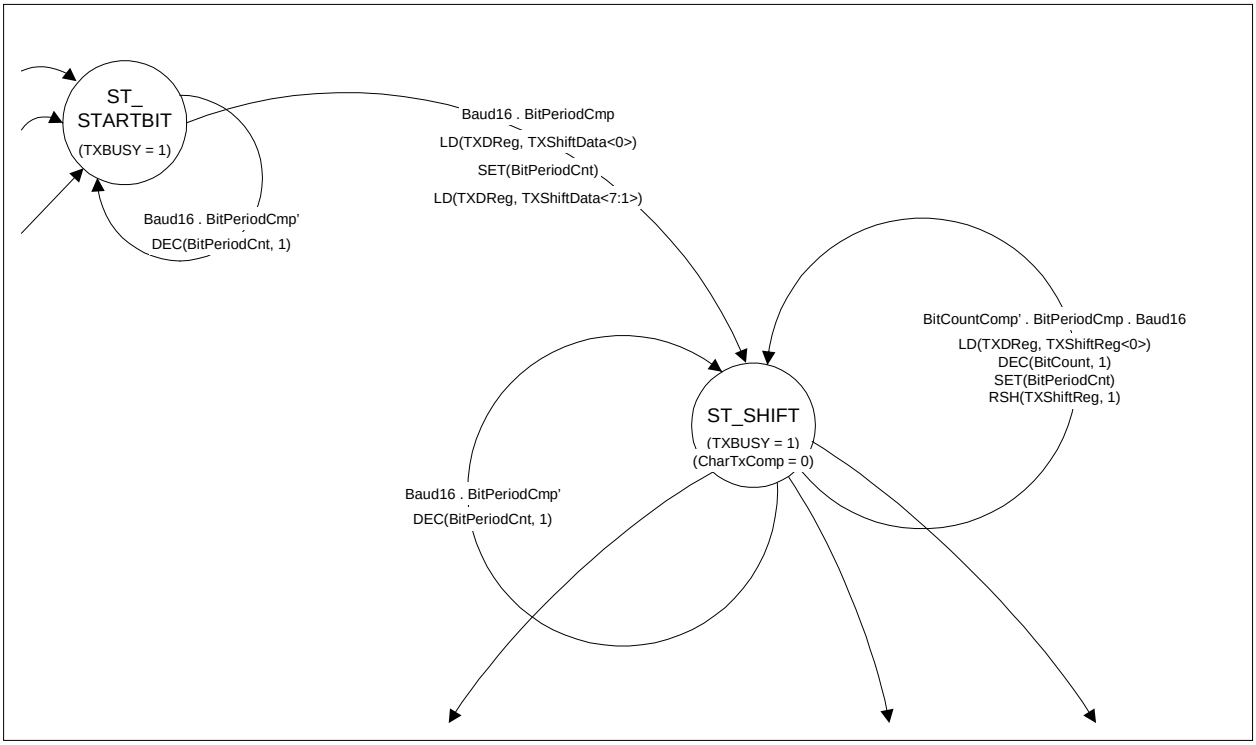

Figure 29 Transmit state machine: StartBit and Shift

After the start bit has been timed, the state machine enters the ST_SHIFT state. At this transition, the TXDReg is loaded with the least significant bit in the transmit data on the TXShiftData[7:0] signal, the Transmit shift register, TXShiftReg[6:0] is loaded with bits [7:1] of the transmit data on the TXShiftData[7:0] input and the BitPeriodCnt counter is reloaded with “1111”. In the ST_SHIFT state, the BitPeriodCnt counter decrements by 1 with every Baud16 pulse. When the BitPeriodCnt counter reaches zero, a bit time has elapsed and hence the BitCnt counter is decremented. Also, the next bit to be transmitted is loaded into TXDReg and BitPeriodCnt is reloaded with “1111”.

The section of the state machine where the parity and stop bits are timed is shown in Figure 30 Transmit state machine: Shift, Parity, Xstop & StpRdPtrInc. When in the ST_SHIFT state, if the bit time corresponding to the last data bit has elapsed:

- If Parity is enabled, the parity bit is loaded into the TXDReg.

  The parity bit is dependent upon the Stick Parity Select (SPS) and Even Parity Select (EPS) bits that have been programmed in the UARTLCR_H register. If SPS is low i.e. not in force, then the even or odd resultant parity bit is loaded into TXDReg. If SPS is in force, then if EPS is low, a “1” is loaded in the TXDReg, else if EPS is high, a “0” is loaded into the TXDReg. Also, the BitPeriodCnt counter is loaded with “1111” and the state machine transitions to the S_PARITY state. Table 1 summarises the behaviour of stick parity.

- If Parity is disabled, but 2 stop bits are required, the TXDReg is set for indicating a stop bit (which is in addition to the mandatory stop bit), the BitPeriodCnt counter is loaded with “1111” and the state machine transitions to the ST_XSTOP state.

- If neither parity nor 2 stop bits are enabled, the TXDReg is set for indicating the (mandatory) stop bit, the BitPeriodCnt counter is loaded with “1111” and the state machine transitions to the ST_STPRDPTRINC state.

When the state machine is in the ST_PARITY state, the BitPeriodCnt counter decrements by 1 with every Baud16 pulse. When the BitPeriodCnt counter has counted down to 0, it indicates that the bit time has elapsed:

- If 2 stop bits are required, the TXDReg is set for indicating a stop bit (which is in addition to the mandatory stop bit), the BitPeriodCnt counter is loaded with “1111”, the CharTxComp signal is set to ‘1’ and the state machine transitions to the ST_XSTOP state.

- If only one stop bit is required, the state machine transitions to the ST_STPRDPTRINC state after loading the BitPeriodCnt counter with “1111”, the CharTxComp with 1 and the TXDReg signal with 1 to indicate the mandatory stop bit.

When the state machine is in the ST_XSTOP state, the BitPeriodCnt counter decrements by 1 with every Baud16 pulse. When the BitPeriodCnt counter has counted down to 0, it indicates that the bit time has elapsed. The TXDReg signal is set to 1 and the BitPeriodCnt counter is loaded with “1111”. The state machine then transitions to the ST_STPRDPTRINC state.

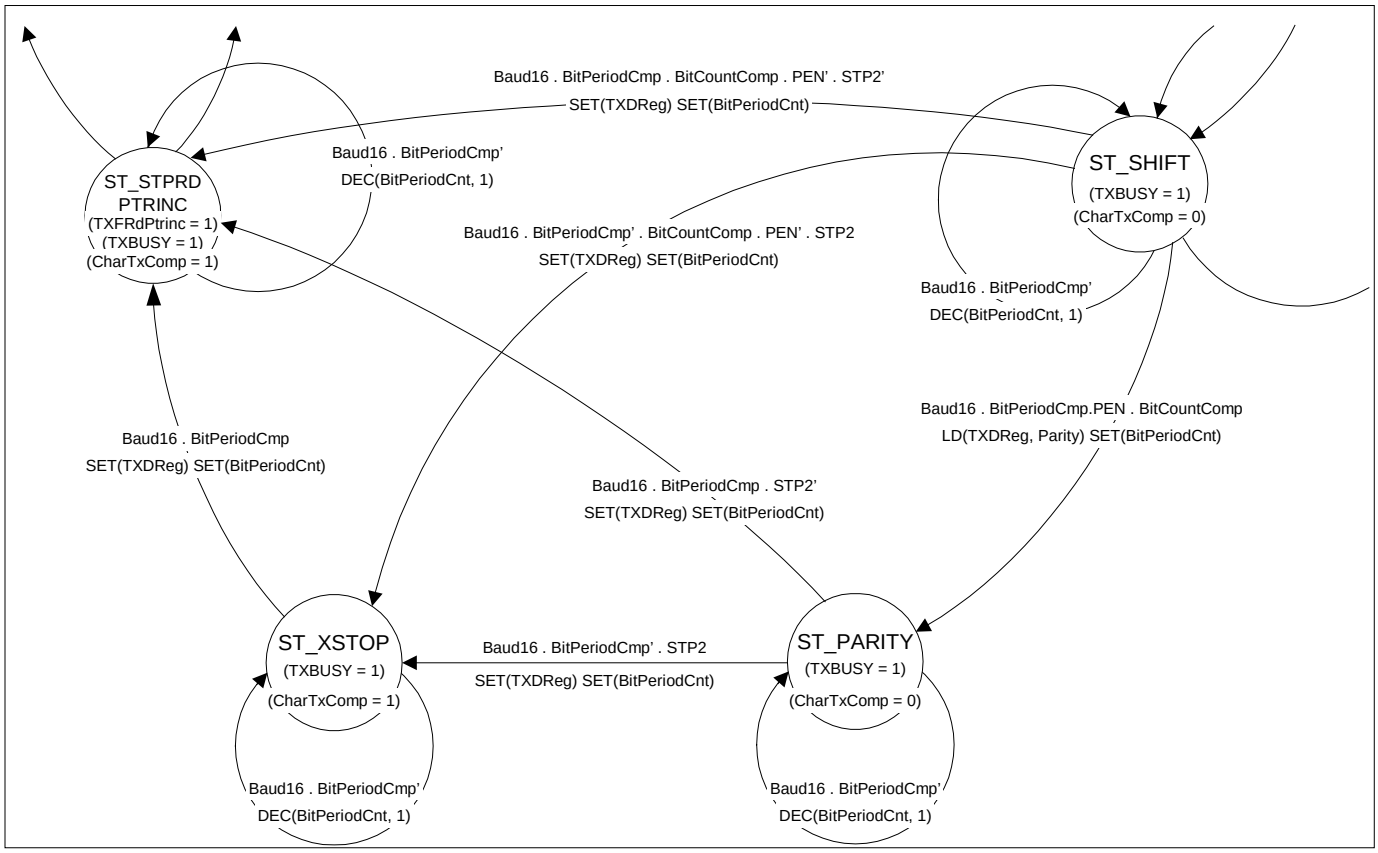

Figure 30 Transmit state machine: Shift, Parity, Xstop & StpRdPtrInc

In the ST_STPRDPTRINC state, the stop bit is timed by decrementing the BitPeriodCnt counter with every Baud16 pulse. Also, the TXFRdPtrInc signal is asserted to indicate to the Transmit FIFO that a data byte has been transmitted and the read pointer can now be incremented.

The section of the state machine involving interaction with the transmit FIFO is shown in Figure 31 Transmit state machine: Idle, StartBit, StpRdPtrInc, Break & Exit Break. After asserting the TXFRdPtrInc signal, the state machine waits for the RdPtrIncDone signal. When this signal is asserted, if TXDataAvlblSync is also asserted indicating that there is further data in the FIFO to be transmitted and if the TXEnable signal is still asserted, the state machine waits for the completion of the bit period and transitions to the ST_STARTBIT state for the next start bit. Else, it transitions to the ST_IDLE state. If CTSEn is asserted, the transition to the ST_STARTBIT state can only occur if the nCTSSyncUARTCLK signal is asserted, otherwise it transitions to the ST_IDLE state.

Once UART state-machine transits to ST_STARTBIT state, BRK bit samples only in state ST_STPRDPTRINC. In state ST_STPRDPTRINC, if BRK is sampled as high, the state machine transitions to the ST_IDLE state. If BRK is still set, the state machine transitions to the ST_BREAK state where it remains so long as BRK is set. When BRK is cleared, the state machine transitions to the ST_EXITBREAK state after pulling the TXD line back to 1. This state is used to implement a 1 bit time of waiting to avoid potential glitches on the nSIROUT line when the SIR endec is enabled.

If the UART is disabled i.e. UARTEn or TXE is de-asserted during transmission of a character, the transmission of that character is completed and then transmission stops i.e. If the UART is disabled when in state ST_IDLE, the state machine remains in that state, if it is disabled when in state ST_ STPRDPTRINC, the state machine transitions to state ST_IDLE, and in all other states it transitions as normal.

To generate odd parity (i.e. when ‘EPS’ is ‘0’), the data bits are XORed and the result is XORed with ‘1’.To generate even parity (i.e. when the ‘EPS’ bit in UARTLCR_H is ‘1’), the data bits are XORed and the result is XORed with ‘0’. In the actual implementation, the data bits are XORed and the result is XORed with the inverted ‘EPS’ signal. The stick parity bit generation depends upon the state of EPS, PEN and the stick parityselect (SPS) bit. If the Stick Parity bit and the Even Parity Select bit are set then the parity bit is transmitted as a 0. If the Stick Parity bit is set but the Even Parity Select bit is not set then the parity bit is transmitted as a 1. If the Stick Parity bit is not set then the parity bit follows the normal parity checking method. Table 1 summarises this behaviour.

Table 1 The state of the parity bit to be set or checked.

Parity Enable (PEN)	Even Parity Select (EPS)	Stick Parity Select (SPS)	Parity bit (transmitted or checked)
0	X	X	-
1	1	0	Even Parity
1	0	0	Odd Parity
1	0	1	1
1	1	1	0

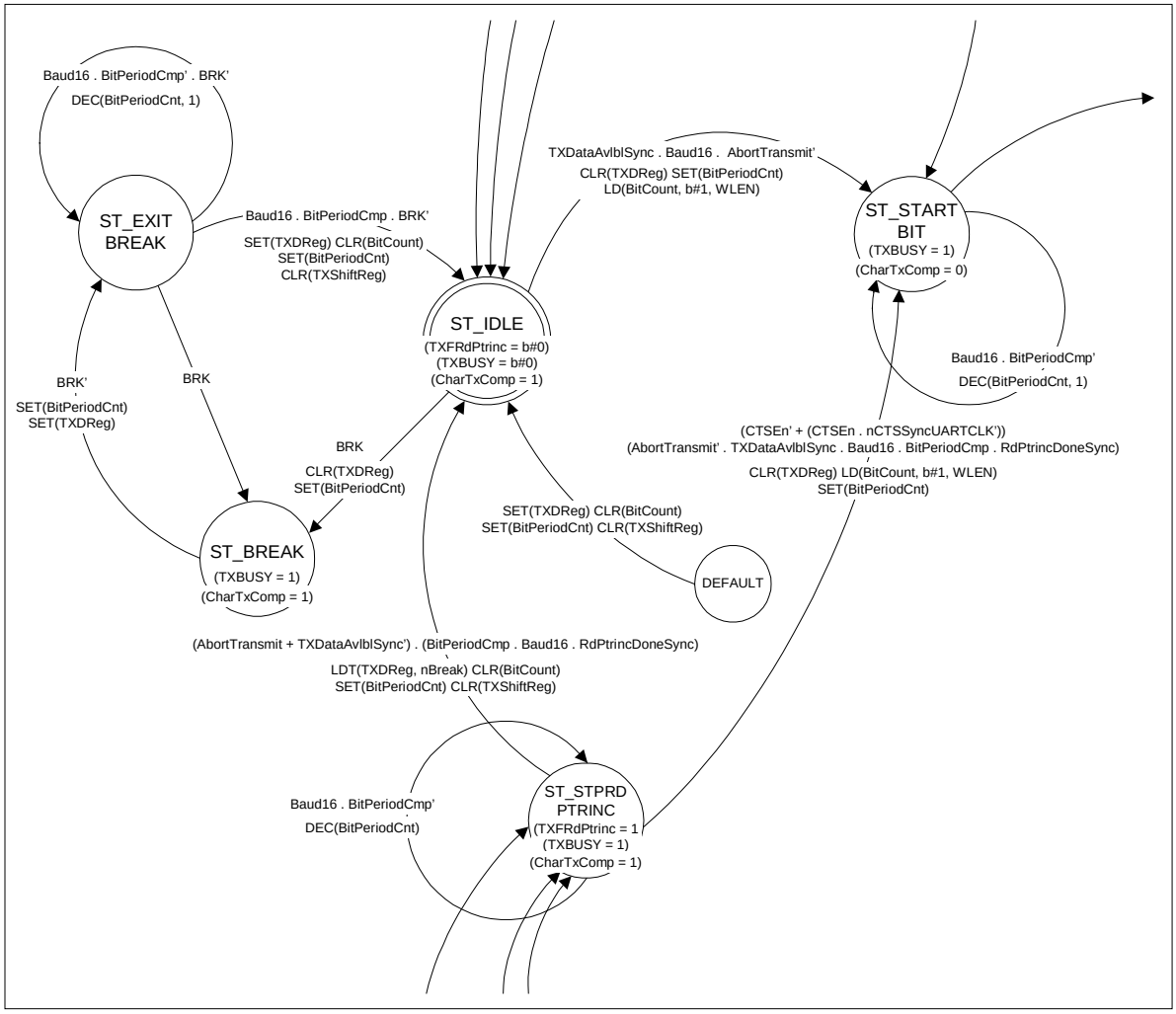

Figure 31 Transmit state machine: Idle, StartBit, StpRdPtrInc, Break & Exit Break

5.10 Receive FIFO (UartRXFIFO)

The Receive FIFO instantiates the Receive FIFO control logic (UartRXFCntl) and the Register File (UartRXRegFile). Writes to the Receive FIFO occur from the Receive section, which operates on UARTCLK. Reads from the Receive FIFO occur through the APB interface operating on PCLK.

The Receive FIFO is similar to the transmit FIFO in most respects. The difference is that each element in the Receive FIFO is 12 bits wide. It is implemented as a circular buffer with read and write pointers. The FIFO can also be disabled to operate in the one-byte holding buffer mode.

When a full byte of data has been received, the Receive section drives the data and the corresponding error bits on RXFIFOData[11:0]. It then asserts the RXFWr signal, which gets synchronised to PCLK and is seen as the RXFWrSync signal. After the write is complete, the RXFWrDone signal is asserted by the FIFO to indicate to the Receive section that the current write is complete.

When the FIFO is already full and an attempt to perform another write is seen, the OverrunDet signal is asserted. This signal is cleared when the UARTECRWrEn signal is asserted.

The RXFRdData[11:0] output from the FIFO is driven with the contents of the location pointed to by the current value of the read pointer.

The Receive FIFO block also contains the multiplexor for the TESTFIFO mode. In TESTFIFO data is written straight to the RXFIFOData signal, whereas in normal mode the RXFIFOData signal is generated from the received data.

5.11 Receive FIFO Controller (UartRXFCntl)

The Receive FIFO controller controls the read pointer and the write pointer. It also contains logic to detect and signal the Overrun condition. The raw and masked status of the receive interrupt, UARTRXINTR, and the Receive timeout interrupt, UARTRTINTR, are also generated by this block. The receive hardware flow control signal Request To Send, nUARTRTSint, is also generated from within this block.

The RXFWrSync signal is clocked through a flip-flop clocked by PCLK to generate a delayed version DelRXFWrSync. The two signals are used to detect the rising edge of the RXFWrSync signal. Writes to the FIFO and increment of the write pointer occur on the basis of the rising edge on the RXFWrSync signal. If the FIFO is not already full, the data on the RXFIFOData[11:0] input is clocked into the register file location currently being pointed to by the write pointer. At the end of the write, the write pointer is incremented and the RXFWrDone signal is asserted. This signal is de-asserted after the RXFWrSync input signal is de-asserted. The Wrap signal is set when the write pointer has wrapped around but the read pointer has not. The Wrap signal is used to calculate the FIFO fill level, in the same way as for the Transmit FIFO. When the FIFO is disabled data on the RXFIFOData[11:0] bus is clocked into the FIFO location pointed to by the current value of the read pointer on the rising edge of PCLK on which the RXFWrSync signal is asserted and the DelRXFWrSync signal is not asserted. The FIFO read pointer and write pointer are unchanged, while the HldBufValid signal is asserted when the write is complete.

The RXFRdPtrInc signal is a one-PCLK-wide pulse used to increment the read pointer. The UARTDRrd signal is a decode of PADDR[11:2], PSEL and PWRITE. The RXFRdPtrInc signal is generated by ANDing PENABLE with UARTDRrd. When the RXFRdPtrInc input is asserted, the Read pointer is incremented if the FIFO is not already empty. Read data from the FIFO is latched into the PRDATA register on the rising edge of PCLK on which UARTDRRd is high. The read pointer changes value on the subsequent rising edge of PCLK on which RXFRdPtrInc is sampled high. When the FIFO is disabled and the RXFRdPtrInc signal is asserted, the HldBufValid signal is de-asserted on the rising edge of PCLK on which the RXFRdPtrInc signal is sampled high. The RXFRdPtrInc signal has PENABLE as one of its gating terms and hence, is one PCLK wide.

The raw status of the Receive interrupt, UARTRXRIS, is asserted when either of the following conditions is satisfied:

- When the receive FIFO is enabled and the FIFO is filled to a level equal to or greater than the programmed interrupt level. The programmed interrupt level is set by writing to the UARTIFLS register.
- When the FIFO is disabled and the holding buffer is full.

The interrupt is cleared when the FIFO contents are read out so that there are the less than the programmed number of entries in the FIFO in the FIFO-enabled mode or the Holding buffer is empty in the FIFO-disabled mode, or when a 1 is written to the corresponding bit in the UARTICR register.

The OverrunDet and FIFOOverrunDet output signals are asserted when the Receive FIFO is full and a further data write access is detected without a simultaneous read access. When there is a write to the UARTECR register, the OverrunDet signal is cleared. When the next successful write to the Transmit FIFO occurs then FIFOOverrunDet signal is cleared. The FIFOOverrunDet status gets written into the receive FIFO along with the break, frame and parity errors, whereas the OverrunDet status is used in the Receive Status register. This is so that the Overrun error bit in the receive FIFO corresponds to the character being read.

If the receive hardware flow control enable bit is set within the UARTCR register, then the nUARTRTSint is asserted low when the receive FIFO level falls below the programmed watermark level, which effectively issues a request to send data signal from the receiving to the transmitting UART. The nUARTRTSint signal is de-asserted high when the receive FIFO level is greater than or equal to the programmed watermark level.

If the receive hardware flow control enable bit is not set within the UARTCR register, then the nUARTRTSint signal is controlled by programming of the RTS bit within the UARTCR register.

5.12 Receive FIFO Register File (UartRXRegFile)

The Receive Register file is similar to the transmit register file except for the fact that the receive register elements are 12 bits wide. Write data is clocked into the location pointed to by the current value of the write pointer (WrPtr[3:0]) on the rising edge of PCLK on which the write enable (RegFileWrEn) is sampled high. The RXFRdData[11:0] bus is driven with the contents of the FIFO location pointed to by the current value of the read pointer (RdPtr[3:0]).

5.13 Receive section (UartReceive)

The Receive section instantiates the Receive control state machine (UartRXCntl), the Receive Shifter / Error Checker (UartRXParShft) and the Receive line idle detector (UartDataStp) blocks.

The RXFESync input signal from the Receive FIFO indicates whether the FIFO is empty or not. This signal is fed to the UartDataStp block which triggers the DataStp signal if the FIFO is not empty and there has been no activity on the receive line for 32 bit periods.

The RXD input to the UartRXCntl block is used to detect the start bit. The RXD signal is sampled 3 times, at Baud16 intervals, and the majority value is kept, which is the RXDSampled signal. The actual shifting of data is done in the UartRXParShft block, which also has the RXDSampled line as an input. The UartRXCntl block times bit periods and generates control signals which indicate to the UartRXParShft block when to sample bits. The ShiftEn signal is a one-UARTCLK clock wide signal, which is used by the control logic to indicate to the UartRXParShft block that the input RXDSampled line has to be sampled for a data bit. Similarly, the SampleParity signal is used to indicate when the Parity bit is to be sampled. The Interrupt enable bit for the Receive timeout interrupt is available as the Receive Timeout interrupt mask, RTIMSync input signal. The deassertion of the RTIMSync signal causes the watchdog counter in the UartDataStp block to stop counting and the counter remains at its maximum value of 0x1FF.

Framing error and Break are detected by the UartRXCntl block and are made available as the FramingError and Break signals respectively. These bits are then combined with the received data (RecdData[7:0]) and the Parity error signal (ParityError) from the UartRXParShft block and the Overrun error (FIFOOverrunDet) and this is driven on the RXFIFOData[11:0] output. When a framing error is detected, the control logic does not wait for a high to low transition for detecting the start bit of the next byte. It waits for one bit time after the stop bit where the framing error was detected and samples the input RXD line for the next start bit level. When Break is detected, the receiver writes one zero character into the receive FIFO and waits for the receive line to go high again to restart reception.

The Framing Error and Break signals are used to generate the raw and masked status of the Framing Error and Break interrupts. The raw status of the Framing error interrupt (UARTFERIS) is generated when there is a rising edge detected on the Framing error signal. It is cleared when a 1 is written to the corresponding bit in the UARTICR register. The masked status (UARTFEMIS) is generated by ANDing the raw status with the Framing error interrupt mask (FEIMSync). The raw status of the Break error interrupt (UARTBERIS) is generated when there is a rising edge detected on the Break signal. It is cleared when a 1 is written to the corresponding bit in the UARTICR register. The masked status (UARTBEMIS) is generated by ANDing the raw status with the Break error interrupt mask (BEIMSync).

The ReloadWD signal is used to reload the watchdog counter whenever there is a stop bit detected in the UartDataStp block. The UartDataStp block detects the receive line remaining idle for 32 bit periods whilst there is data still present in the receive FIFO.

Handshaking with the Receive FIFO occurs through the RXFWr signal and the RXFWrDoneSync signal. This mechanism is similar to the handshaking mechanism in the transmit FIFO.

5.14 Receive Shifter / Error Checker (UartRXParShft)

This block contains logic, which performs the following functions:

- Shifting-in the sampled serial data
- Parity checking

Shifting-in is done every time the ShiftEn signal is asserted by the control logic. Shifting-in of serial data takes into account the number of bits in the current byte. For example, for 5 bits of data, serial data is shifted into bit 4 of the shift register and then shifted right with every bit period. This ensures that, at the end of 5 samples, the data received is right-justified. The remaining bits are filled with zeros.

Parity checking is performed on the received data byte. If the Stick Parity Select (SPS) is enabled this is done by XORing the actual parity bit with the inverted ‘EPS’ bit, otherwise by XORing the expected parity bit (obtained by XORing the data bits with the inverted ‘EPS' bit) with the actual parity bit.

The DtPrZero output is asserted if the data bits shifted in and the Parity bit are all zero. This signal is used to detect Break which is signalled when all the bits in a character frame (data bits, parity bits and stop bits) are zero.

5.15 Receive Control state machine (UartRXCntl)

The Receive control state machine controls the detection of the start bit and the shifting-in of serial data.

The counters used in this state machine are:

- BitPeriodCnt[3:0]. 4-bit counter to count 16 Baud16 pulses
- BitCnt[2:0] Counter to track the number of bits being shifted in.

The comparators used in this state machine are:

- BitPeriodCmp. To compare and check if BitPeriodCnt has reached “0000”
- BitCmp. To compare and check if BitCnt has reached “000”.

The CharRxComp signal is used to indicate that reception of the current character has completed.

The AbortReceive indicates the condition when an illegal divisor value has been programmed (i.e. the ZeroBaud input signal is asserted).

When the AbortReceive condition is satisfied, the reception stops and the state machine goes to the S_IDLE state. The UARTIBRD and UARTFBRD registers are not updated until the current character has completed. Therefore, the AbortReceive signal will only be asserted after the current character has completed transmission or reception.

When the UART is disabled (i.e. the UARTEnSync signal or the RXESync signal is de-asserted) the UART completes the reception of the current character before going to the ST_IDLE state where it remains until the UART is re-enabled.

Figure 32 Receive state machine: Idle, StartBit & Shift shows a section of the receive state machine. When the RXDSampled line goes low, the BitPeriodCnt counter is loaded with 8 if the Uart is operating in normal mode, or 4 if the Uart is operating in either of the IrDa modes. This is to ensure that the sampling takes place in the middle of the pulse, and accounts for the shorter pulses for logic 0’s in the IrDa decoding. The state machine then transitions to the S_STARTBIT state. In this state, the counter counts down with every Baud16 pulse. When the BitPeriodCnt counter reaches 0, if the RXDSampled line is still low, the state machine enters the S_SHIFT state after loading the BitPeriodCnt counter with 15 and the BitCnt counter with the number of bits per data frame. Else, if RXDSampled line has gone high before BitPeriodCnt reaches zero, the start bit is invalidated, the CharRxComp signal is set to ‘1’ and the state machine returns to the S_IDLE state. In the S_SHIFT state, with every Baud16 pulse, BitPeriodCnt decrements. When it reaches 0, the middle of the next data bit has been reached and hence

the ShiftEn signal is asserted for one UARTCLK clock. The BitCnt counter also decrements. The BitPeriodCnt counter is reloaded with 15.

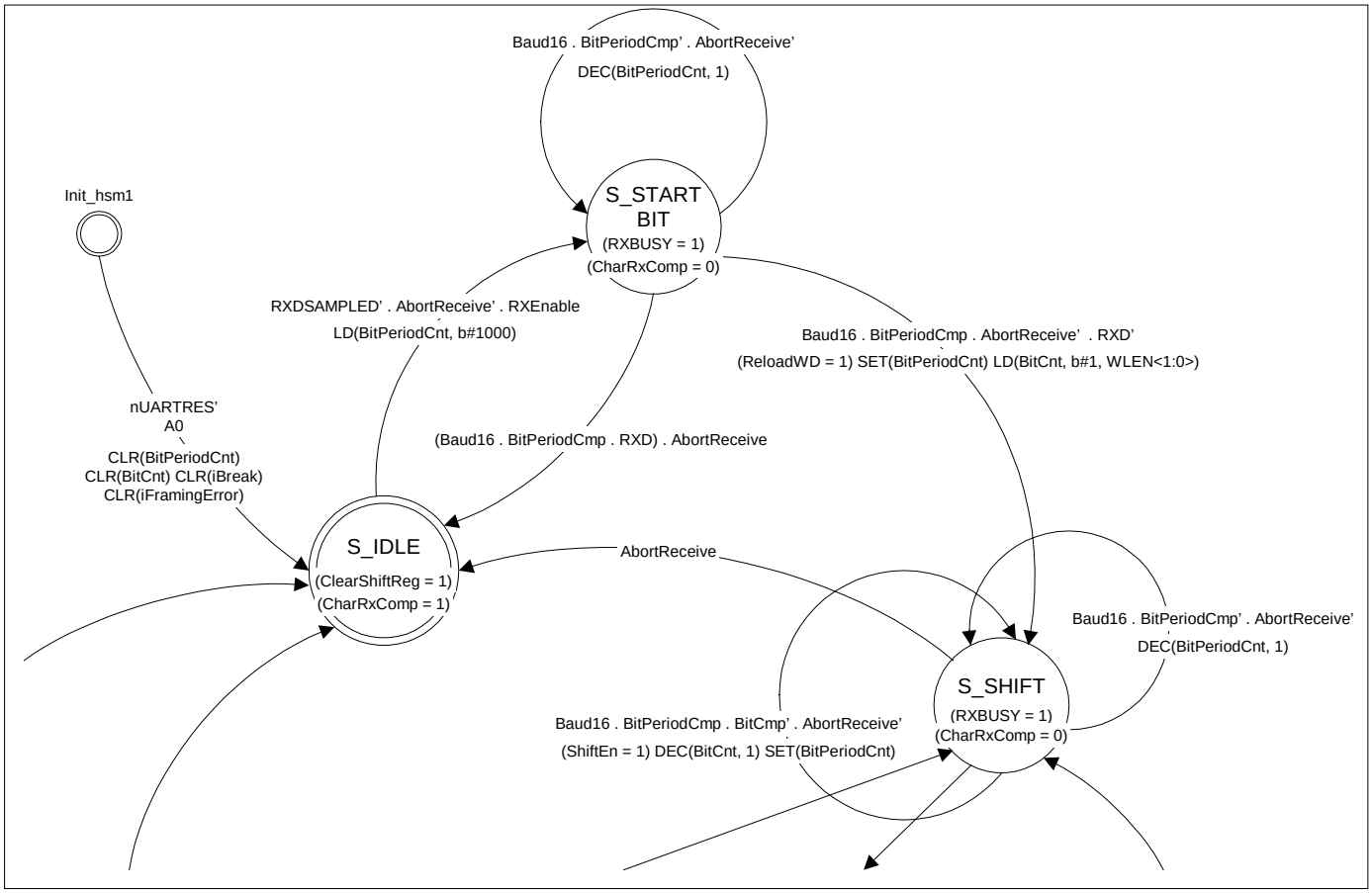

Figure 32 Receive state machine: Idle, StartBit & Shift

Figure 33 Receive state machine: Shift, Parity, StopBit, Xstop & FifoWrite illustrates another section of the state machine. When the programmed number of bits have been shifted in, the state machine moves to the S_PARITY state if parity is enabled. Else, it moves to the S_STOPBIT state, where the stop bit sampling time is determined. During these transitions, the ShiftEn signal is asserted to sample the last data bit. In the S_PARITY state, the BitPeriodCnt counter decrements with every Baud16 pulse. When the BitPeriodCnt counter has counted down to zero, a bit time is said to have elapsed and the SampleParity signal is asserted for one UARTCLK, the BitPeriodCnt counter is reloaded with “1111” and the state machine transitions to the S_STOPBIT state.

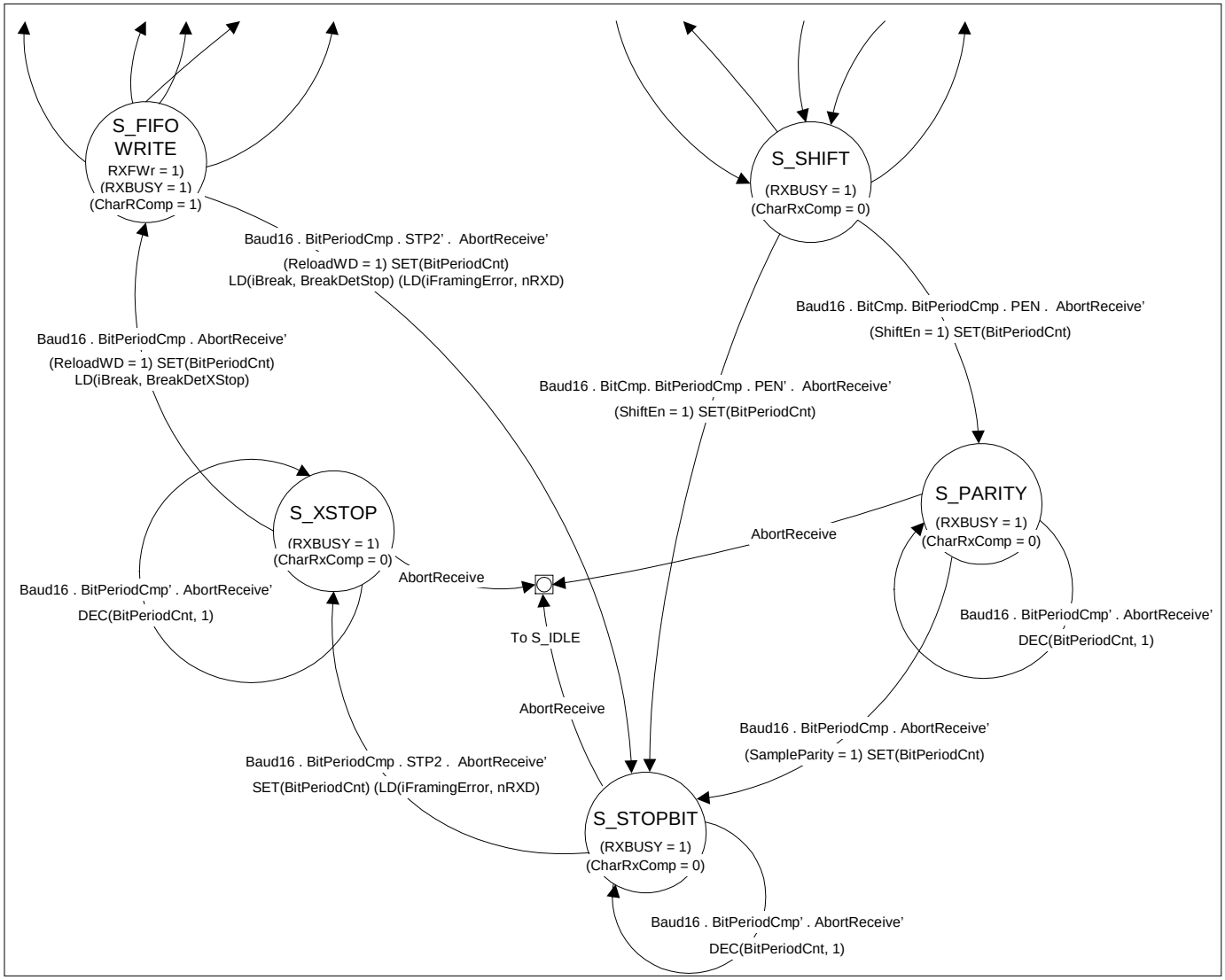

Figure 33 Receive state machine: Shift, Parity, StopBit, Xstop & FifoWrite

After waiting for a bit time in the S_STOPBIT state, if 2 stop bits have been programmed, the RXDSampled line is sampled to check for Framing error, the BitPeriodCnt counter is reloaded with “1111” and the state machine transitions to the S_XSTOP state. If only one stop bit has been programmed, the RXDSampled line is sampled for Framing error, the DtPrZero (Data bits and Parity bit sampled as low) input signal is sampled to detect Break, CharRxComp is set to ‘1’, and the state machine transitions to the S_FIFOWRITE state. In the S_XSTOP state, after a bit time has elapsed, the RXDSampled input and the DtPrZero inputs are sampled to detect Break, CharRxComp is set to ‘1’, and the state machine transitions to the S_FIFOWRITE state.

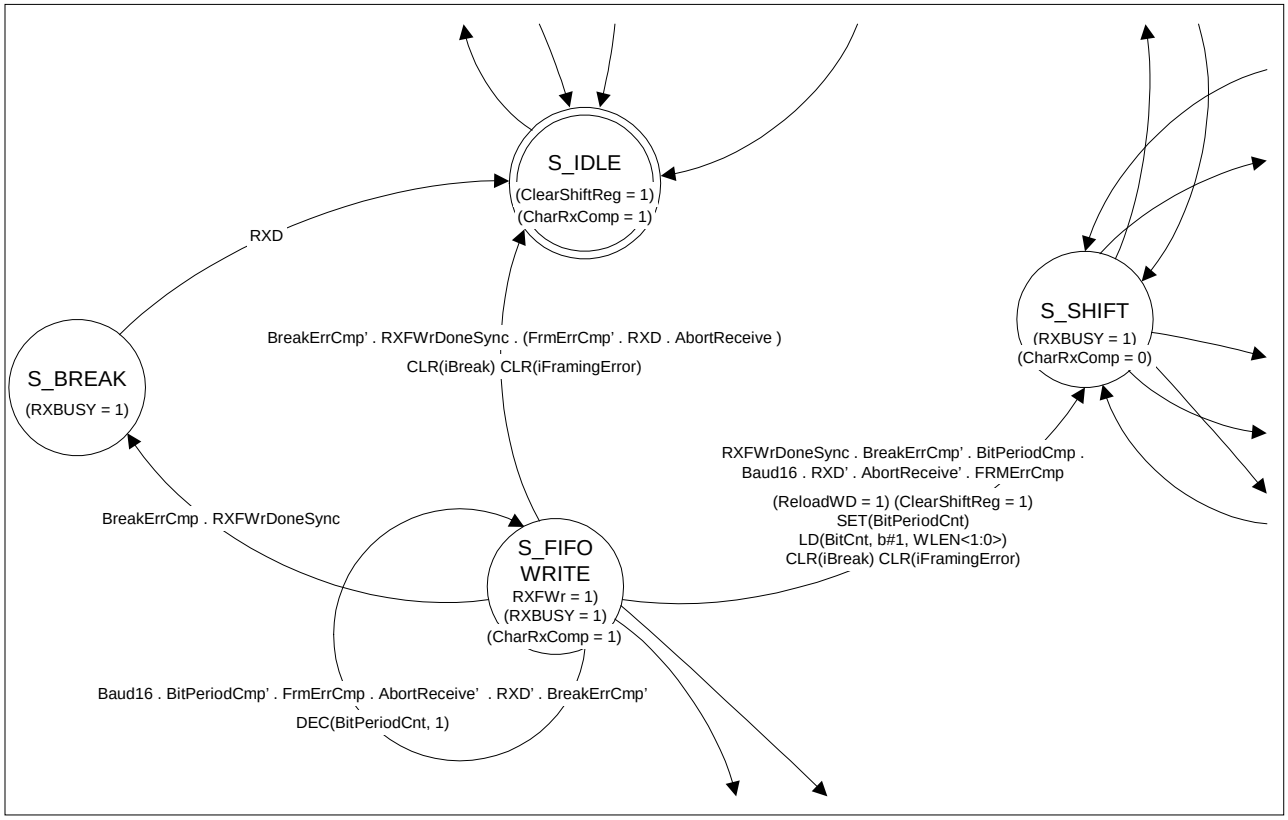

Figure 34 Receive state machine: Shift, FifoWrite & Break

Figure 34 Receive state machine: Shift, FifoWrite & Break illustrates the remaining section of the receive state machine. In the S_FIFOWRITE state, the RXFIFOWr signal is asserted to indicate to the Receive FIFO that received data is available for write and CharRxComp is asserted to indicate that reception of the current character has completed. The BitPeriodCnt counter decrements with every Baud16 pulse. The decrement operation is performed only if a framing error has been detected without a Break having been detected. This is necessary to ensure that the reception is synchronised to the receive bit stream even if a framing error is detected. This ensures that subsequent data characters are not lost. If neither framing error nor break error is detected, then the state machine transitions to the S_IDLE state after the RXFWrDoneSync signal is asserted indicating the completion of the write to the Receive FIFO. If Break is detected, then the state machine transitions to the S_BREAK state in which it remains till the RXDSampled input goes high indicating the end of Break. If framing error has been detected, the state machine remains in the S_FIFOWRITE state for one bit time and then checks if the RXDSampled line is low (indicating the start bit of the next frame). If the RXDSampled line is low, the state machine transitions to the S_SHIFT state after loading the BitPeriodCnt counter with “1111”, the BitCnt counter with the programmed number of data bits per frame and clearing the CharRxComp signal.

5.16 Receive line idle detector (UartDataStp)

This block detects the condition when the receive FIFO is empty and there has been no activity on the RXD line for 32 bit periods. To count 32 bit periods, a 9-bit counter (to count 16 Baud16 pulses per bit period x 32 bits -> 512 Baud16 pulses) is used. This counter counts down from 511 with every Baud16 pulse and when it reaches zero, the DataStp signal is asserted. If a stop bit is detected, the ReloadWD input signal is asserted by the Receive control state machine and the counter reloads back to 511. In the state machine for this block, the ReloadCounter signal is used to detect the condition when the ReloadWD signal or the RXFESync signal is asserted or the RTIESync signal is not asserted. When the ReloadCounter signal is set, the counter remains at 511 and does not count down. The state machine is shown in Figure 35 UartDataStp state machine.

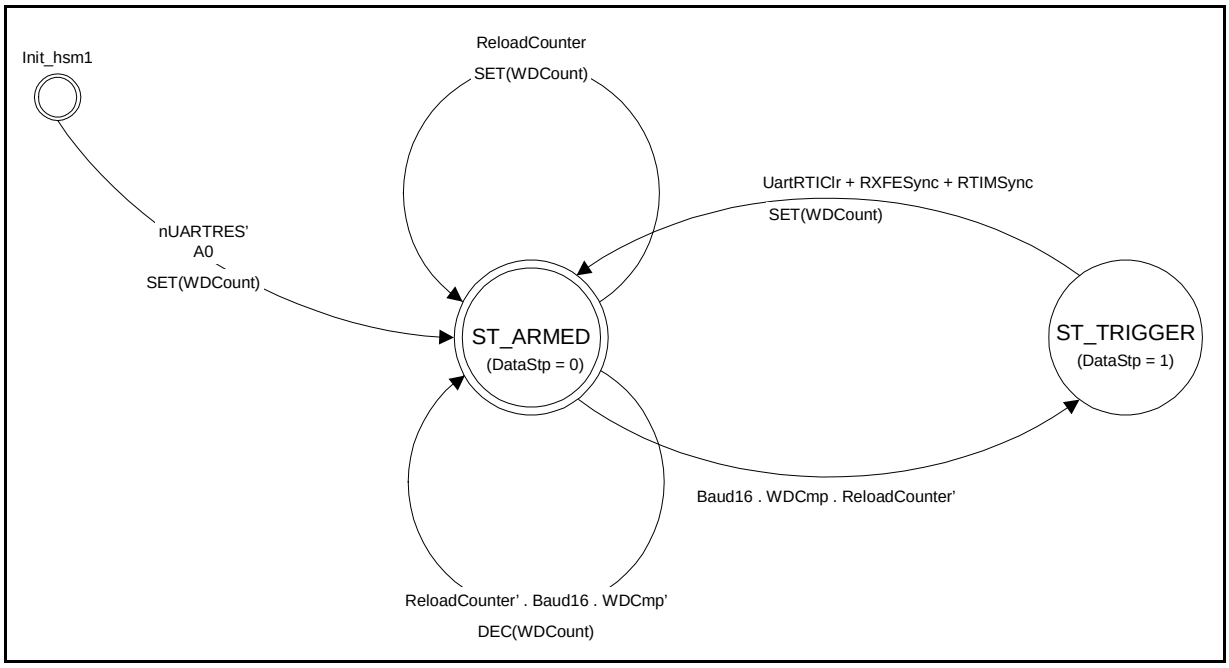

Figure 35 UartDataStp state machine

5.17 SIR Encoder / Decoder (UartIrDA)

The IrDA block contains an encoder for encoding the transmit bit stream to a pulse stream compliant with the IrDA standards. This block also has a decoder, which converts the IrDA compliant input pulse stream into a RS-232 bit stream for further use by the UART receiver.

In the SIR protocol, zeroes are transmitted as a high pulse and ones are transmitted with a low level. The width of the pulse is specified as 3 times the bit period of the Baud16 signal. Hence, the actual pulse width varies with the transmit baud rate.

The IrDA block also supports a low power transmit mode. In this mode, the width of the pulse is determined by the period of the IrLPBaud16 internal signal. Thus, the pulse width is independent of the baud rate of transmission. The period of the IrLPBaud16 signal can be controlled by writing a suitable value into the UARTILPR register. The value written into this register should be such that

$$\text{Low power divisor (ILPDVSR)} = (F_{UARTCLK} / F_{IrLPBaud16})$$

IrLPBaud16 should have a frequency in the range $1.42\text{MHz} < F_{IrLPBaud16} < 2.12\text{ MHz}$ in order to meet the IrDA specification [Ref.4] on the pulse width of the bit stream.

In normal mode transmission, the nSIROUT line is held at a default value of 0. When a low is detected on the TXD line, a 4-bit counter, Tcount, is initialised to 15. The counter counts down with every Baud16 pulse when the TCStart signal is high. When the count value becomes 10 , 9 or 8, the output nSIROUT signal is asserted. Thus a 3-Baud16 wide pulse is produced on the output. If the input TXD line remains low after one bit period, the counter reloads and the process repeats.

In the case of reception, the default values of the SIRINSync input and this block's RXD output are high. Two parallel 2 stage pipelines, one sampling when IrLPBaud16 is low, the other when IrLPBaud16 is high, are used to reject pulses that are less than 2 IrLPBaud16 pulses wide. If, in either of the pipelines the SIRINSync signals is detected low for two or more IrLPBaud16 periods, then the second stage of the respective pipeline will go low. If the SIRINSync pulse is less than one IrLPBaud16 wide, the respective second stage will remain high. The second stage outputs of each pipeline are logically 'anded' such that any of them going low will force the resultant output low. The implementation has been optimised and has the limitation that SIRINSync pulses between 1.5 and 2 IrLPBaud16 pulses wide may or may not be accepted. However, it does meet the IrDA specification that requires acceptance of pulses that are greater than or equal to 1.41us wide. The latter value corresponds to two pulses of 1.42MHz IrLPBaud16 .

When a valid low is detected on the SIRIN line, the Rload signal is set and on the next Baud16 pulse, a 4-bit counter, Rcount is loaded with a value of 12. Simultaneously, the RXD line is pulled low. The counter Rcount then counts down with every Baud16 pulse. On the count of 1, RXD line is pulled high and the counter and the Rload signal are reset to 0. There is no distinction made between normal mode and low-power mode in the case of reception.

In the Low power mode transmission, when a low is detected on the input TXD line, Tcount is loaded with 15. As mentioned before, this counter counts down on every Baud16 pulse. When this counter reaches 10, a 2 bit counter, PdTcount, is loaded with 3 and the nSIROUT line is then pulled high. This counter counts down with every IrLPBaud16 pulse. When this counter reaches 1 and an IrLPBaud16 pulse is seen, the nSIROUT line is pulled low.

The TXBUSY and RXBUSY signals are used to implement the half-duplex behaviour of the IrDA block. When the transmitter is busy as indicated by the assertion of the TXBUSY signal, any activity on the SIRIN input line is ignored (since the IrDA detector would have got saturated). Similarly, when the RXBUSY signal is asserted, the IrDA transmitter is inhibited from encoding any activity on the TXD line onto the nSIROUT output.

The receiver setup time for the IrDA transceivers must be implemented in software. Also, while reception of data is in progress, software should ensure that the transmit FIFO is empty. The UART receive state machine has to transit through the S_IDLE state after each frame has been received because the receive line is asynchronous. When the receiver is idle for the intermediate time between bytes, the transmitter could start off transmitting if the transmit FIFO is not empty.

5.18 Modem status interrupt (UARTMSINTR) generator (UartModem)

The nDCDSyncUARTCLK, nDSRSyncUARTCLK, nCTSSyncUARTCLK and nRISSyncUARTCLK signals have been synchronised to the UARTCLK domain in the UartSynctoUCLK block. When in UART mode i.e. SIRENSync being low, the UartModem block detects any change in these signals and generates the individual interrupts UARTDCDRIS, UARTDSRRIS, UARTCTSRIS and UARTRISRIS respectively. Also, UARTMSINTR, which is the combinatorial OR of the individual interrupts, is also output from this block. The four individual raw interrupts are output from the block as UARTRISmod[3:0] along with their masked UARTMIS[3:0] version. Masking of the individual interrupts is controlled by a simple "and" of the signal with the respective UARTIMSC register bit, synchronised to the UARTCLK domain.

Edge detection is achieved by performing an XOR of the current and delayed versions of each signal, the result being a one UARTCLK wide pulse in each case.

Individual interrupts can be cleared by writing a '1' to the respective UARTICR register bit, which propagates an active high clear pulse to the respective interrupt logic. The Modem interrupts are effectively disabled when SIRENSync is high i.e. when operating in IrDA mode.

When an interrupt is asserted, if there are other edges detected in the modem signals before the clear signal arrives, then these signal changes are ignored.

If a clear signal and an edge detection occur at the same time, the respective interrupt is de-asserted for one UARTCLK clock period and then asserted again so as to accommodate systems with edge-triggered interrupt controllers. Each individual interrupts ‘Edged1’ signal is used for this purpose.

5.19 Asynchronous Interrupt Generation (UartInterrupt)

This block is a pure combinatorial block. The interrupts from the UARTCLK domain are ANDed with the relevant interrupt clear signals. Interrupts are generated asynchronously, but synchronously removed via a PCLK write to the relevant UARTICR register bit.

Combined interrupt clear signals are created for the modem, error and receive timeout interrupts.

The combined modem status interrupt clear signal, MSINTRCLR, is the OR of the UARTDSRIC, UARTDCDIC, UARTCTSIC and UARTRIIC. The MSINTRCLR is then ANDed with the UARTMSINT signal to create the MSINTR output.

The combined error interrupt clear signal, EINTRCLR, is the OR of the UARTEOIC, UARTBEIC, UARTPEIC and UARTFEIC. The EINTRCLR signal then ANDed with the ERRINTR signal, which is the OR of the UARTEINTRfbp and UARTOEMIS signals, to create the EINTR interrupt output.

The receive timeout interrupt, DataStp, is ANDed with the receive timeout clear signal, UARTRTIC, to create the RTINTR interrupt output.

Finally a combined interrupt output, UARTINT, the OR of the above three interrupts with the UARTTXMIS and UARTRXMIS interrupts is output from the block.

5.20 DMA Interface (UartDMA)

This block provides an interface to connect to a DMA controller. The DMA operation of the UART is controlled through the UART DMA Control Register (UARTDMACR). Requests for data to be written to the transmit FIFO or read from the receive FIFO in single or burst accesses are controlled by the four registered outputs, TXDMASREQ, TXDMABREQ, RXDMASREQ and RXDMABREQ. These outputs are asynchronously cleared to zero on application of PRESETn and subsequent next values are clocked through by PCLK.

The DMA transmit enable signal, DMATXEn, is asserted when the UART enable (UARTEN), the transmit logic enable (TXE) and the transmit DMA (TXDMAE) bits are set in the UARTCR and UARTDMACR registers. The DMATXEn signal can also be asserted when the FIFOs need to be tested directly by setting the TXDMAE and TESTFIFO bits within the UARTDMACR and UARTTCR registers.

If DMATXEn is not asserted or UARTTXDMACLRSync is active, then the TXDMASREQ and TXDMABREQ signals are cleared to zero.

If there is one or more spaces available in the transmit FIFO, signified by the TXFLTE15Full signal being set, then TXDMASREQ will be asserted on the next PCLK rising edge.

If the transmit FIFO level falls below that of the programmed watermark level, signified by TxIntLevel being set, then the TXDMABREQ will be asserted on the next PCLK rising edge.

If there is data available within the receive FIFO, signified by the RXGTE15Full signal being set, then TXDMASREQ will be asserted on the next PCLK rising edge.

If the receive FIFO level exceeds that of the programmed watermark level, signified by RxIntLevel being set, then the TXDMABREQ will be asserted on the next PCLK rising edge.

When the respective DMA transfer is about to complete, the DMA controller will raise the relevant UARTTXDMACLR or UARTRXDMACLR signal, terminating the transfer. After termination, if more accesses are required, then the relevant request signals will be clocked out on the next PCLK rising edge after removal of the respective clear signal.

5.21 Synchronisers to UARTCLK domain (UartSynctoUCLK)

This block contains all synchronisers in the design that synchronise signals transferring from the external world or PCLK domain to the UARTCLK domain. In general, synchronisation is achieved by passing the signal through two d-types clocked by UARTCLK. The synchronised versions of the following signals have the corresponding signal names appended with the ‘Sync’ suffix.

- The SIRLP signal is synchronised from the PCLK domain to the UARTCLK domain. It is required by the IrDA block to signify that low power mode of operation is to be used.
- The UARTEN and the SIREN signals are synchronised from the PCLK domain to the UARTCLK domain. They are used by the UART to enable/disable the UART transmission/reception and IrDA encoding/decoding.
- The TXE and RXE signals are synchronised from the PCLK domain to the UARTCLK domain. They provide individual enables for the transmitter and receive logic.
- TXDataAvlbl signal is synchronised from the PCLK domain to the UARTCLK domain. It is required for the transmit section to detect if data is available within the transmit FIFO.
- The RXFE signal is synchronised from the PCLK domain to the UARTCLK domain. The resultant RXFESync signal is required by the receive logic to detect the condition when the receive FIFO is not empty and there has been no activity on the receive line for 32 bit periods.

- The CTSEn signal is synchronised from the PCLK domain to the UARTCLK domain. It is required by the transmit control logic to enable hardware flow control.
- The LCRUpdateSync and ILPRUpdateSync signals are synchronised from the PCLK domain to the UARTCLK domain. They form part of the control logic required to synchronise the contents of the LCR and UARTILPR register to the UARTCLK domain. These signals are required by the Baud rate generator.
- The FBRDUpdateSync signal is synchronised from the PCLK domain to the UARTCLK domain. It forms part of the control logic required to synchronise the contents of the UARTFBRD register to the UARTCLK domain. These signals are required by the Fractional Baud rate generator.
- The UARTRXD signal is synchronised from the external world to the UARTCLK domain. It is the synchronised version of the asynchronous receive input pad.
- The SIRIN signal is synchronised from the external world to the UARTCLK domain. It is the synchronised version of the asynchronous IrDA receive input pad.
- The RXFWrDone signal is synchronised from the PCLK domain to the UARTCLK domain. It is required by the receive logic to detect the completion of a write into the receive FIFO.
- The RdPtrIncDone signal is synchronised from the PCLK domain to the UARTCLK domain. It is required to detect the completion of the read pointer increment operation in the transmit FIFO.
- The DCDIM, DSRIM, CTSIM, RIIM, RTIM, FEIM, PEIM and BEIM interrupt mask signals are synchronised from the PCLK domain to the UARTCLK domain. They effectively enable the propagation of the raw interrupts through to the final masked interrupt outputs.
- The nDCD, nDSR, nCTS and nRI signals are synchronised from the external world to the UARTCLK domain. They are the modem control and status signals.
- The UARTDCDIC, UARTDSRIC, UARTCTSIC, UARTRIIC, UARTFEIC, UARTPEIC, UARTBEIC and UARTRTIC signals are synchronised from the PCLK domain to the UARTCLK domain. They are the synchronised versions of the individual interrupt clear signals.

5.22 Synchronisers to PCLK domain (UartSynctoPCLK)

This block contains all synchronisers in the design that synchronise signals transferring from the external world or UARTCLK domain to the PCLK domain. In general, synchronisation is achieved by passing the signal through two d-types clocked by PCLK. The synchronised versions of the following signals have the corresponding signal names appended with the ‘Sync’ suffix.

- The TXBUSY signal is synchronised from the UARTCLK domain to the PCLK domain. It is used by the Transmit FIFO logic to generate the BUSY signal which is also readable as a bit in the UARTFLG register through the APB interface.
- The TXFRdPtrInc signal is synchronised from the UARTCLK domain to the PCLK domain. It is required by the Transmit FIFO to detect when the Read pointer can be incremented.
- The RXFWr signal is synchronised from the UARTCLK domain to the PCLK domain. It is required by the Receive FIFO to detect when a received data byte is available for writing into the receive FIFO.
- The Abort signal is synchronised from the UARTCLK domain to the PCLK domain. It is required by the transmit FIFO to detect the condition when the transmission of the current data byte has been aborted.
- The DataStp signal is synchronised from the UARTCLK domain to the PCLK domain. It becomes the UARTRTRIS interrupt status bit within the UARTRIS raw interrupt status register which can be read via the APB interface.

- The nDCD, nDSR, nCTS and nRI modem signals are asynchronous inputs which are synchronised to the PCLK domain. They become the modem status flags within the UARTFR flag register and can be read via the APB interface.
- The UARTRISmod [3:0] and UARTMISmod [3:0] bus signals are synchronised from the UARTCLK domain to the PCLK domain. They become the modem raw and masked interrupt status bits within the UARTRIS and UARTMIS registers respectively.
- The UARTRISerr [2:0] and UARTMISerr [2:0] bus signals are synchronised from the UARTCLK domain to the PCLK domain. They become the error raw and masked interrupt status bits within the UARTRIS and UARTMIS registers respectively.
- The UARTTXDMACLR and UARTRXDMACLR signals are asynchronous inputs which are synchronised to the PCLK domain. They are used to clear the UARTTXDMABREQ, UARTTXDMASREQ, UARTRXDMABREQ and UARTRXDMASREQ signals.
- CharTxComp signal is synchronised from UARTCLK domain to the PCLK domain. It signals the completion of transmission of a character.

5.23 Test logic (UartTest)

The Test block performs the following functions:

- Test mode control via UARTTCR register
- Integration testing using UARTITIP and UARTITOP read/set registers.
- Test read access and write access of the Tx and Rx FIFOs respectively.
- Loopback facility of UARTTXD to UARTRXD and nSIROUT to SIRIN in test mode.

The UARTCR test control register contains three programmable bits:

- ITEN, the integration test enable. When set, the UART is placed into integration test mode. This mode allows point to point connections to be checked between other modules when the UART is instantiated within a system. Two registers, UARTITIP (Integration Test Input) and UARTITOP (Integration Test Output) are used to implement this test mode.
- TESTFIFO, the test FIFO enable bit. When set, data can be read from the Tx FIFO or written to the Rx FIFO via the UARTTDR test data register.
- SIRTest, used in conjunction with the UARTTCR LBE (loop back enable) bit. When both are set, an internal inverted path is enabled between the nSIROUT and SIRIN. When the SIRTest bit is low and the LBE bit is set, a non-inverted path is enabled between UARTTXD and UARTRXD. If LBE is low, then the SIRTest bit has no effect.

5.23.1 Integration Test Logic – Intra-chip and Primary Inputs

The following sections describe the integration testing for the block inputs:

- Intra-chip inputs
- Primary inputs

Intra-chip inputs

Figure 36 Input integration test harness explains the implementation details of the input integration test harness. The ITEN bit is used as the control bit for the multiplexor, which is used in the read path of the UARTTXDMACLR and UARTRXDMACLR intra-chip inputs. If the ITEN control bit is deasserted, the UARTTXDMACLR and UARTRXDMACLR intra-chip inputs are routed as the internal UARTTXDMACLR and UARTRXDMACLR inputs respectively, otherwise the stored register values are driven on the internal line. All other read only bits in the UARTITIP register are connected directly to the primary input pins.

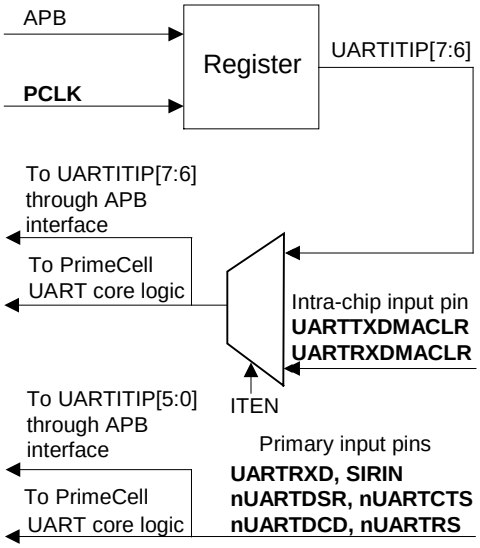

Figure 36 Input integration test harness

When you run integration tests with the PrimeCell UART in a standalone test setup:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the UARTITIP[7:6] register bits to the UARTRXDMACLR and UARTTXDMACLR signals.
- Write a 1 and then a 0 to each of the UARTITIP[7:6] register bits, and read the same register bits to ensure that the value written is read out.
- When you run integration tests with the PrimeCell UART as part of an integrated system:
- Write a 0 to the ITEN bit in the control register. This selects the normal path from the external UARTRXDMACLR pin to the internal UARTRXDMACLR signal, and the path from the external UARTTXDMACLR pin to the internal UARTTXDMACLR pin.
- Write a 1 and then a 0 to the internal test registers of the DMA controller to toggle the UARTRXDMACLR signal connection between the DMA controller and the PrimeCell UART. Read from the UARTITIP[6] register bit to verify that the value written into the DMA controller, is read out through the PrimeCell UART. Similarly, write a 1 and then a 0 to the internal registers of the DMA controller to toggle the UARTTXDMACLR signal connection between the DMA controller and the PrimeCell UART. Read from the UARTITIP[7] register bit to verify that the value written into the DMA controller, is read out through the PrimeCell UART.

Primary inputs

The primary inputs are tested using the integration vector trickbox by looping back primary outputs as follows:

- UARTTXD to UARTRXD
- nSIROUT to SIRIN
- nUARTRTS to nUARTCTS
- nUARTOUT1 to nUARTDCD
- nUARTDTR to nUARTDSR
- nUARTOUT2 to nUARTRI.

Write a 1 to the ITEN bit in the control register. 1s and 0s are driven onto the primary output lines through the UARTITOP register bits [5:0], and read back through the UARTITIP register bits [5:0].

5.23.2 Integration Test Logic – Intra-chip and Primary Outputs

Integration Vector Trickbox

Integration testing of the primary outputs and primary inputs are carried out in conjunction with an Integration Vector Trickbox. Figure 37 Integration Vector Trickbox shows the block diagram of the UART Integration Vector Trickbox. Please refer to the document Primary I/O Integration Vector Strategy for the description of a general Integration Vector Trickbox and the testing flow.

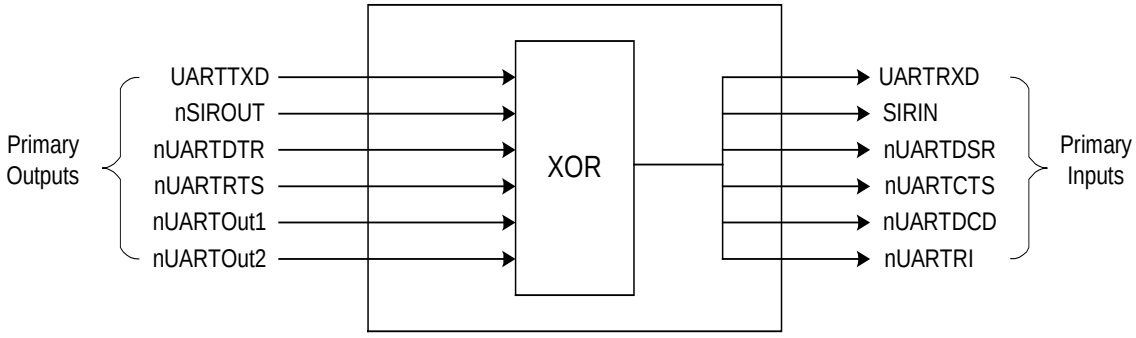

Figure 37 Integration Vector Trickbox

The pin connections are verified as follows. The primary output pins UARTTXD, nSIROUT, nUARTDTR, nUARTRTS, nUARTOut1 and nUARTOut2 are XORed together and routed back to the primary input pins UARTRXD, SIRIN, nUARTDSR, nUARTCTS, nUARTDCD and nUARTRI via the Integration Vector Trickbox. All the primary outputs can be accessed through the UARTITOP register. Different data patterns are written to the output pins using the UARTITOP register and the XORed data is read back from the primary input pins through the UARTITIP register.

Intra-chip outputs

Use this test for the following outputs:

- UARTTXDMASREQ, UARTTXDMABREQ, UARTRXDMASREQ , UARTRXDMABREQ
- UARTINTR, UARTMSINTR, UARTRXINTR , UARTTXINTR , UARTRTINTR , UARTEINTR

When you run integration tests with the PrimeCell UART in a standalone test setup:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the UARTITOP0[15:6] register bits to the intra-chip output signals.
- Write a 1 and then a 0 to the UARTITOP0[15:6] register bits, and read the same register bits to verify that the value written is read out.

When you run integration tests with the PrimeCell UART as part of an integrated system:

- Write a 1 to the ITEN bit in the control register. This selects the test path from the UARTITOP0[15:6] register bits to the intra-chip output signals.
- Write a 1 and then a 0 to the UARTITOP0[15:6] register bits to toggle the signal connections between the DMA controller/interrupt controller and the PrimeCell UART. Read from the internal test registers of the DMA controller/interrupt controller to verify that the value written into the UARTITOP0[15:6] register bits is read out through the PrimeCell UART.

Figure 38 Output integration test harness, intra-chip outputs explains the implementation details of the output integration test harness for intra-chip outputs.

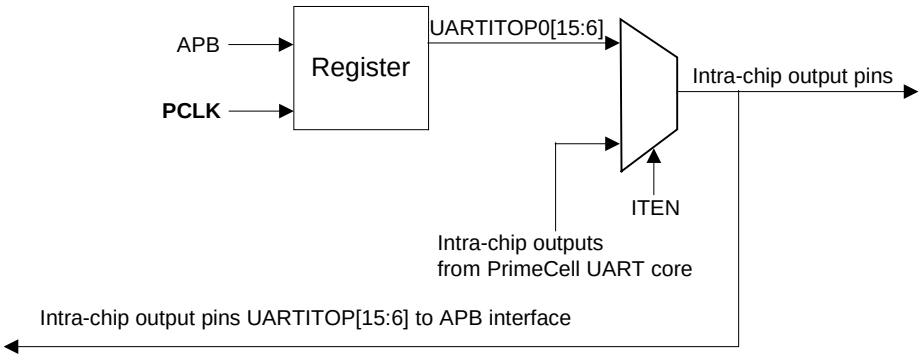

Figure 38 Output integration test harness, intra-chip outputs

Primary outputs

Integration testing of primary outputs and primary inputs is carried out using the integration vector trickbox, described above.

Use this test for the following outputs:

- UARTTXD, nSIROUT, nUARTDTR, nUARTRTS, nUARTOut1, nUARTOut2

Verify the primary input and output pin connections as follows:

The primary output pins (listed above) are looped back to the primary input pins through the integration vector trickbox.

- All the primary outputs can be accessed through the UARTITOP register. Different data patterns are written to the output pins using the UARTITOP registers.
- The looped back data is read back through the UARTTIP register.

Figure 39 Output integration test harness, primary outputs explains the implementation details of the output integration test harness in the case of primary outputs.

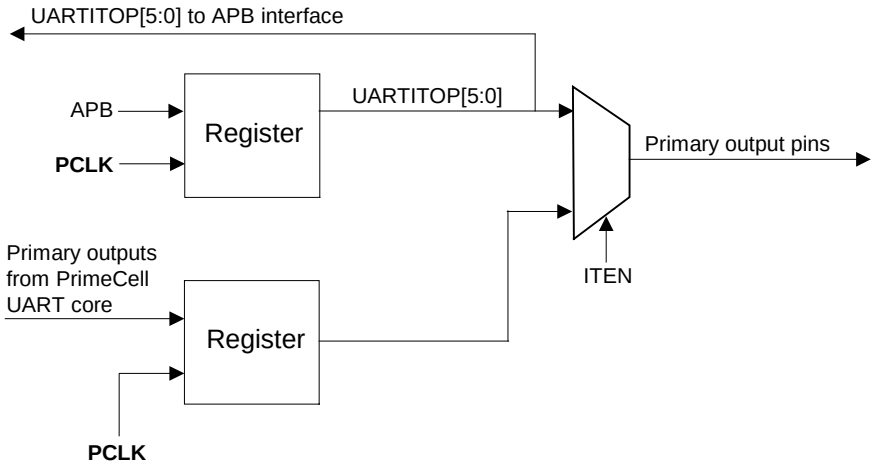

Figure 39 Output integration test harness, primary outputs

The test sequence is as follows:

- Write all ‘0’s to all the primary output lines. This should cause all the primary inputs to be ‘0’ which is read via the UARTITIP register.
- Write a ‘1’ to UARTTXD, and a ‘0’ to all the other primary inputs. This should cause a ‘1’ to appear on all the primary inputs which is read via the UARTITIP register. Then write a ‘0’ to it and the primary inputs should all return to ‘0’.
- The above sequence is repeated for all the primary output lines, so each one in turn has a ‘1’ and a ‘0’ written to it.

Therefore, the test sequence will be (with the pins in the order as on figure 1 beginning with UARTTXD):

0-0-0-0-0-0

1-0-0-0-0-0

0-0-0-0-0-0

0-1-0-0-0-0

0-0-0-0-0-0

0-0-1-0-0-0

0-0-0-0-0-0

0-0-0-1-0-0

0-0-0-0-0-0

0-0-0-0-1-0

0-0-0-0-0-0

0-0-0-0-0-1

6 UART FUNCTIONAL VERIFICATION DETAILS

The UART Functional Verification testbench is an APB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the verification/vhdl and verification/verilog directories respectively. The test vectors that are applied to the BusTalk testbenches are in the verification/bustest directory.

6.1 UART VHDL Verification Testbench

The ARM PrimeCell UART VHDL test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The testbench has been designed such that it can be used within most common VHDL simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. Figure 40 VHDL testbench for simulating test vectors illustrates the Bustalk VHDL testbench for running test vectors.

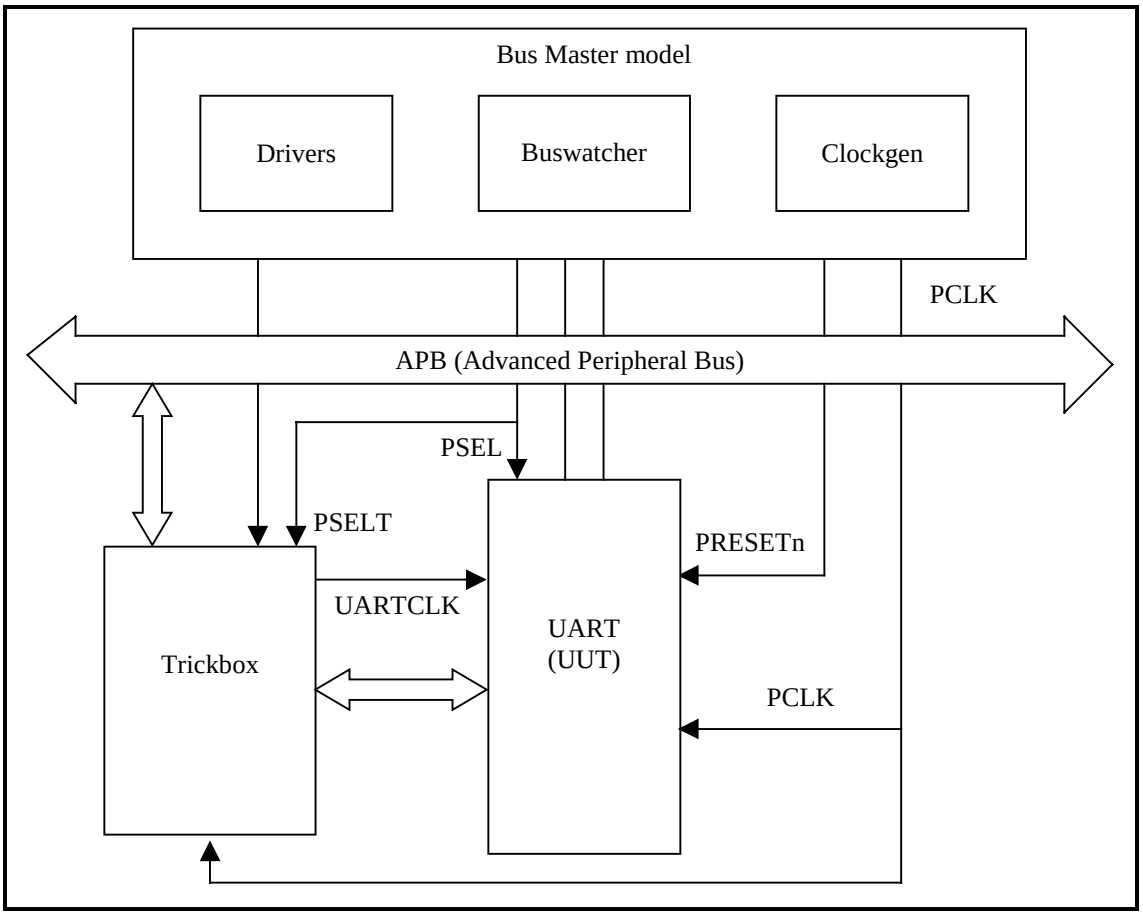

Figure 40 VHDL testbench for simulating test vectors

Figure 41 VHDL Testbench hierarchy illustrates the hierarchy for the VHDL testbench.

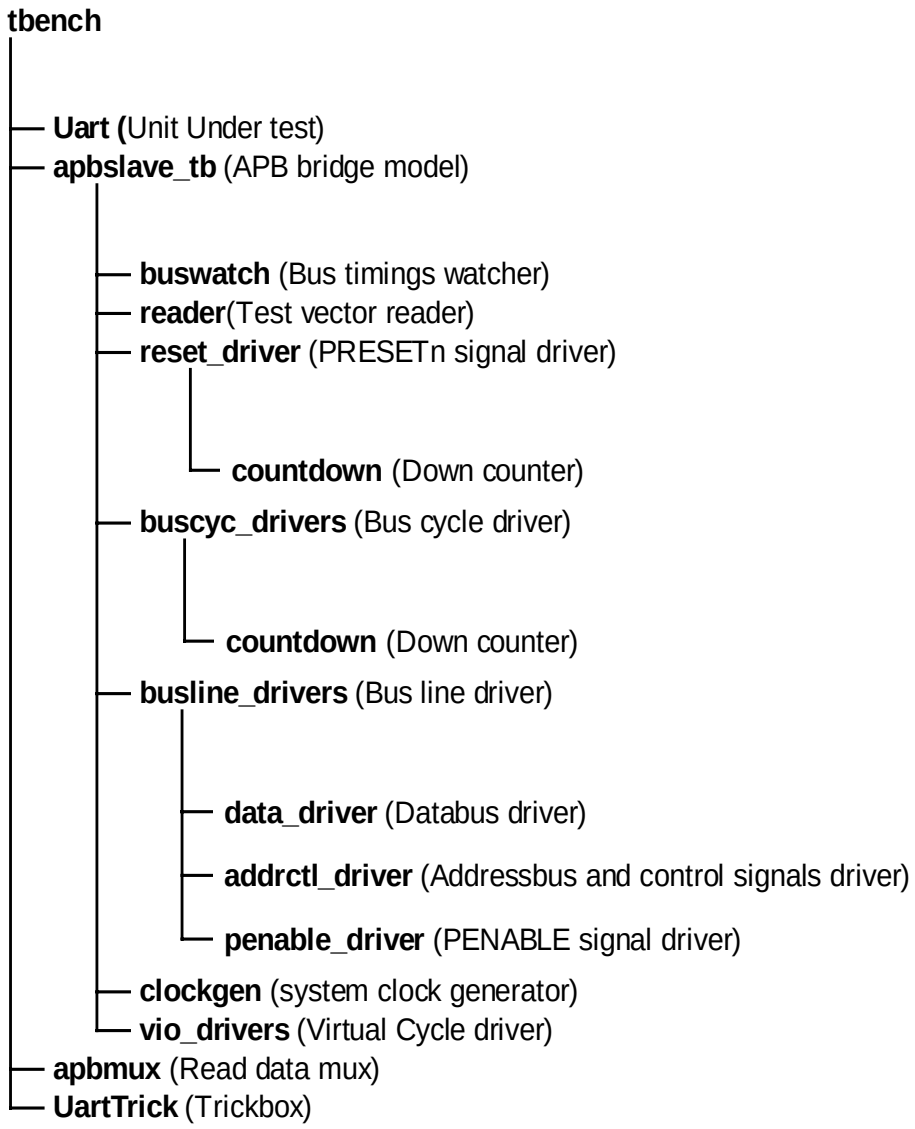

Figure 41 VHDL Testbench hierarchy

The testbench, tbench.vhd is used for functional simulations of the UART. This module instantiates the UART (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The UARTCLK is generated by the trickbox to facilitate frequency modifications from within the test code.

The UARTCLK and it's domain inputs to the UART are driven by the trickbox. All the Scan input pins are tied low to keep them from interfering with the test process.

The default test frequency is set at 100MHz. This is the same frequency to which the UART was constrained during synthesis. PCLK is set to the frequency defined in tbench.vhd using the tclkh and tclkl generic values when instantiating apbslave_tb.vhd.

6.1.1 Unit Under Test

The Unit Under Test may be one of the following

- UART VHDL RTL
- UART VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in verification/vhdl .

6.1.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This module issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.vhd, instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader module reads the test vectors from the infile.bif file (from the verification/bustest/invec directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch module checks the Read/Write Databus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the TrickBox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

While the default behaviour of the Bridge in the testbench is like a Rev E device, there is an option to force the bridge to issue commands with Rev D timings.

6.1.3 Trickbox

The functionality of the UART is verified via a trickbox. The trickbox is an APB Slave which mimics the behaviour of an UART Slave device. Like the UART, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel converter to line up data received serially from the UART and it has a parallel to serial converter to transmit the data stored in its FIFO. The trickbox has a checker module which flags off any violations in the UART transmit/receive protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

6.1.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.vhd file in the verification/vhdl/tbench directory. The buswatch module which is in the verification/vhdl/buswatcher directory checks the Read/Write Databus timings and reports any violations in the timings.

6.1.5 APB Multiplexer

The read data buses from the UART and the trickbox are muxed together in this module before being connected to the APB Bridge model.

6.1.6 Test Vectors

The test vectors for the verification of the UART are coded into separate C files depending on the logical region of the UART which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.1.7 Generics

The UART testbench provides some configurability options using generics and constants. These generics and constants are configurable through the tbench.vhd file.

XonPSEL

XonPSEL is used in the tbench.vhd file that resides in verification/vhdl/tbench.

This generic is used to eliminate the ‘X’s that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If XonPSEL is set to 0 in the top level of the testbench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to 1, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.vhd, the reset_driver.vhd and the buscyc_drivers.vhd which reside in verification/vhdl/bid.

This generic is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If Verbosity is set to 0, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.vhd which resides in verification/vhdl/bid.

This generic is used to stop the simulation if a read is unsuccessful. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged, but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the buswatch.vhd which resides in verification/vhdl/buswatcher.

This generic is used to suppress the checking of the PRDATA timings when the PRESETn signal is asserted. If SuppressOnReset is TRUE, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If SuppressOnReset is set to FALSE, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

Mesg_enb

Mesg_enb is used in the buswatch.vhd which resides in verification/vhdl/buswatcher.

This generic is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this generic is set to FALSE. If Mesg_enb is set to TRUE, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to FALSE in testbenches which include more than one slave.

Data_check

Data_check is used in the apbmux.vhd which resides in verification/vhdl/tbench.

This constant is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When Data_check has a value 1, these error messages are flagged. When it has a value 0, the error messages are not flagged. This constant is used in testbenches which include more than one slave.

6.1.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the UART test vectors on the VHDL source code and the VHDL netlist.

Run time logs for the RTL simulations are:

- verification/vhdl/Uart_rtl/Uart_rtl_modelsim.log

Run time logs for the netlist simulations are:

- verification/vhdl/Uart_scaninsert_vhdl_net_max/Uart_scaninsert_vhdl_net_max.log

6.2 UART Verilog Verification Testbench

The ARM PrimeCell UART Verilog test world uses a variant of a top level testbench which is based on a bus compliance testbench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test bench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The test bench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the test bench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific test bench to which it is being targeted. Figure 42 Verilog Testbench for simulating test vectors illustrates the BusTalk Verilog testbench for simulating test vectors.

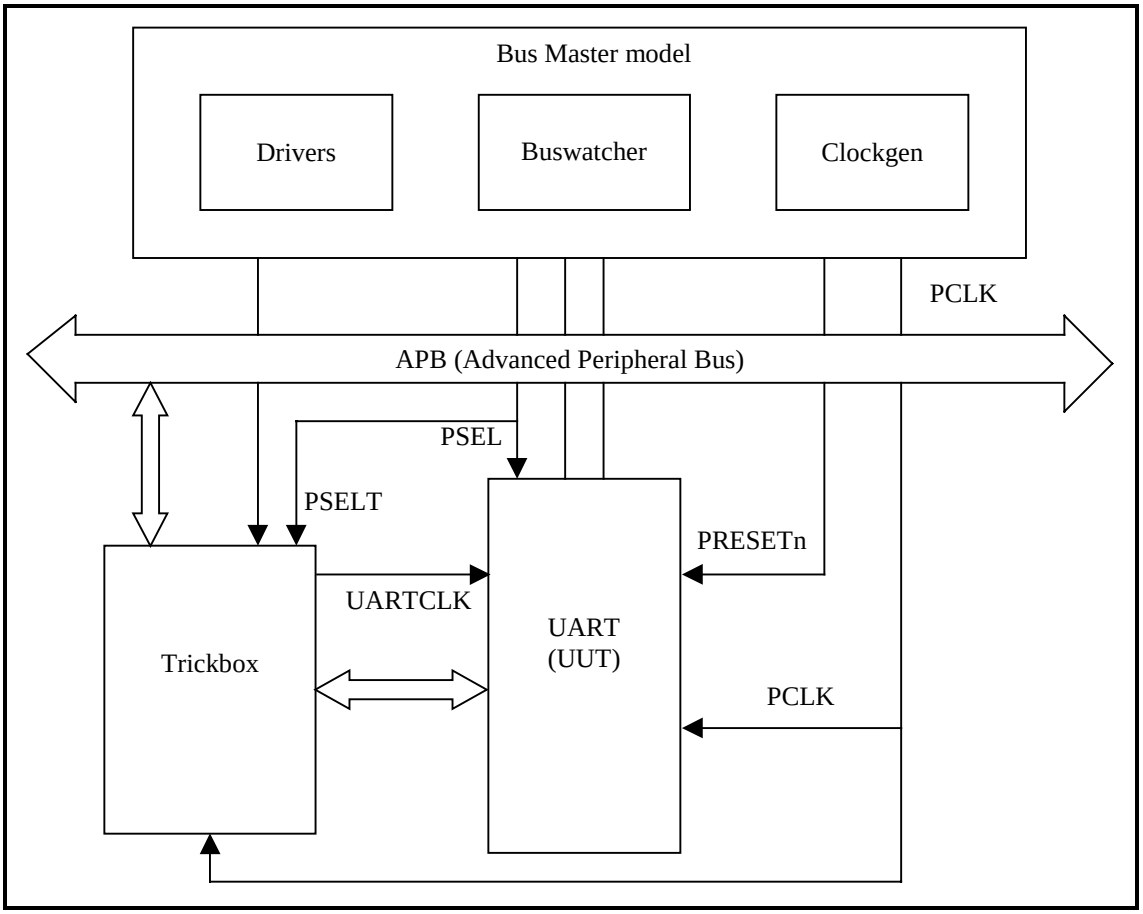

Figure 42 Verilog Testbench for simulating test vectors

Figure 43 Verilog Testbench hierarchy illustrates the hierarchy for the Verilog testbench.

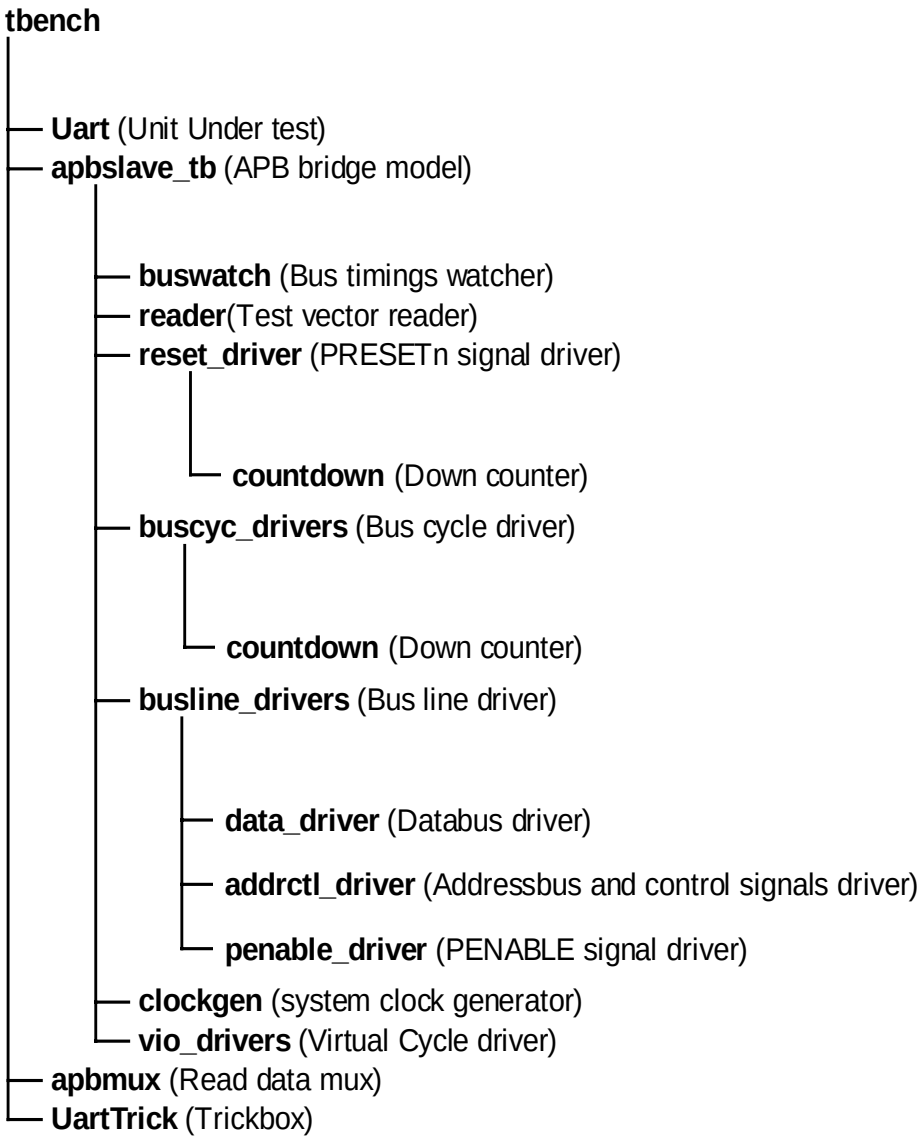

Figure 43 Verilog Testbench hierarchy

The testbench, tbench.v is used for functional simulations of the UART. This module instantiates the UART (Unit Under Test), the behavioural model of the APB Bridge, the trickbox and the APB Read mux. The UARTCLK is generated by the trickbox to facilitate frequency modifications from within the test code.

All the UARTCLK and UARTCLK domain inputs to the UART are driven by the trickbox. All the scan input pins are tied low to keep them from interfering with the test process.

The default test frequency is set at 100 MHz. This is the same frequency to which the UART was constrained during synthesis. UARTCLK is set to the PCLK frequency defined in timing.v using the Tclkh and Tclkl ‘define values when instantiating apbslave_tb.v in tbench.v. Note that the ‘define values override the local parameters defined in clockgen.v in the verification/verilog/common directory. All the non-AMBA inputs to the UART, except the scan data inputs, are driven by the Trickbox.

6.2.1 Unit Under Test

The Unit Under Test may be one of the following

- UART Verilog RTL
- UART Verilog netlist from Verilog Source
- UART Verilog netlist from VHDL Source

One out of these three versions may be referenced via the corresponding links to uut directory in verification/verilog directory.

6.2.2 APB Bridge Model

The UUT is stimulated primarily by the APB bridge behavioural model. This block issues commands on the APB bus under program control.

The APB bridge model, apbslave_tb.v instantiates the buswatcher, the reader, the reset driver, the bus cycle drivers, the bus line drivers and the clock generator. The reader block reads the test vectors from the bif.sim file (from the verification/bustest/invec directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the PRESETn signal with the correct timings. The buscyc_drivers instantiates an APB cycle driver for each cycle type, i.e. PSW, PSR, PI, PO, etc. The busline_drivers drives the databus, the address and control signals with the correct timings. The buswatch block checks the Read/Write data bus timings. The clockgen generates the system clock, PCLK.

The APB bridge model also provides the Select line for the Trickbox.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the PSELT select line.

Accesses to any address outside the above range will select the UUT via the PSEL select line.

The Poll Command in APB:

Usage: PO(expected value, mask , addr, [limit], [tag]);

On encountering a PO command in the vector file, the APB bridge model issues a read cycle (PSR) to the specified address (addr). If the read value matches the (expected) value, the next command in the vector file is executed. If the values do not match, reads are repeatedly issued to the same address until either the expected value is sampled or the number of reads issued exceeds the [limit] parameter. If the [limit] is not specified, the Polling continues indefinitely until the expected value is returned. One limitation of the POLL command is that two successive PO commands cannot be issued back-to-back. They should be separated by at least one command like PSR , PSW, PI etc.

6.2.3 Trickbox

The functionality of the UART is verified via a Trickbox model. The trickbox is an APB Slave which mimics the behaviour of an UART Slave device. Like the UART, the trickbox also contains receive and transmit FIFOs. The trickbox has a serial to parallel converter to line up data received serially from the UART and it has a parallel to serial converter to transmit the data stored in its FIFO. The trickbox has a checker module which flags off any violations in the UART transmit/receive protocol. The trickbox communicates with the APB through the APB-Interface module. All the registers in the trickbox are maintained in a separate register module.

6.2.4 Protocol Checker

The testbench includes a protocol checker which reports any protocol or timing violations during accesses to any APB slave. Timing parameters can be set using the timings.v file in the verification/verilog/tbench directory. The buswatch block which is in the verification/verilog/buswatcher directory checks the Read/Write data bus timings and reports any violations in the timings.

6.2.5 APB Multiplexer

The read data buses from the UART and the Trickbox are multiplexed together in this block before being connected to the APB Bridge model.

6.2.6 Vector Sets

The test vectors for the verification of the UART are coded into separate C files depending on the logical region of the UART which these vectors are testing. Make options have been provided to use all the test vectors together or to use them individually.

6.2.7 Parameters

The UART testbench provides some configurability options via parameters and defines. All the parameters, except MESG_ENB and SUPPRESSONRESET are configurable through the tbench.v file. MES_ENB and SUPPRESSONRESET are configurable through the buswatch.v that resides in verification/verilog/buswatcher. The define, DATA_CHECK is configurable through the apbmux.v that resides in verification/verilog/tbench.

XonPSEL

XonPSEL is used in the tbench.v file that resides in verification/verilog/tbench.

This define is used to eliminate the ‘X’s that appear on the PSEL, PWRITE and PADDR lines when these lines are not being actively driven. If XonPSEL is set to 0 in the top level of the testbench, these signals will go to ‘0’ when the corresponding line driver is inactive. If set to 1, the signals go to ‘X’ when the line driver is idle.

Verbosity

Verbosity is used in the data_driver.v, the reset_driver.v and the buscyc_drivers.v which reside in verification/verilog/bid.

This parameter is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. Every poll operation in a POLL command sequence is also reported. If Verbosity is set to 0, a message is issued only if an error occurs on a read. Similarly during a POLL command, a message is flagged only if a POLL time-out occurs.

HaltOnMismatch

This is used in the data_driver.v which resides in verification/verilog/bid.

This parameter is used to stop the simulation if a read is unsuccessful. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

SUPPRESSONRESET

SUPPRESSONRESET is used in the buswatch.v which resides in verification/verilog/buswatcher.

This parameter is used to suppress the checking of the PRDATA timings when PRESETn signal is asserted. If SUPPRESSONRESET is 1, the PRDATA set-up and hold timing violations are not flagged when PRESETn is asserted. If SUPPRESSONRESET is set to 0, PRDATA set-up and hold timing violations are flagged irrespective of PRESETn.

MESG_ENB

MESG_ENB is used in the buswatch.v which resides in verification/verilog/buswatcher.

This parameter is used to enable or disable the flagging of error messages when PRDATA does not go to zero when the UUT is not selected. The flagging of these error messages is disabled when this parameter is set to 0. If MESG_ENB is set to 1, the error messages are flagged whenever PRDATA goes to a non-zero value when the UUT is not selected. This generic will have to be set to 0 in testbenches which include more than one slave.

DATA_CHECK

DATA_CHECK is used in the apbmux.v which resides in verification/verilog/tbench.

This define is used to enable or disable the flagging of error messages when the PRDATA driven by a slave has a non-zero value when it is not selected. When DATA_CHECK has a value 1, these error messages are flagged. When it has a value 0, the error messages are not flagged. This define is used in testbenches which include more than one slave.

6.2.8 Running Simulations

The user's guide gives step by step instructions for running simulations using the UART test vectors on the Verilog source code and the Verilog netlist.

Run time logs for the RTL simulations are:

- verification/verilog/Uart_rtl/Uart_rtl_modelsim.log
- verification/verilog/Uart_rtl/Uart_rtl_LDV.log

Run time logs for the netlist simulations are:

- verification/verilog/Uart_noscan_vhdl_net_min/Uart_noscan_vhdl_net_min_modelsim.log
- verification/verilog/Uart_scanready_verilog_net_typ/Uart_scanready_verilog_net_typ_LDV.log

The code coverage results are:

- verification/verilog/Uart_rtl/Uart_rtl_modelsim_cover.log

6.3 UART Trickbox

The Trickbox is a programmable APB device designed to provide a test engineer a means to drive the non-APB input pins of the UART and sample the non APB responses of the same.

The Trickbox reads the data transmission line of the UART and performs serial to parallel conversion on data received on it. The Trickbox is capable of converting parallel data to serial data and driving the serial data on the data reception lines of the UART. The transmit and receive paths of the Trickbox are buffered with internal FIFO memories allowing up to 16 bytes to be stored independently in both the transmit and receive modes. The Trickbox has a programmable baud rate generator, frequency measurement and error checking capabilities.

The following sections detail the functionality of the Trickbox.

6.3.1 Trickbox Signal Description

APB signal descriptions

Name	Type	Source / Destination	Description
PCLK	Input	From Test Bench	APB clock
BnRES	Input	From Test Bench	Bus reset signal, Active Low
PADDR [7:2]	Input	From Test Bench	Subset of APB address bus
PRDATA [7:0]	Output	To Test Bench	Subset of unidirectional APB read data bus
PWDATA [7:0]	Input	From Test Bench	Subset of unidirectional APB write data bus
PSELT	Input	From Test Bench	Trickbox select signal from decoder. When set to 1 this signal indicates the slave device is selected and that a data transfer is required.
PENABLE	Input	From Test Bench	APB enable signal. PENABLE is asserted HIGH for one cycle of PCLK to enable a bus transfer cycle.
PWRITE	Input	From Test Bench	APB transfer direction signal, indicates a write access when HIGH, read access when LOW.

UART Data signals

Name	Type	Source/Destination	Description
UARTTXD	Input	From UART	UART transmitted serial data output, defaults to the marking state 1, when reset.
UARTRXD	Output	To UART	UART received serial data input, defaults to the marking state 1, when reset. Trickbox drives serial data on this line.
UARTCLK	Output	To UART	UART clock signal.
nUARTRST	Output	To UART	UART Reset signal for clocked elements in the UARTCLK domain.

UART Infrared data signals

Name	Type	Source/Destination	Description
nSIROUT	Input	From UART	SIR transmitted serial data output from the UART, active LOW. In the idle state, this signal remains 0, (the marking state). When this signal is set to 1, an infrared light pulse is generated which represents a logic 0.
SIRIN	Output	To UART	SIR received serial data input. In the idle state, the Trickbox drives a 1 on this line. A low pulse is driven on this signal to represent a logic value of 0.

UART Interrupt lines

Name	Type	Source/Destination	Description
UARTMSINTR	Input	From UART	UART Modem status interrupt (active HIGH).
UARTRXINTR	Input	From UART	UART Receive FIFO interrupt (active HIGH).
UARTTXINTR	Input	From UART	UART Transmit FIFO interrupt (active HIGH).
UARTRTINTR	Input	From UART	UART Receive Timeout Interrupt (active HIGH).
UARTEINTR	Input	From UART	UART Error status interrupt (active HIGH).
UARTINTR	Input	From UART	UART Interrupt (active HIGH).

UART Modem Control lines

Name	Type	Source/Destination	Description
nUARTCTS	Output	To UART	UART Clear to Send modem status input, active LOW.
nUARTDCD	Output	To UART	UART Data Carrier Detect modem status input, active LOW.
nUARTDSR	Output	To UART	UART Data Set Ready modem status input, active LOW.
nUARTRI	Output	To UART	UART Ring Indicator, active LOW

UART Scan test line

Name	Type	Source/Destination	Description
SCANMODE	Output	To UART	UART scan test hold input. This signal must be asserted HIGH during scan testing to ensure that internal data storage elements can be asynchronously reset. It must be driven LOW during normal use or when applying manufacturing test vectors.

6.3.2 Trickbox functional description

The architectural behaviour of the Trickbox is similar to that of the UART itself. The Trickbox contains the following major blocks:

APB Interface and the Register Block

The APB is a local secondary bus which provides a low-power extension to the higher AHB (or ASB) within the AMBA system hierarchy. The APB interface generates read and write decodes for accesses to status/control registers and transmit/receive FIFO memories. The register block stores data written or to be read across the APB interface. All registers are READ / WRITE, although the writeability feature of some registers is not used. Such registers are specified explicitly.

Trickbox register summary

Address (Offset from TrickboxBase)	Type	Width	Reset Value	Name	Description
0x00	R/W	8	0x00	UTDR	Data written to the Transmit FIFO and data read from the Receive FIFO
0x04	R	2	0x00	UTRSR	Receive status Register (Read)
0x08	R/W	8	0x00	UTLCR_H	Line Control Register, high byte
0x0c	R/W	8	0x00	UTLCR_M	Line Control Register, middle byte
0x10	R/W	8	0x00	UTLCR_L	Line Control Register, low byte
0x14	R/W	8	0x00	UTCR	Control Register
0x18	R	5	0x--	UTINTR	Interrupt Identification Register (Read)
0x1c	R	8	0x09	UTFR	Trickbox Flag Register (Read)
0x20	R	1	0x00	UT_FREQ_CTR_ERR	Trickbox Frequency Error Register
0x24	R/W	8	0x00	UT_FORCED_ERRS	Trickbox Forced Error Register
0x28	R/W	6	0x1f	UT_SET_PINS	Trickbox Set Pin Register
0x2c	R	2	0x00	UT_CHECK_PINS	Trickbox Check Pin Register (Read)
0x30	R/W	8	0x00	UTILPR	Trickbox IRLP Baud16 Counter Load Value Register
0x34	R/W	8	0x00	UTBITSFT_DATA	Trickbox IRLP Pulse Shift Register
0x38	R/W	6	0x00	UTBITSFT_DATA_2	Trickbox IRLP Pulse Shift Register
0x3C	W	8	0x00	UTCLKREG	UARTCLK Timing Register
0x40	W	2	0x00	RSTMODE_REG	Synchronisation & Reset bits Register
0x44	R/W	4/2	0x0	UTDMACR	Trickbox DMA Control Register
0x48	R/W	8	0x00	UTSTPARITY	Trickbox Stick Parity Register
0x4C	R/W	6	0x00	UTFBRD	Trickbox Fractional Baud Rate register

Transmit and Receive FIFOs

The transmit FIFO is a First-In, First-Out memory buffer, 8-bit wide, 16 word deep. Any data written to the Trickbox data register is stored in this FIFO till it is read out by the transmit logic. The FIFO can be disabled to act as a one-byte holding register.

The receive FIFO is a First-In, First-Out memory buffer, 11-bit wide, 16 word deep. Received data and corresponding error bits are stored in the receive FIFO by the receiver logic until read out by the CPU across the APB interface. The FIFO can be disabled to act as a one-byte holding register.

Transmitter Logic

The transmitter logic performs parallel-to-serial conversion on the data read from the transmit FIFO. Control logic outputs the serial bit stream beginning with a start bit, data bits, LSB (Least Significant Bit) first, followed by parity bit and then stop bits according to the programmed configuration in control registers. The baudrate is determined according to the following formula

Bitwidth = 16 * Divisor * UartClk

The divisor is the value programmed in UARTLCR_L/M register pair.

Receiver Logic

The receiver logic performs serial-to-parallel conversion on the received bit stream after a valid start pulse has been detected. The Trickbox needs to detect the high to low edge of the start bit to correctly interpret the data bit stream. Parity and framing error checking are also performed and the data with associated parity and framing error bits are written to the receive FIFO. The receiver logic also allows reception of data from the Uart which has been generated at a baud rate by the use of the fractional baud rate divider. The pulse width is determined according to the following formulae.

UartBaud = 64 * Divisor + FracDiv

ActBitPerInt = INTEGER((UartBaud* 16) / 64)

IrDA logic

The IrDA protocol followed by the Trickbox is reverse to that of the UART. The Trickbox (when enabled in the IrDA mode) reads the modulated bit stream on the nSIROUT line where it is held at logic value 0 in the idle state and when transmitting a data bit of 1. A bit value of 0 is represented by a high pulse of width 3/16th of the actual bit period. The time between two successive bit 0' s is one bit period. The IrDA pulse width is calculated by the Trickbox according to the formula

IrDA Pulse width = 3 * Divisor * UartClk

The Trickbox transmits a return-to-zero bit stream to the UART on the SIRIN line where in the idle state the SIRIN line is driven to 1 and a bit value of zero is represented by a low pulse (width $3/16^{th}$ of a bit period).

In the Low Power IrDA mode, the Trickbox functionality is almost identical to the High Power IrDA mode except that the pulse width is determined by the formula

IrDA Low Power width = 3 * ILPR_DVSR * UartClk

Based on the values programmed in the UTBITSFT_DATA register and the UTBITSFT_DATA_2 register, the bit stream transmitted by the Trickbox to the UART on the SIRIN line can be jittered.

The width of the IrDA pulse can be reduced on the basis of the value programmed into the UT_CR register.

Clock and Reset generation

The value programmed into the UTCLKREG register decides the time period in nanoseconds, of the internally generated UARTCLK. In case the value programmed is an odd number, the low phase of the clock signal has a 1 ns longer duration than the high phase.

The value programmed into bit 0 of the RSTMODE_REG register decides the level on the nUARTRST signal driven to the UART. The new value programmed will appear on the nUARTRST line coincident with the next falling edge of UARTCLK. When PRESETn goes low, nUARTRST is also activated asynchronously.

If a '1' is written to bit 1 of the RSTMODE_REG register, PCLK is routed to the UARTCLK output; else internally generated UARTCLK will be routed out. This feature is useful in ensuring phase equivalence in cases where PCLK and UARTCLK have the same frequency. The second and third bits of this register will enable PCLK and the internally generated UARTCLK respectively. To avoid glitches during switching, it has to be ensured that switching takes place only after the corresponding clock enable status bits stored in the 4^{th} and 5^{th} bit position of the register is set. The trickbox uses the same UARTCLK as that selected as an output.

6.3.3 Trickbox Register Description

The base address of the Trickbox is not fixed and may be different for any particular system implementation. However, the offset of any particular register from the base address is fixed.

UTDR: Data register [8] (+0x00)

This is an 8-bit read/write register for all data transfers to or from the internal Trickbox. If the FIFO is enabled, data written to this register is pushed onto the transmit FIFO. If the FIFO is not enabled, it is stored in the holding register (bottom word of the FIFO). This write initiates transmission from the Trickbox if the Trickbox is enabled. If the Trickbox is disabled, the data is held in the FIFO till the Trickbox is enabled.

On reads, this register gives the 8-bit data byte received by the Trickbox and stored in the receive FIFO, and the UTRSR register holds three status bits associated with that data character. Data received and error status is pushed onto the receive FIFO holding register if the Trickbox is enabled and the receive FIFO is not full. The value read from UTDR is zero on reset and when receive FIFO is empty. Data bytes received by the Trickbox when the receive FIFO is full are just ignored.

Bits	Name	Read/Write	Function
7:0	DATA	R/ W	Receive (read) data character Transmit (write) data character

The received data must be read first from UTDR followed by the status error associated with the data from UTRSR. This read sequence cannot be reversed.

UTRSR: Receive Status Register [2] (+0x04)

This is a read only register. Receive status is read from this register. The status information corresponds to the data character read from UTRD prior to reading UTRSR. When all the bytes in the receive FIFO of the Trickbox are read out then the UTRSR automatically gets cleared. All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
7:2		R	0 (No function)
1	Framing Error (FE)	R	Trickbox framing error flag. When this bit is set to 1, it indicates that the received character did not have a valid stop bit (a valid stop bit is

			1). This bit is associated with the character at the top of the FIFO.
0	Parity Error (PE)	R	Trickbox parity error flag. When this bit is set to 1, it indicates that the parity of the received data character does not match the parity selected in UTLCR_H(bit 2). This byte is only associated with its data byte and once read, the bit value will change to the parity error of the next data byte.

UTLCR_H: Trickbox line control register, High byte [6] (+0x08)

This register accesses bits 22 to 16 of the Trickbox bit rate and line control register (UTLCR). All the bits are cleared to 0 when reset. Note that bit 7 and bit 0 are different to the corresponding bits in UARTLCR_H.

Bits	Name	Read/Write	Function
7	Bit Shift Enable (BITSFT_ENABLE)	R/W	Enable the IrDA Bit Shift mode.
6:5	Word-length (WLEN)	R/W	Trickbox data character word length. The select bits indicate the number of data bits transmitted or received in a frame as follows: 11 = 8 bits 10 = 7 bits 01 = 6 bits 00 = 5 bits
4	Enable FIFOs (FEN)	R/W	If this bit is set to 1, transmit and receive FIFO buffers are enabled (FIFO mode). When cleared to 0 the FIFOs are disabled (character mode) that is, the FIFOs become 1-byte deep holding registers.
3	Two Stop Bits Select (STP2)	R/W	If this bit is set to 1, two stop bits are transmitted at the end of the frame.
2	Even Parity Select (EPS)	R/W	If this bit is set to 1, Even parity generation and checking is performed during transmission and reception, which checks for an even number of 1s in data and parity bits. When cleared to 0 then Odd parity is performed which checks for an odd number of 1s. This bit has no effect when parity is disabled by Parity Enable (bit 1) being cleared to 0.
1	Parity Enable (PEN)	R/W	If this bit is set to 1, parity checking and generation is enabled, else parity is disabled and no parity bit added to the data frame.
0	Reserved		Read as 0

UTLCR_M : Trickbox line control register, middle byte [8] (+0x0c)

This register accesses bits 15 to 8 of the UTLCR register. All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
7:0	Baud Rate [15:8] (BAUD DIVMS)	R/W	Most significant byte of baud rate divisor. These bits are cleared to 0 at reset.

UTLCR_L : Trickbox line control register, Low byte [8] (+0x10)

This register accesses bits 7 to 0 of the UTLCR register. All the bits are cleared to 0 when reset.

Bits	Name	Read/Write	Function
7:0	Baud Rate [7:0] (BAUD DIVLS)	R/W	Least significant byte of baud rate divisor. These bits are cleared to 0 at reset.

Note : UTLCR_H, UTLCR_M and UTLCR_L form a single 23-bit wide register (UTLCR) which is updated on a single write strobe generated by an UTLCR_H write. This means that, in order to internally update the contents of UTLCR_M or UTLCR_L, a UTLCR_H write must always be performed at the end.

UTCR : Trickbox Control register [8] (+0x14)

All the bits are cleared on reset.

Bits	Name	Read/Write	Function
7	Frequency Measurement Enable (FR_ENABLE)	R/W	Setting this bit to 1 enables the frequency measurement feature in the Trickbox. If this bit is set then the Trickbox internally measures the time between two edges and sets the frequency error bit to 1 if the time does not match the expected bit width. This frequency measurement technique works only if a data pattern of 0x55 is transmitted in the Normal Mode and a data pattern of 0x00 is transmitted in the IrDA mode.
6:3	IrDA decrease factor (IrDA_DEC)	R/W	This value represents the actual width of the IrDA pulse in ratio with the expected width. If these bits are e.g. = 0011 then the actual width of the IrDA pulse will be $3/16^{th}$ of the expected width of the IrDA pulse. Similarly if these bit values are 0101 then the actual width of the IrDA pulse will be $5/16^{th}$ of the expected width of the IrDA pulse.
2	Trickbox IrDA low power mode	R/W	IrDA low power mode bit. If this bit is cleared to 0, Zero bits are transmitted as an active high pulse with a width of $3/16^{th}$ of the bit period. If this bit is set to 1, Zero bits are transmitted with a pulse width, which is 3 times the period of the IrLPBaud16 input signal, regardless of the selected bit rate.
1	Trickbox IrDA enable	R/W	IrDA enable bit. This bit has no effect if the Trickbox is not enabled by bit 0 being set to 1. When the IrDA mode is enabled data is transmitted on SIRIN and received on nSIROUT line. UARTRXD line remains in the marking state (set to 1) and signal transitions on UARTTXD line will have no effect. When IrDA is disabled, SIRIN remains cleared to 0 and signal transitions on nSIROUT signal will have no effect.
0	Trickbox Enable	R/W	Trickbox is enabled if this bit is set to 1.

UTIIR : Trickbox Interrupt Identification Register [8] (+0x18)

Interrupt status is read from UTIIR. All the bits represent the status of each of the interrupt pins of the UART and have an indeterminate value at any point of time but since the interrupts are expected to be disabled on reset in the UART, this register is likely to contain a reset value of 0x00.

Bits	Name	Read / Write	Function
7	Interrupt Error	R/W	This error bit is set if the UARTINTR is not the OR value of the other four interrupts generated by the UART. This bit should be explicitly cleared by writing to it.
6			Nothing
5	UARTEINTR	R	UART combined Error Interrupt Status bit. This bit is set to one when an error occurs in the reception of data by the UART.
4	UART Interrupt (UARTINTR)	R	UART Interrupt Status bit. This bit is set to 1 if the UARTINTR interrupt is asserted.
3	Receive Timeout Interrupt Status (RTIS)	R	Receive Timeout Interrupt Status bit. This bit is set to 1 if the UARTRTINTR receive interrupt is asserted.
2	Transmit Interrupt Status (TIS)	R	Transmit Interrupt Status bit. This bit is set to 1 if the UARTTXINTR transmit interrupt is asserted.
1	Receive Interrupt Status (RIS)	R	Receive Interrupt Status bit. This bit is set to 1 if the UARTRXINTR receive interrupt is asserted.
0	Modem Interrupt Status (MIS)	R	Modem Status Interrupt Status bit. This bit is set to 1 if the UARTMSINTR modem status interrupt is asserted.

UTFR : Trickbox Flag Register[8] (+0x1c)

After reset TXFE, RXFE and TXHE are set to 1.

Bits	Name	Read / Write	Function
7	Receive Busy (RXBUSY)	R	The Trickbox is receiving data.
6	Transmit Busy (TXBUSY)	R	The Trickbox is transmitting data.
5	Transmit FIFO Full (TXFF)	R	If the FIFO is disabled, this bit is set when the transmit holding register is full. If the FIFO is enabled, this bit is set if the transmit FIFO is full.
4	Transmit FIFO Half Empty (TXHE)	R	If the FIFO is disabled, this bit is set when the transmit holding register is empty. If the FIFO is enabled, this bit is set if the transmit FIFO contains 8 bytes.
3	Transmit FIFO Empty (TXFE)	R	If the FIFO is disabled, this bit is set when the transmit holding register is empty. If the FIFO is enabled, the TXFE bit is set when the transmit FIFO is empty.
2	Receive FIFO Full (RXFF)	R	If the FIFO is disabled, this bit is set when the receive holding register is full. If the FIFO is enabled, this bit is set when receive FIFO is full.
1	Receive FIFO Half Full (RXHF)	R	If the FIFO is disabled, this bit is set when the receive holding register is full. If the FIFO is enabled, this bit is set when receive FIFO is full.

0	Receive FIFO Empty (RXFE)	R	If the FIFO is disabled, this bit is set when the receive holding register is empty. If the FIFO is enabled the RXFE bit is set when the receive FIFO is empty.

UT_FREQ_CTR_ERR: Trickbox Frequency Error Register [1] (+0x20)

After reset, the Frequency Error Bit is set to 0. Note that this frequency measurement technique works only if a data pattern of 0x55 is transmitted in the Normal Mode and a data pattern of 0x00 is transmitted in the IrDA mode.

Bit	Name	Read / Write	Function
0	Frequency Measurement Error	R	This bit is set when the Trickbox Frequency Measurement Bit is set in UTCR and there is an error in the data stream. That is if the time between any two successive edges in the data stream differs from that of the expected bit width by more 5 ns, this bit is set to 1. It should be manually cleared by writing a value 0 to this register.

UT_FORCED_ERRS : Trickbox Forced Error Register [8] (+0x24)

All bits are cleared to value 0 on a reset. All bits should be 0 for normal transmission.

Bit	Name	Read/Write	Function
7	Receive Jitter Sign	R/W	If this bit is 1 then the jitter factor is –ve. The sampling of data bits is done before the exact centre of the subsequent data bits. If this bit is 0, then the jitter factor is +ve. The sampling of data bits is done after the exact centre of the subsequent data bits.
6:5	Receive Jitter Factor	R/W	These bits determine the number of UARTClock cycles by which the jittering takes place. A value of 00 = no jitter (sampling takes place after 8 clk cycles) 01 = jitter of 1 clock cycle. If the receive Jitter sign is –ve, then sampling of data is done after 7 clock cycles. If receive jitter sign is +ve, then sampling of data is done after 9 clock cycles 10 = jitter of 2 clock cycles 11 = jitter of 3 clock cycles
4	Transmit Jitter Sign	R/W	If this bit is 1 then the jitter factor is -ve. The start bit of the data frame transmitted by the Trickbox is lengthened. This causes the UART to sample all the subsequent data bits before the midpoint of that bit. If this bit is 0 then the jitter factor is +ve. The start bit of the data frame transmitted by the Trickbox is shortened. This causes the UART to sample all the subsequent data bits after the midpoint of that bit.
3:2	Transmit Jitter Factor	R/W	These bits determine by how much the length of the start bit is changed. A value of 00 = no jitter (start bit length = expected bit width length). 01 = jitter of 1 clock cycle. If the transmit Jitter sign is set to 1, the start bit is longer by 1 UARTCLK periods. If the transmit jitter sign is set to 0, the start bit is shorter by 1 UARTCLK period.

			10 = jitter of 2 clock cycles 11 = jitter of 3 clock cycles
1	Framing Error	R/W	This bit will introduce a framing error in the data stream if set to 1. The stop bit will not be 1. If the extra stop bit is enabled then the extra stop bit also will not be a logic value 1. It will be 0.
0	Parity Error	R/W	This bit introduces a parity error in the data frame. If the parity enable bit is set in the control register of the Trickbox, then the parity bit will be forcefully set to the wrong value regardless of the parity type if this bit is 1. This bit will have no effect if the parity is not enabled.

UT_SET_PINS : Trickbox Set Pin Register [5] (+0x28)

This register provides the ability to drive certain input lines of the UART (which are output lines of the Trickbox).

Bit	Name	Read / Write	Function
7	Scanmode pin	R/W	Programmable Test stimulus to SCANMODE
6			
5	RI	R/W	Programmable Test stimulus to RI
4	SIRIN	R/W	Programmable Test stimulus to SIRIN
3	RX	R/W	Programmable Test stimulus to RX pin. This will have effect only if the Trickbox is disabled.
2	DSR	R/W	Programmable Test stimulus to nUARTDSR
1	DCD	R/W	Programmable Test stimulus to nUARTDCD
0	CTS	R/W	Programmable Test stimulus to nUARTCTS

UT_CHECK_PINS : Trickbox Check Pin Register [5] (+0x2c)

This register provides the ability to read certain output lines of the UART (input lines of the Trickbox)

Bit	Name	Read / Write	Function
5	Out2	R	Status of OUT2 pin of the UART
4	Out1	R	Status of Out1 pin of the UART
3	RTS	R	Status of RTS pin of the UART
2	DTR	R	Status of DTR pin of the UART
1	nSIROUT	R	Status of nSIROUT pin of the UART
0	UARTTXD	R	Status of UARTTXD pin of the UART

UTILPR : Trickbox IRLP Baud16 Counter Load Value Register [5] (+0x30)

Bit	Name	Read / Write	Function
7:5	Reserved	-	Reserved
4:0	IrDA Low Power Divisor [4:0] (IRLP_DVSR)	R/W	5-bit low power divisor value. These bits are cleared to 0 at reset.

UTBITSFT_DATA : Trickbox IRLP Pulse Shift Register [8] (+0x34)

Bit	Name	Read / Write	Function
7:5	Start Pulse	R/W	This value specifies the starting IrDA pulse from which pulses need to be jittered.
4:2	End Pulse	R/W	This value specifies the ending IrDA pulse up to which pulses need to be jittered.
1:0	Shift Factor	R/W	This value specifies the time duration by which each IrDA pulse is to be jittered. 00 – No Jitter 01 – Shift Factor = 0.5 * (Baud16 Period) 10 – Shift Factor = Baud16 Period 11 – Shift Factor = 1.5 * (Baud16 Period)

UTBITSFT_DATA_2 : Trickbox IRLP Pulse Shift Register 2 [6] (+0x38)

Bit	Name	Read / Write	Function
7:6	Reserved	-	
5:3	Pulse Number	R/W	This value specifies the IrDA pulse which needs to be jittered.
2:0	Shift Factor	R/W	This value specifies the time duration by which the IrDA pulse is to be jittered. The time duration is specified in terms of the number of Baud16 periods by which the jittering is to be done.

UTCLKREG : Trickbox UARTCLK Timing Register [8] (+0x3C)

Bit	Name	Read / Write	Function
7:0	UARTCLK Time Period	W	This value specifies the time period of UARTCLK in nanoseconds.

RSTMODE_REG : Trickbox Synchronisation and Reset Bits Register [2] (+0x40)

Bit	Name	Read / Write	Function
7:6			Reserved
5	PCLKON	R	Status bit denoting PCLK can be routed out without glitches
4	REFCLKON	R	Status bit denoting internally generated UARTCLK can be routed out without glitches
3	REFCLKGEN	R/W	Enables internally generated UARTCLK to be routed out to UARTCLK
2	PCLKGEN	R/W	Enables PCLK to be routed out to UARTCLK
1	Clock interconnect status (CLKSTATUS)	R/W	1 – PCLK is routed to UARTCLK 0 – UARTCLK is internally generated in the Trickbox.
0	Reset level (RESETBIT)	R/W	The value programmed in this bit decides the level on the nUARTRST

UTDMACR : Trickbox DMA Status and Clear Register [6] (+0x44)

The UTDMACR register provides the DMACLR signals and a mechanism for checking the status of the four DMA request lines. Writing a “1” to bit 1 or bit 0 will generate a 2 PCLK wide pulse on the UARTTXDMACLR and UARTRXDMACLR lines respectively. For single transfers this should be asserted one clock cycle after the request has gone high, and for bursts it should be asserted during the transmission of the last word in the burst. The DMACLR signals should be de-asserted 1 clock cycle after the request line has been de-asserted.

Bit	Name	Read / Write	Function
7:6			Reserved
5	UARTRXDMABREQ	R	UART receive DMA burst request status
4	UARTTXDMABREQ	R	UART transmit DMA burst request status
3	UARTRXDMASREQ	R	UART receive DMA single request status
2	UARTTXDMASREQ	R	UART transmit DMA single request status
1	UARTRXDMACLR	W	Clears the UART receive DMA requests when set.
0	UARTTXDMACLR	W	Clears the UART transmit DMA requests when set.

UTSTPARITY : Trickbox Stick Parity Register [8] (+0x48)

The UTSTPARITY contains the Stick Parity Select (SPS) bit. Writing a “1” to this register bit enables the Stick Parity function. A write of “0” to this bit clears the Stick Parity function and normal parity checking occurs. The Stick Parity bit is assigned to bit position 7, whilst bits 0 – 6 are reserved.

Bit	Name	Read / Write	Function
7	SPS	R/W	Trickbox Stick Parity bit
6:0			Reserved

UTFBRD : Trickbox Fractional Baud Rate Divisor [6] (+0x4C)

The UTFBRD contains the Fractional Baud rate divisor. This is used when receiving data from the Uart which has been transmitted at a baud rate generated by usong the Uart fractional baud rate divisor. The transmit section of the trickbox does not support this function.

Bit	Name	Read / Write	Function
7:6			Reserved
5:0	UTFBRD	R/W	Fractional Baud rate divisor. These bits are cleared to 0 at reset.

Functional Tests

The following sections describe the UART functional verification tests, which use the UART Trickbox .

6.3.4 Register Tests

Reset Value Tests

After a reset, each of the following registers are read to check if they contain the specified reset value.

Register Name	Reset Value	Register Name	Reset Value
UARTRSR	0x00	UARTMIS	0x00
UARTLCR_H	0x00	UARTDMACR	0x00
UARTCR	0x300	UARTPeriphID0	0x11
UARTFR	0x90	UARTPeriphID1	0x10
UARTILPR	0x00	UARTPeriphID2	0x04
UARTIBRD	0x00	UARTPeriphID3	0x00
UARTFBRD	0x00	UARTPCellID0	0x0D
UARTIFLS	0x12	UARTPCellID1	0xF0
UARTIMSC	0x00	UARTPCellID2	0x05
UARTRIS	0x00	UARTPCellID3	0xB1
UARTTCR	0x00	UARTITOP	0x00

Register Read Write tests

A data pattern is written to each of the writeable registers UARTLCR_H, UARTCR, UARTILPR, UARTTCR, UARTTMR and UARTTISR.

The register contents are read back to see if it contains the written value. If the values are the same then the register is declared to be writeable and readable and the test passes.

6.3.5 FIFO Disabled Tests

These are a series of tests which test transmission and reception of data by the UART through the Trickbox. These tests are conducted for different combinations of baud rate, word length, parity type and stop bits. The FIFOs are disabled.

Transmission Test

In each transmit test the UART and the Trickbox are disabled, a byte is written to the data register of the UART. Both the devices are enabled and then allowed to run till the byte is completely transmitted. The data byte is then read from the Trickbox and checked for error free reception. The flags in the UARTFR (flag register) are checked to see if they have the expected values before, during and after transmission of the byte. The Rx FIFO flags are expected to be unchanged throughout the transmission test. The BUSY flag goes high as soon as a byte is written to the transmit FIFO and remains high as long as the UART is transmitting the byte. Tx FIFO flags TXFE (transmit

FIFO empty) is true when the transmit FIFO is empty. TXFF is asserted as soon as a byte is written to the transmit FIFO but is deasserted when this byte is shifted to the shift register of the UART for transmission.

Reception Tests

In the receive test, the UART and Trickbox are disabled, a byte is written to the data register of the Trickbox. Both the devices are enabled and then allowed to run till the receive FIFO becomes full. The data byte is read from the UARTDR register followed by the status from the UARTRSR register. The reception is expected to be error free. Flag checks are conducted before and after reception of the byte. The transmit FIFO flags (TXFE, TXFF and BUSY) are unaffected. They represent the idle state of the transmit FIFO. The RXFF flag goes high only after a byte has been received and goes low after the receive FIFO has been read out. At all other times, RXFF flag is 0 and RXFE flag is 1. This test is repeated for 16 different bytes.

Simultaneous Tests

These two tests are similar to the transmit and receive tests except that they are conducted simultaneously. Sixteen bytes are transmitted from the Trickbox to the UART and 16 from the UART to the Trickbox. The flags are verified whether they reflect the status of the transmit and receive FIFOs correctly.

The FIFO disabled tests are conducted for the following configurations.

- Divisor value 2, Word length 8, No parity, No extra stop bit
- Divisor value 4, Word length 6, Odd parity, No extra stop bit
- Divisor value 3, Word length 7, Even parity, No extra stop bit
- Divisor value 1, Word length 8, No parity, Extra stop bit
- Divisor value 2, Word length 5, Odd parity, Extra stop bit

6.3.6 FIFO Enabled Tests

Back to Back Transmissions

The purpose of these tests is to check the FIFO operations as well as bursts of back to back transmissions and receptions of data by the UART. A configuration of divisor value of 2, word length 5, no parity and no extra stop bit has been selected for this test. The FIFOs are enabled for this test.

- Transmission tests

  Initially the UART and the Trickbox are disabled and configured.. One byte is written to the UART. The UART and Trickbox are then both enabled. The byte is transmitted from the UART to the Trickbox

  Flags are checked before, during and after transmission. The data byte is verified whether it has been received error free by the Trickbox. The UART is again disabled and 2 bytes are written to it. The 2 bytes are again transmitted to the Trickbox and verified. The number of bytes written to the UART increase with each iteration and after 16 bytes are transmitted at one shot, then the number of bytes transmitted per iteration reduces to 1. A total of 200 bytes are transmitted this way to check the stability of the FIFOs. The receive FIFO flags are expected to remain unchanged throughout the transmission test. The BUSY flag remains high as long as the bytes are present in either the transmit FIFO or shift register (being transmitted).

- Reception tests

  This test transmits 200 bytes from the Trickbox to the UART. Here the UART and Trickbox are disabled, configured and then enabled. They are not disabled till the entire test is over. The bytes are sent in bursts of increasing number till a maximum of 16 bytes are received by the UART in one shot. The number of bytes wraps around to 1 again. The transmit empty flag is expected to remain asserted throughout the test, while the receive FIFO empty flag is asserted before reception and deasserted after one byte has been received. The receive FIFO full flag is checked to be asserted when 16 bytes are received by the UART.

- Simultaneous tests

  This test is a combination of the previous two tests. The UART is kept enabled throughout the test to check the wrap around feature of the read and write pointers of the FIFOs.

6.3.7 UART Disable Tests

This is a FIFO enabled test case where the Tx FIFO is filled, UART is enabled and allowed to run till it starts transmitting. The UART is disabled and then enabled. This tests a disable cycle within a character.

Next it is enabled for a single character duration, and disabled for a character duration. This tests that the character completes and then transmission resumes correctly.

Next it is allowed to run till all the remaining bytes have been transmitted. The data is then read out from the TrickBox to check if the Tx FIFO of the UART retained the bytes during its temporary disabling. The TX FIFO is then written to, the UART enabled, allowed to run till it starts transmitting and then disabled. One byte of data is read out of the Trickbox and the rest of the data should not be transmitted.

With single byte in the Tx FIFO, UART is disabled during its transmission. After the expected cycles a character takes to be transmitted, the UART is enabled. This tests that the character completes and when the UART is enabled, UART transmits nothing.

In the receive test the trickbox is written to, then the UART is disabled when data starts to be transmitted and only one word should be received in the UART.

In the hardware flow control test, hardware flow control is enabled and UART Tx FIFO is filled - then the UART is disabled during the transmission of the first byte. The UART is then re-enabled and allowed to run till transmission completes. This tests that the character completes after UART disabled and transmission resumes correctly after re-enabling, under hardware flow control

6.3.8 Interrupt Tests

FIFO Disabled Tests

These tests check the generation of transmit and receive interrupts. The values of UARTTXINTR, UARTRXINTR and UARTINTR are checked through the registers (UARTRIS, UARTMIS) and UTIIR (Interrupt Identification Registers of the UART and Trickbox).

The raw and masked interrupts are checked throughout the test sequence by appropriate disabling and enabling of the respective interrupt mask bits within the UARTIMSC register.

One byte is written to the UART and the UART is enabled and allowed to run. The transmit interrupt is checked for a high value when the transmit FIFO is empty, a low value when a data byte is written to the transmit FIFO, and a high value when the data byte has been shifted out of the transmit FIFO to the shift register for transmission

One byte is written to the Trickbox and the Trickbox and UART are both enabled and allowed to run till the UART receives the data byte. The receive interrupt is checked whether it is deasserted when the receive FIFO is empty, asserted after it receives the whole data byte and deasserted once again after this data byte has been read out of the receive FIFO.

Simultaneous occurrence of transmit and receive interrupts is also checked by writing data bytes to the Trickbox and UART and allowing full duplex transmission.

FIFO Enabled Tests

The UART and Trickbox are initially disabled, then UART interrupts are enabled. One byte of data is written to the UART, the interrupts are checked to be zero. The UART is then enabled, the byte transmitted and the Tx interrupt checked to be set. The interrupts are then cleared, the UART is disabled and another byte is written to the Tx FIFO. The UART is then enabled, the byte transmitted and the interrupt checked to be set again.

These tests are similar to the FIFO disabled case except that enough bytes are transmitted from the UART and the Trickbox in order to set the interrupts based upon the programmed watermark levels. The transmit interrupt is expected to go low when there are more than the watermark level written to the transmit FIFO of the UART. It is asserted again when the level falls below the watermark level in the transmit FIFO. The receive interrupt is expected to be asserted as soon as the watermark level is reached within the UART receive FIFO. It remains asserted as the receive FIFO fills up as more bytes arrive on the data line. The receive interrupt is cleared when the receive FIFO is emptied to below the watermark level.

Simultaneous occurrence of transmit and receive interrupts is checked by writing to the Trickbox and UART, allowing full duplex transmission.

6.3.9 Baud Tests

These tests use the frequency measurement and frequency error detection capabilities of the Trickbox. The UART and the Trickbox are disabled, configured for a baud rate and then enabled. The Trickbox is enabled with the frequency measurement bit in its control register enabled. One word is written to the transmit FIFO and allowed to run. When the byte has been completely received, the data is read from the Trickbox for error free reception. A data byte of 0x55 is transmitted to gauge the bit period. If any of the bit widths in the entire data frame fall outside the range of 5 ns, then an error bit is set in the Trickbox register UT_FREQ_ERR. This test is conducted for seven different divisor values of

1, 3, 5, 11, 19, 29, 39

6.3.10 Fractional Baud Rate Divider Tests

These tests use the frequency measurement and frequency error detection capabilities of the Trickbox, similarly to the current Baud Tests.

The Trickbox register, UTFBRD, is for the fractional part of the baud rate divisor. It is a 6 bit wide read/write register. The contents of this register together with the contents of the UTLCR_M and UTLCR_L registers make up the whole baud rate divisor.

The frequency measurement and frequency error detection is carried out in the UartTrBaud block. The divisor value is used in a function that generates an integer to be used in the calculation of the bitwidth. The divisor, which comprises an integer part and a fractional part, is converted into a form where it can be used in the calculation of the bitwidth:

Baud Rate Divisor = $BRD_I + BRD_F = BRD_I + (m/64) = (64*BRD_I + m)/64$

where BRD_I is the integer part of the divisor, BRD_F is the fractional part of the divisor, and m is the 6-bit fractional number which is programmed into the counter.

Therefore, the calculation for the bitwidth becomes:

```
Bitwidth <= (to_integer(unsigned(CON16) * unsigned((unsigned(Divisor) * unsigned(CON64)) +
        CONV_INTEGER(unsigned(FracDiv))) * unsigned (CLKPERIOD)) * 1ns) / 64
```

where Divisor = integer part and FracDiv = fractional part.

Normal mode - Tx only

This test is carried out in normal mode with the UART transmitting to the Trickbox only. It is repeated using three different divisor values and three different fractional values (nine times in total).

The UART and trickbox are configured with the FIFO's enabled and the required baud rate. The fractional part of the baud rate is written to the UARTFBRD and UTFBRD registers. Sixteen words are written to the UART Tx FIFO.

The UART and Trickbox are enabled with the frequency measurement capability, sixteen words of data are transmitted from the UART to the Trickbox. The data is read out of the Trickbox RXxFIFO and checked for no errors, signified by the contents of the UTRSR register. The bit width is test passes if bit 0 in the UT_FREQ_CTR_ERR register is not set.

The UART and Trickbox are then disabled.

Normal and Irda modes

This test is carried out for all modes of operation. It uses the loopback facility of the UART. It is repeated for four different integer divisor values and for each of those, ten fractional values are used (forty times in total).

The UART is configured with the FIFO's enabled and the required baud rate. The fractional part of the baud rate is written to the UARTFBRD register.

Sixteen words are written to the UART Tx FIFO, then the UART is enabled. If the test is being carried out in either of the Irda modes then the SIRTEST bit is set in the UARTTCR register. The sixteen words of data are transmitted and received by the UART in loopback mode. The data is read out of the UART Rx FIFO to check for correct reception. The UART is then disabled. The fractional values are cleared back to 0 by writing 0's to the UARTFBRD register.

6.3.11 Modem Tests

These tests check the modem status flags and the interrupt UARTMSINTR. A Modem Status Interrupt should occur if a change occurs on any of the modem status lines nUARTDCD, nUARTDSR, nUARTCTS or nUARTRI. This function causes a change on each of these lines in turn and checks whether the interrupt has occurred or not. The modem lines are driven through the UT_SET_PINS register in the Trickbox. The modem pin values are checked in the UART flag register UARTFR whilst the individual raw and masked interrupts are checked in the UARTRIS and UARTMIS registers. The actual values on the interrupt pins UARTMSINTR and UARTINTR are checked through the Trickbox register UTIIR. The individual interrupts can be cleared by writing a "1" to the relevant bit within the UARTICR register.

Another test checks the assertion and deassertion of modem status flags and pins during reception and transmission in various modes. In the normal mode of operation the individual interrupts should change as before. In the IrDA mode the flags in UARTFR should reflect the changes on the modem lines, but the modem status interrupt should not be asserted even though enabled.

6.3.12 Error Tests

These tests check the error handling functionality of the UART. Different types of errors are intentionally introduced into the data stream of the Trickbox. The various error detection tests are described in the following sections.

Overrun Tests

- FIFO Disabled Case

  In this case, the UART FIFOs are disabled and two bytes are transmitted to the UART from the Trickbox. The data is then read out followed by the status register. The status register should return an error status with the overrun bit is set. The data byte value read out should be the first byte of data transmitted. The

error is cleared by writing to the UARTECR register and the status register is read again. This time the status returned should have the overrun error bit cleared.

- FIFO Enabled Case

  This test vector tests the overrun detection when the UART FIFOs are enabled. FIFOs are enabled in the UART and the Trickbox. 17 bytes of data are transmitted from the Trickbox to the UART. The first byte is read out of the UARTDR register followed by the status register. The status register continues to show the overrun bit is set till it is cleared by a write to the UARTECR register. The first 16 bytes should have been received error free from the UART. The 17th byte (the byte which caused the overrun to occur) will not get written into the receive FIFO at all.

Parity Error Check Tests

In this test, a data frame with a parity error is driven on the UARTRXD line by the Trickbox. The parity error can be introduced by setting the PARITY_ERROR bit in the UT_FORCED_ERRS register of the Trickbox. The data byte is received by the UART and the status register is checked to see if the parity error bit is set or not. If the parity error bit is not set in the UARTRSR then the test fails.

Stick Parity Tests

Stick Parity Tx – parity bit 1:

The UART and Trickbox are enabled with the Fifo's disabled, Stick Parity enabled and the Even Parity bit set to 0. The Stick Parity function in the Trickbox is set by writing to bit 7 in the UTSTPARITY register. A byte is written to the UART Tx FIFO and transmitted to the Trickbox Rx FIFO. The received byte is read from the Rx FIFO and it's parity checked by reading the Parity Error bit within the UARTRSR register . The UART and Trickbox are then disabled.

Stick Parity Tx – parity bit 0

The UART and Trickbox are enabled with the Fifo's disabled, Stick Parity enabled and the Even Parity bit set to 1. The Stick Parity function in the Trickbox is set by writing to bit 7 in the UTSTPARITY register. A byte is written to the UART Tx FIFO and transmitted to the Trickbox Rx FIFO. The received byte is read from the Rx FIFO and it's parity checked by reading the Parity Error bit within the UARTRSR register . The UART and Trickbox are then disabled.

Stick Parity Rx – parity bit 1, no error

The UART and Trickbox are enabled with the Fifo's disabled, Stick Parity enabled and the Even Parity bit set to 0. The Stick Parity function in the Trickbox is set by writing to bit 7 in the UTSTPARITY register. A byte is written to the UART Tx FIFO and transmitted to the Trickbox Rx FIFO. The received byte is read from the Rx FIFO and it's parity checked by reading the Parity Error bit as zero within the UARTRIS register . The UART and Trickbox are then disabled.

Stick Parity Rx – parity bit 0, no error

The UART and Trickbox are enabled with the Fifo's disabled, Stick Parity enabled and the Even Parity bit set to 1. The Stick Parity function in the Trickbox is set by writing to bit 7 in the UTSTPARITY register. A byte is written to the UART Tx FIFO and transmitted to the Trickbox Rx FIFO. The received byte is read from the Rx FIFO and it's parity checked by reading the Parity Error bit as zero within the UARTRIS register . The UART and Trickbox are then disabled.

Stick Parity Rx – parity bit 1, parity error

The UART and Trickbox are enabled with the Fifo's disabled, Stick Parity enabled and the Even Parity bit set to 0. The Stick Parity function in the Trickbox is set by writing to bit 7 in the UTSTPARITY register. The parity bit mask is set by writing a "1" to bit 8, the Parity Error Interrupt Mask within the UARTMISC register.

A parity error is forced to occur by writing a "1" to bit 0 of the UT_FORCED_ERRS register in the Trickbox. A byte is written to the Trickbox Tx FIFO and transmitted to the UART Rx FIFO.

The received byte is read from the UART Rx FIFO and it's parity checked by reading the Parity Error bit as "1" within the UARTRIS and UARTMIS registers. The Trickbox UARTEINTR and UARTINTR bits are also set within the UTIIR register. The UART and Trickbox are then disabled.

Stick Parity Rx – parity bit 0, parity error:

The UART and Trickbox are enabled with the Fifo's disabled, Stick Parity enabled and the Even Parity bit set to 1. The Stick Parity function in the Trickbox is set by writing to bit 7 in the UTSTPARITY register. The parity bit mask is set by writing a "1" to bit 8, the Parity Error Interrupt Mask within the UARTMISC register.

A parity error is forced to occur by writing a "1" to bit 0 of the UT_FORCED_ERRS register in the Trickbox. A byte is written to the Trickbox Tx FIFO and transmitted to the UART Rx FIFO.

The received byte is read from the UART Rx FIFO and it's parity checked by reading the Parity Error bit as "1" within the UARTRIS and UARTMIS registers. The Trickbox UARTEINTR and UARTINTR bits are also set within the UTIIR register. The UART and Trickbox are then disabled.

Framing Error Tests

The framing error test is similar to the parity bit error test. A low value is driven on the UARTRXD line instead of the stop bit by setting the FRAMING_ERROR bit in the UT_FORCED_ERRS register of the Trickbox. The data byte is received by the UART and the status register is checked to see if the framing error bit is set or not. The test fails if the framing error bit is not set.

Break Error Tests

The break error test checks the transmit break function. Two bytes are written to the UART FIFO. After the UART just starts transmitting the first byte, the Break bit is set in the register UARTLCR_H of the UART. The 1st character finishes being transmitted and then 50 idle cycles are allowed to run. During this period the transmit pin is sampled through the Trickbox to see if it has been driven to zero. The Trickbox is then disabled, UART break is released and then the Trickbox is enabled. The UART is allowed to run till it stops transmitting. The data byte is then read out from the Trickbox data register. The transmission of the first byte completes as the break bit in the UARTLCR_H register is only updated at the end of the current character. The second byte in the FIFO should have been transmitted after the break is released. This check is also done in both of the IrDA modes.

Receive Timeout Interrupt tests

The UART and Trickbox are configured and enabled. The receive timeout interrupt is enabled in the UART. One byte of data is transmitted from the Trickbox to the UART. After this byte has been completely transmitted, the devices are kept enabled and idle for 32 bit periods. The Receive Timeout interrupt is checked to be asserted only after 32 bit periods after receiving the first byte and not before.

The UART is enabled with the receive timeout interrupt enabled. The timeout interrupt is monitored to check that it is not active whilst the UART is enabled and the Rx FIFO is empty.

The final test drives the transmit pin low soon after the UART is enabled. The Receive Timeout interrupt should get asserted after a 32 bit period.

Illegal Baud Tests

The Illegal baud test checks the activity on the transmit line if an illegal divisor (0) has been programmed into the UARTIBRD register. The UART is enabled and the TX line is checked, it should stay high. The UART is disabled, configured with a non-zero divisor value and then enabled. The UART is allowed to run till it finishes transmitting the byte. The byte should be received correctly by the Trickbox which was previously prepared with the same configuration. The UART is disabled and then configured with the other illegal divisor case i.e. 0xFFFF programmed into UARTIBRD and a non-zero value in UARTFBRD. The UART is enabled and the TX line is checked, it should stay high. The UART is disabled, configured with a legal divisor value and then enabled. The UART is allowed to run till it finishes transmitting the byte. The byte should be received correctly by the Trickbox which was previously prepared with the same configuration.

6.3.13 Tolerance Tests (Jittered Input tests)

These series of tests transmit a slightly skewed data stream. The start bit of every data frame is lengthened or shortened by a jitter factor programmed into the Trickbox register UT_FORCED_ERRS. The jitter factor can be any integer value between –3 to +3 (both inclusive). A jitter factor of +2 for instance means that the start bit is shortened by $2/16^{th}$ of the bit period, so that the sampling of the data bit values is done 2 BaudClks later than usual. The data received by the UART should be identical to that transmitted by the Trickbox. The Jitter value should not affect the data reception. Similarly a jitter factor of –1 means that the start bit is lengthened by $1/16^{th}$ of the bit period, so that the sampling of the data bit values is done 1 Baud Clock period earlier than the midpoint. In this case too the data should be received by the UART error free. The jittered inputs are supplied with all programmable jitter factors in the Trickbox.

6.3.14 IrDA Tests

These tests are identical to the FIFO enabled and FIFO disabled tests of the Normal Mode except that the simultaneous transmission and reception test cases were omitted because only Half Duplex transmission is allowed. The following tests were conducted in the IrDA mode and IrDA low power mode also just like the Normal mode.

- FIFO disabled tests
- FIFO enabled tests
- Baud Tests
- Modem Tests (mentioned earlier)
- Error Tests (Overrun, Parity Error, Framing Error)
- Break Tests
- Receive Timeout Interrupt Tests

Apart from the above mentioned tests the following three special tests were conducted in the IrDA mode.

Shortened IrDA pulse tests

These tests check whether the UART rejects IrDA pulses which are smaller in width than 1 IrLPBaud16 Clock periods. The actual width of the IrDA pulse is programmed through the Trickbox as a fraction of the expected bit width. The details of this register are described in the register section of the Trickbox. The values less than 1 and equal to 1 will get rejected. Pulse widths of greater than 1 and less than 2 clocks width are indeterminate cases which may or may not be received by the UART. These values are not tested. Pulse widths of greater than 2 are accepted as valid pulses. Tests are conducted for these.

This test is conducted in IrDA low power mode also.

Half Duplex tests

These tests check the Half Duplex property of the UART in the IrDA mode. Some data bytes are written into the UART's transmit FIFO and the UART is enabled to start transmission. After some time, data bytes are written into the Trickbox transmit FIFO to cause serial data transfer over the RXD line. The UART should ignore all these bytes as long as transmission is active.

This is done by checking the flag register of the UART. The RXFE (receive fifo empty flag) should remain set throughout the test.

This test is conducted in IrDA low power mode also.

IrDA Tolerance Tests

These tests jitter the IrDA pulse stream by a time duration corresponding to one half the time period of the Baud16 signal. The data received by the UART is checked for correct reception.

6.3.15 Loopback Test

In this test, the internal loopback mode is enabled. Transmit data is written into the Transmit FIFO of the UART and transmission is allowed to complete. After the completion of transmission, the Receive FIFO of the UART is read and the data is checked for correctness.

To test the modem signals in the loopback mode, data is written to the modem signals nUARTRTS, nUARTDTR, nUARTOut2 and nUARTOut1. The signals nUARTCTS, nUARTDSR, nUARTRI and nUARTDCD are checked for correct transmission of the data.

6.3.16 Modem Hardware Flow Control tests

Request To Send (RTS) only: FIFO Disabled

The UART and Trickbox are enabled with their Fifo's disabled. The Request To Send signal, nUARTRTS, checked to be de-asserted high, in the UT_CHECK_PINS register. The RTS flow control is enabled and nUARTRTS is checked as being asserted low.

A byte is written to the Tx Fifo allowed to complete it's transmission, causing nUARTRTS to become de-asserted high indicating no more room in the UART Rx FIFO. Data is read out of the Rx FIFO and the nUARTRTS signal is checked as being asserted low.

The RTS flow control is disabled and a byte is written to the Tx FIFO and allowed to complete its transmission. There nUARTRTS is checked to ensure it remains de-asserted high.

While RTS flow control is disabled, values written to bit 11 of the UARTCR are checked that their inverse values are transferred to the nUARTRTS pin. This is checked via the UT_CHECK_PINS in the Trickbox.

The UART and Trickbox are then disabled.

Request To Send (RTS) only: FIFO Enabled

The UART and Trickbox are enabled with their Fifo's enabled. The watermark level set to half full and the TESTFIFO mode enabled.

The Request To Send signal, nUARTRTS, is checked as being de-asserted high, in the UT_CHECK_PINS register. The RTS flow control is enabled and nUARTRTS is checked as being asserted low.

Eight bytes are written to the Tx Fifo and allowed to complete it's transmission. The signal nUARTRTS remains asserted low indicating that the watermark has not been exceeded in the UART Rx FIFO. A further two bytes are written to the Trickbox FIFO and allowed to complete their transmission. The signal nUARTRTS is checked as being asserted high, indicating that the watermark level has been exceeded in the UART Rx FIFO.

Three words are read from the UART Rx FIFO, the nUARTRTS signal is checked as being asserted low, indicating that the Rx FIFO level is below the watermark.

The RTS flow control is disabled and another two words are written to the Trickbox Tx FIFO and transmission allowed to complete. The nUARTRTS signal is checked as being the inverse value of that programmed in bit 11 of the UARTCR register.

The remaining eight words are read out from the UART Rx FIFO and the nUARTRTS line is checked to have not changed.

The UART and Trickbox are then disabled.

Clear To Send (CTS) only: FIFO Disabled

The trickbox is enabled with its FIFOs disabled. It's nUARTCTS signal is checked as being set to its inactive high state.

A byte is written to the UART Tx FIFO.

The UART is enabled with its FIFOs disabled and it's CTS flow control enabled. The UART_TXD line is checked to remain at its inactive high state. The Trickbox Rx FIFO is also checked to be empty.

The Trickbox nUARTCTS signal is asserted by writing a 0 to bit 0 of the UT_SET_PINS register. The UART_TXD is checked for data transmission, and then the received value is read out the Trickbox Rx FIFO.

Another word is written to the UART Tx FIFO and then nUARTCTS is de-asserted during the transmission. A check that current character is received correctly is performed, by waiting until transmission is completed and reading the word out of the Trickbox.

The CTS flow control is then disabled and nUARTCTS is checked to have no effect on the transmission of data. Whilst the nUARTCTS line is currently inactive high, a byte is written to the UART Rx FIFO, transmitted and checked that it is received in the trickbox correctly. Then, nUARTCTS is asserted low and another word is transmitted and again checked for correct reception.

The Trickbox and UART are then disabled.

Clear To Send (CTS) only: FIFO Enabled

The trickbox with it's Fifo's enabled. It's nUARTCTS signal is checked as being set to it's inactive high state.

Eight words are written to UART Tx FIFO. The UART is then enabled with it's FIFO's and CTS flow control enabled. The UART_TXD line is checked to remain at its inactive high state. The Trickbox Rx FIFO is also checked to be empty.

The Trickbox nUARTCTS signal is asserted by writing a 0 to bit 0 of the UT_SET_PINS register. The UART_TXD is checked for data transmission, and then the eight received values are read out the Trickbox Rx FIFO.

The Trickbox nUARTCTS is de-asserted by writing a 1 to bit 0 of the trickbox UT_SET_PINS register.

Eight words are written to the UART Tx FIFO, then nUARTCTS is asserted low. Six characters are then transmitted before de-asserting nUARTCTS high. The correct transmission and reception of the data is checked by reading the six words out of the Trickbox Rx FIFO.

The remaining two words are transmitted by asserting nUARTCTS and they are checked by reading back from the

Trickbox Rx FIFO.

The CTS flow control is disabled and nUARTCTS is checked to have no effect on transmission by writing four words to the UART. After writing the second word, the nUARTCTS signal is de-asserted and but transmission remains in progress. The four words out the trickbox Rx fifo to check that the data was transmitted and received correctly.

The UART and Trickbox are then disabled.

CTS and RTS Test

The UART FIFO watermark levels are programmed to be half full, and sixteen words are written to the UART.

The UART is enabled in loopback mode with its FIFO's enabled, and CTS and RTS flow control enabled.

A check that transmission stops when the receive FIFO is half full by checking for a continuous high on the UART_TXD line after the Receive interrupt has been asserted is performed.

Two words are read from the UART Rx FIFO and a check that transmission restarts is made. The first of these two words is transmitted and received. This triggers the Rx FIFO watermark level, but by the time the nUARTRTS signal is de-asserted the second word has begun to be transmitted and will also be received correctly. Provided there are sufficient words within the UART Tx FIFO, a minimum of two words will always be transferred.

Further double reads are issue and when all the data has been transmitted from the UART Tx FIFO, the remaining data is read out of the Rx fifo.

The UART is then disabled.

6.3.17 DMA Interface

Single Transfer mode: FIFO disabled mode – Tx

The UART and trickbox are enabled with the fifo's disabled. The transmit DMA is enabled by writing to bit 1 of the UARTDMACR register. A check that the DMA transmit single request is asserted, UARTTXDMASREQ, as there is space in the Tx FIFO, and that the burst request is not asserted as the UART is in fifo disabled mode are performed. The request signals are checked via the UTDMACR register in the trickbox.

A word is written to the UART Tx FIFO which is then be transmitted. After writing the word to the UART, an immediate write to bit 0 of the UTDMACR register to generate a DMACLR signal is exercised. The UARTTXDMASREQ signal is checked for reassertion after the transmission of the word has completed.

The UART is disabled, and a check that the UARTTXDMASREQ is de-asserted. The UART is then enabled request line is checked to be is re-asserted.

The Transmit DMA is disabled the UARTTXDMASREQ is checked as being de-asserted. The transmit DMA is enabled and the request line is checked as being reasserted.

The UART, Trickbox and transmit DMA are then disabled.

Single Transfer mode: FIFO disabled mode – Rx

The UART and trickbox are enabled with their fifo's disabled. The receive DMA is enabled by writing to bit 0 of the UARTDMACR register

The DMA receive single request is checked as being not asserted, as the UART Rx FIFO is empty. The latter is checked by reading bit 3 in the UTDMACR register.

A word is written to the trickbox and then received by the UART. The DMA receive single request, UARTRXDMASREQ, is checked as being asserted.

The word is read out of the UART Rx FIFO and the request is cleared by writing a "1" to bit 1 of the trickbox UTDMACLR register.

Another word is transmitted from the Trickbox to the UART. The UARTRXDMASREQ is checked as being asserted. The UART is disabled and the request is checked as being de-asserted. The UART is enabled the request is checked for reassertion.

The Receive DMA is disabled and UARTRXDMASREQ is checked for deassertion. The receive DMA is enabled and the request checked for reassertion.

The UART, Trickbox and receive DMA are then disabled.

Burst Transfer mode: FIFO enabled - Tx

The UART Tx FIFO watermark level is programmed to the three quarter mark. The UART and Trickbox are configured with the FIFO's, TestFifo mode and transmit DMA enabled.

The single and burst transmit requests, UARTTXDMASREQ and UARTTXDMABREQ, are checked as being asserted, signifying that there is space in the UART Tx FIFO for single characters and a burst.

Fifteen words are written to the UART Tx FIFO, in three bursts of four and three single transfers. The requests are cleared after each burst. While there is still at least one burst length space in the UART Tx FIFO, both requests are asserted. When there is less than a burst length of space in the UART Tx FIFO, just the single request is asserted.

One word is successively read from the UART Tx FIFO the burst request is checked to be asserted is after three words have been read out.

The Transmit DMA is disabled and the request checked to be deasserted. The transmit DMA is re-enabled and the request is checked to be reasserted.

The remaining words are read out of the UART Tx FIFO and the single and burst requests are checked for assertion.

The UART, Trickbox, Transmit DMA and TestFifo mode are then disabled.

Burst Transfer mode: FIFO enabled - Rx

The UART Rx FIFO watermark level is programmed to the quarter level mark. The UART and Trickbox are configured with the FIFO's, TestFifo mode and receive DMA enabled.

The single and burst transmit requests, UARTRXDMASREQ and UARTRXDMABREQ, are checked for de-assertion, as the FIFO's are empty.

Fifteen words are written in test mode to the UART Rx FIFO. When there are less than four words in the Rx FIFO, only the single request is set, and that when there are more than 4 words in the fifo both the single and burst receive requests are set.

The DMA controller would service the burst requests. The data is read out of the Rx FIFO four words at a time until there are less than 4 words left. The request is cleared by writing a "1" to bit 1 of the UTDMACR register during the transfer of the last data in the burst. The remaining words are read as single characters, clearing the request after each transfer.

Four words are written into the RXFIFO and the receive single and burst request are checked for assertion. The receive DMA is disabled the requests are checked to be de-asserted. The receive DMA is re-enabled and the requests are checked to be re-asserted. The remaining data is read out of the Rx FIFO.

The UART, Trickbox, and TestFifo mode are disabled and the Rx FIFO watermark level is set to the half full level mark.

DMAONERR test

The UART and Trickbox are enabled with their FIFO's enabled. The Receive DMA is enabled with the DMAONERR bit set (bits 0 and 2 in the UARTDMACR) register. A parity error is generated by writing a "1" to bit 0 of the UT_FORCED_ERRS register in the Trickbox. TESTFIFO mode is also enabled.

Eight words are written to the Trickbox Tx FIFO and transmitted to the UART. The receive single or burst requested are checked to be is de-asserted.

The parity error is removed by writing a "0" to the UT_FORCED_ERRS register. Any interrupts are cleared by writing to the UARTICR register. The receive burst request and single requests are checked for assertion.

Eight words are read out of the UART Rx FIFO. The UART, Trickbox and TestFifo mode are then disabled.

The UART and trickbox are then re-enabled with the Fifo's enabled. The Receive DMA is enabled with the DMAONERR bit set (bits 0 and 2 in the UARTDMACR) register. TESTFIFO mode is also enabled.

Eight words are written to the Trickbox Tx FIFO and transmitted to the UART. The receive single and burst requests are checked for assertion.

A parity error is generated by writing a "1" to bit 0 of the UT_FORCED_ERRS register in the trickbox. One word is transmitted with the parity error. The receive burst request and single requests are checked for de-assertion.

Eight words are read out of the UART Rx FIFO.

The parity error is removed by writing a "0" to the UT_FORCED_ERRS register. Another eight words are transmitted from the Trickbox to the UART. The receive requests are checked to still be de-asserted.

One word out is read out of the UART Rx fifo and the interrupt cleared by writing to the UARTICR register. The receive single and burst requests are checked to be asserted.

The DMA requests are cleared, the data is read out of the UART Rx FIFO. The UART, Trickbox and TestFifo mode are then disabled.

7 UART INTEGRATION TEST DETAILS

The UART integration test vector suite allows the integrator to verify that the UART has been connected into the target system correctly. These vectors test 3 classes of signals:

a) AMBA signals (UART signals connected directly to the APB)

b) Intra-Chip signals (UART signals connected to the DMA and Interrupt Controllers)

c) Primary I/O signals (UART signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The UART Integration testbench is based on an AHB TICTalk testbench. The VHDL and Verilog TICTalk testbenches are used to verify that the UART has been connected into the target system correctly. The VHDL and Verilog TICTalk testbench files reside in the integration/vhdl and integration/verilog directories respectively. The test vectors that are applied to the TICTalk testbenches are in the integration/bustest directory. The test vectors for integration testing are written in BusTalk and then converted into .tif format to be applied to the TICTalk integration testbench.

7.1 UART VHDL Integration testbench

The AMBA TICTalk VHDL testbench used for integration testing of the UART is located in the integration/vhdl/tbench directory. The Integration test vectors are located in the integration/bustest directory.

The test vectors used with this testbench can be applied to the UART when it is part of an AMBA system design.

Figure 44 Testbench for simulating TICTalk test vectors illustrates the testbench to apply TICTalk test vectors.

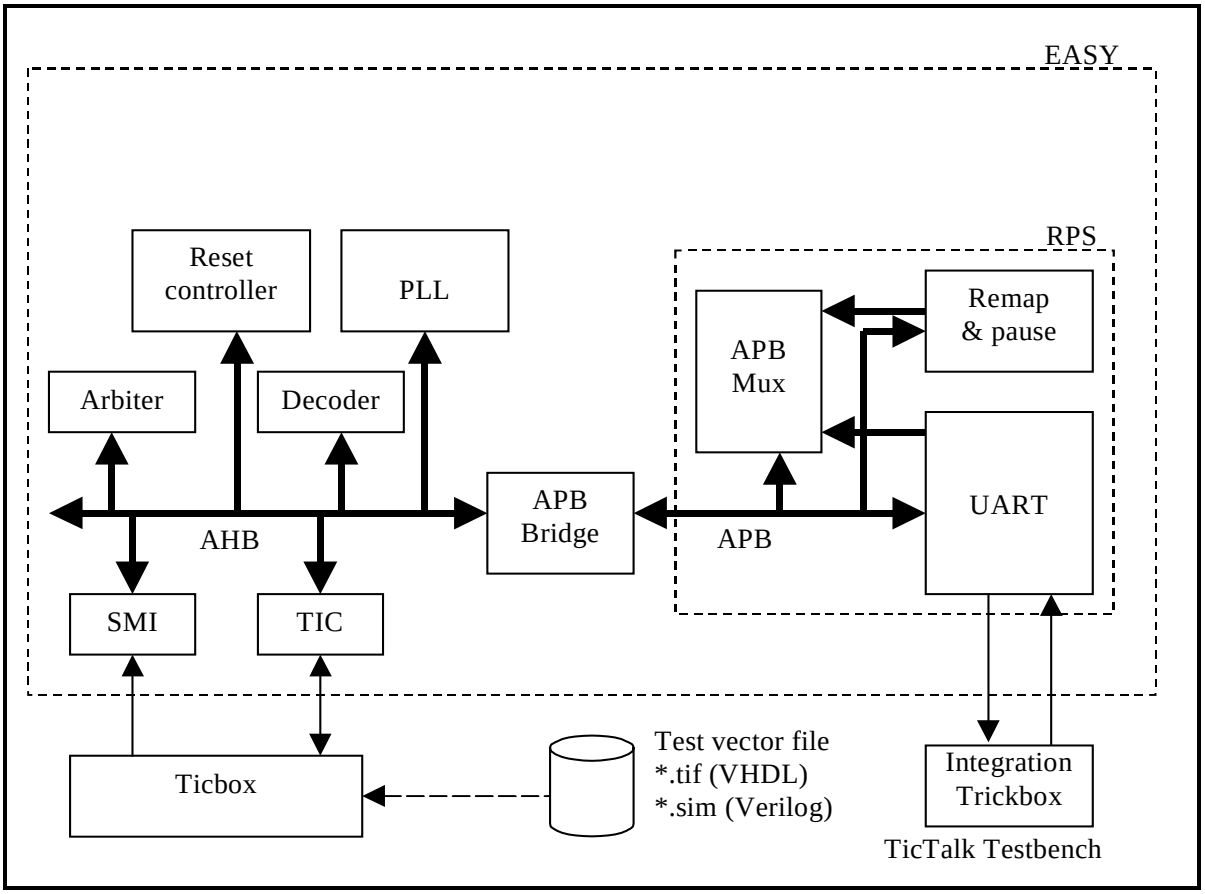

Figure 44 Testbench for simulating TICTalk test vectors

The UART VHDL Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 45 Testbench hierarchy for simulating TICTalk test vectors illustrates the hierarchy for the TICTalk testbench.

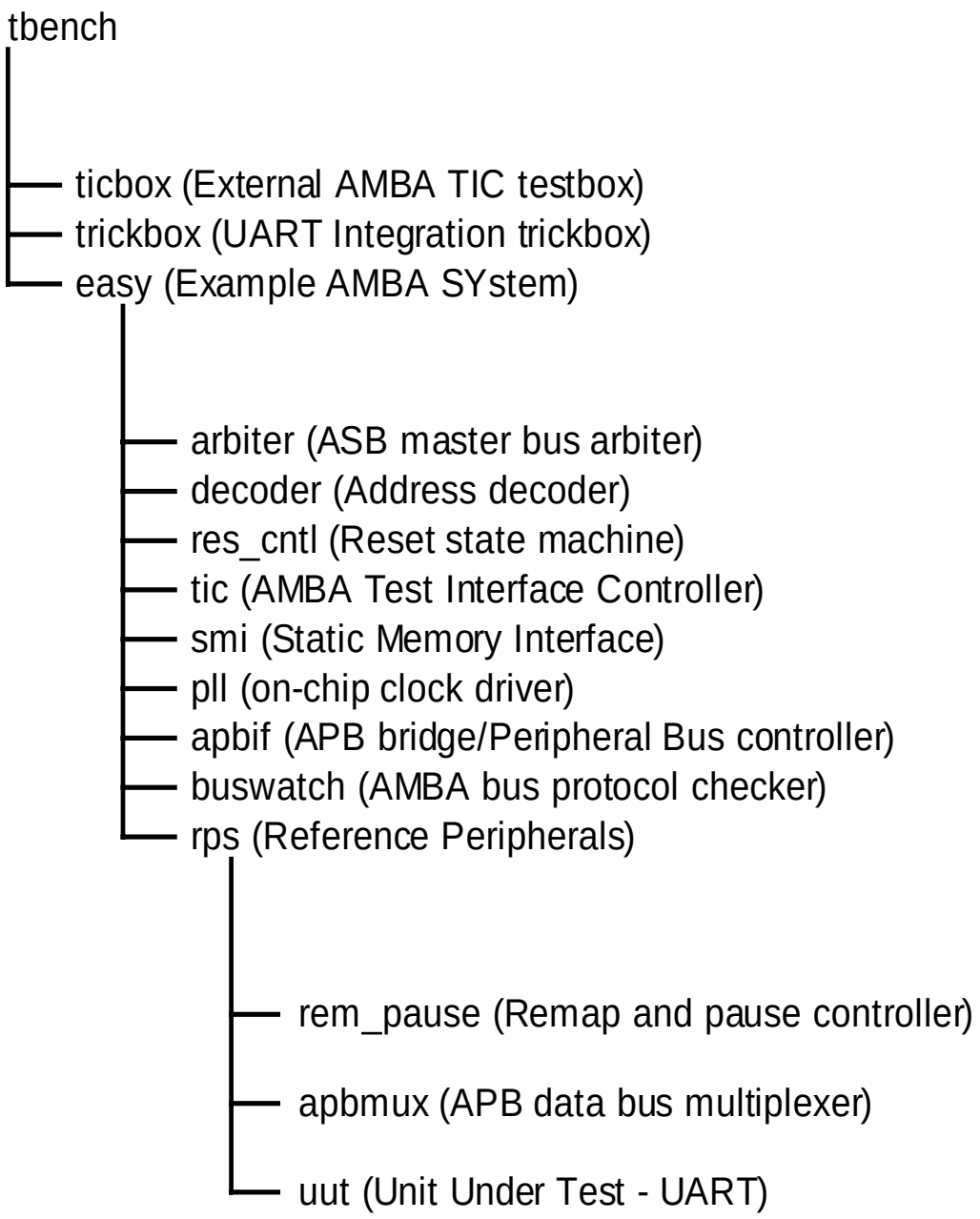

Figure 45 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to tbench.vhd, the top-level of the testbench that is used for running the integration tests of the UART. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The UART is instantiated in the rps.vhd, which is a part of the EASY system. The UARTCLK is generated in the rps.vhd. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test I/O pins.

7.1.1 Unit Under Test

The Unit Under Test may be one of the following

- UART VHDL RTL
- UART VHDL netlist from VHDL RTL Source

One out of these two versions may be referenced via the corresponding link to uut directory in integration/vhdl .

7.1.2 Test Vectors

The test vectors for the integration testing of the UART are coded into separate C files for testing the reset values of all the UART registers and for Integration testing. Make options are provided to run the tests together or individually.

7.1.3 Running Simulations

The README file in the integration/vhdl/tbench directory gives step by step instructions for running simulations using the UART Integration test vectors on the VHDL source code.

Run time logs are available in the following files:

integration/vhdl/Uart_rtl/Uart_rtl_modelsim.log

integration/vhdl/Uart_scanready_vhdl_net_max/Uart_scanready_vhdl_net_max_modelsim.log

7.2 UART Verilog Integration testbench

The AMBA TICTalk Verilog testbench used for integration testing of the UART is located in the integration/verilog/tbench directory. The Integration test vectors are located in the integration/bustest/invec directory.

The test vectors used with this testbench can be applied to the UART when it is part of an AMBA system design.

Figure 46 Testbench for simulating TICTalk test vectors illustrates the testbench to apply TICTalk test vectors.

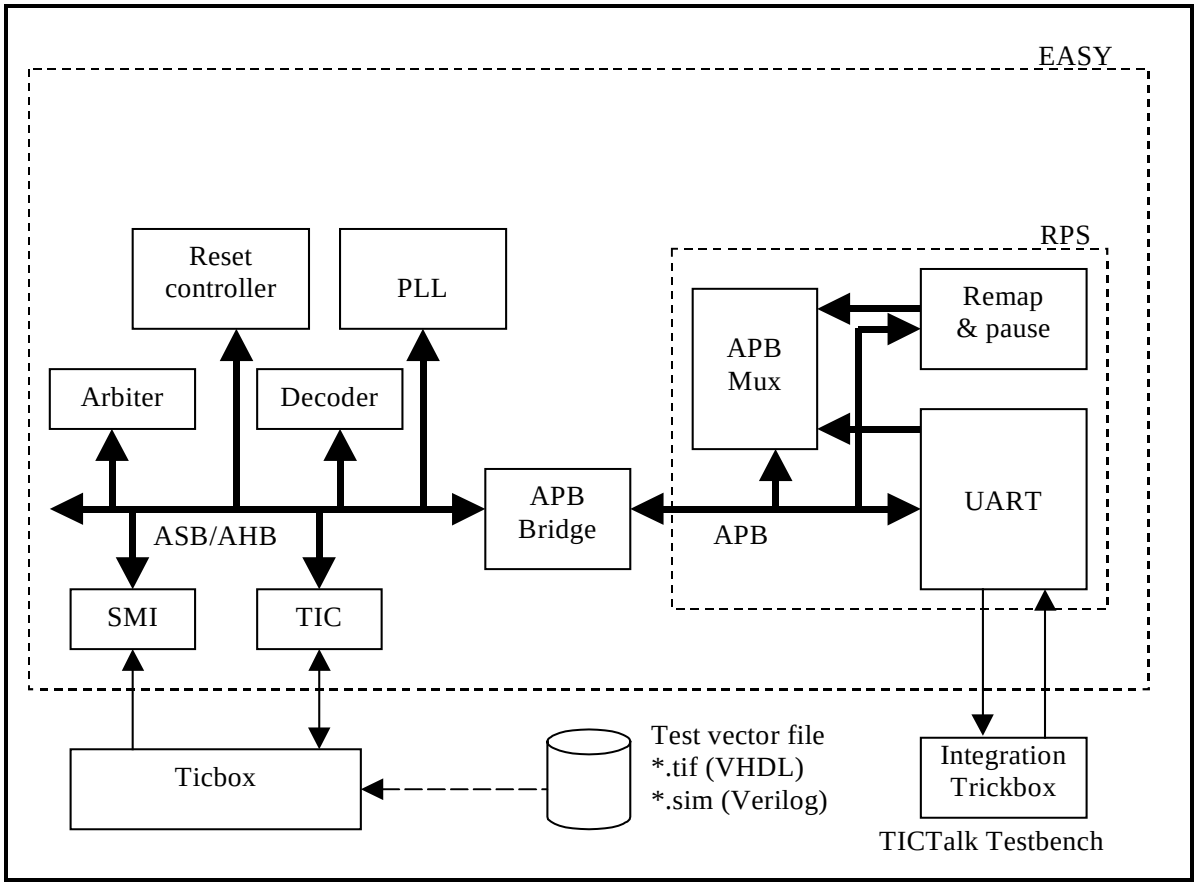

Figure 46 Testbench for simulating TICTalk test vectors

The UART Verilog Integration testbench is a stripped down version of the standard EASY system testbench. For further information refer to the Example AMBA System user guide which is part of ARM Micropack documentation.

Figure 47 Testbench hierarchy for simulating TICTalk test vectors illustrates the hierarchy for the TICTalk testbench.

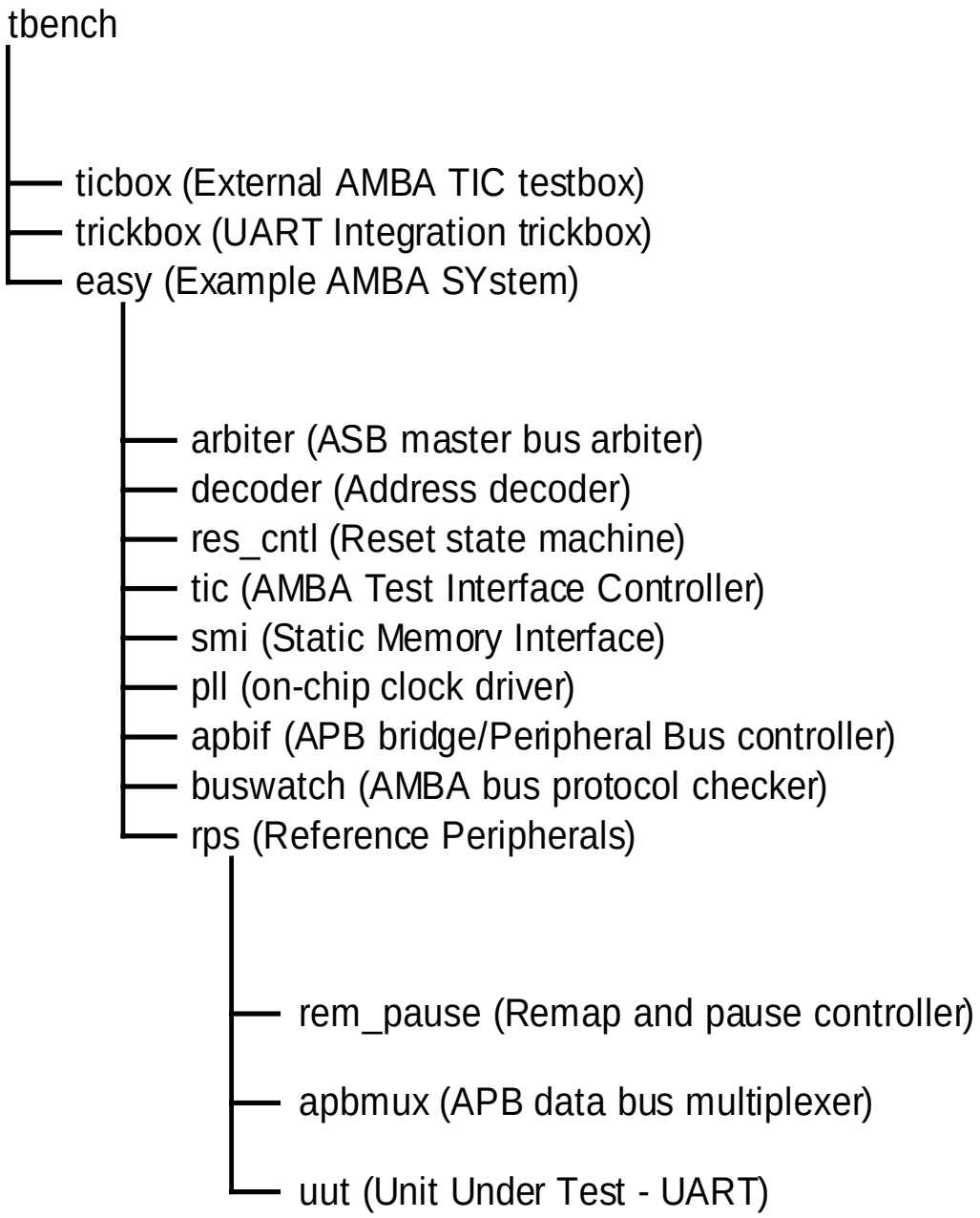

Figure 47 Testbench hierarchy for simulating TICTalk test vectors

The test vectors are applied to tbench.v, the top-level of the testbench that is used for running the integration tests of the UART. This module instantiates the model of the EASY microcontroller, the integration trickbox and the ticbox. The UART is instantiated in the rps.v, which is a part of the EASY system. The UARTCLK is generated in the rps.v. All the Scan input pins are tied low to keep them from interfering with the test process.

The integration trickbox is used to give a loopback facility to test I/O pins.

7.2.1 Unit Under Test

The Unit Under Test may be one of the following

- UART VERILOG RTL
- UART VERILOG netlist from VHDL RTL Source
- UART VERILOG netlist from VERILOG RTL Source

One out of these two versions may be referenced via the corresponding link to the uut directory in integration/verilog .

7.2.2 Test Vectors

The test vectors for the integration testing of the UART are coded into a separate C files for testing the reset values of all the UART registers and for Integration testing. Make options are provided to run the tests together or individually.

7.2.3 Running Simulations

The README file in the integration/verilog/tbench directory gives step by step instructions for running simulations using the UART Integration test vectors on the VERILOG source code.

Run time logs are available in the following files:

integration/verilog/Uart_rtl/Uart_rtl_LDV.log

integration/verilog/Uart_rtl/Uart_rtl_modelsim.log

integration/verilog/Uart_noscan_vhdl_net_min/Uart_noscan_vhdl_net_min_modelsim.log

integration/verilog/Uart_scanready_verilog_net_typ/Uart_scanready_verilog_net_typ_LDV.log

7.3 Integration Tests

7.3.1 Integration Testing of Intra-Chip and Primary Inputs

Please refer back to Section 5.23.1 on Page 62 as it covers the test logic and test sequences.

7.3.2 Integration Testing of Intra-Chip and Primary Outputs

Please refer back to Section 5.23.2 on Page 63 as it covers the test logic and test sequences.